JS

JAVASCRIPT · END-TO-END · SDE2 & SSE

JavaScript

> Event loop, closures, prototypes, promises, `this` – భాష లోపల ఏం జరుగుతోందో తెలిస్తే bugs ఊహించగలరు.

ఇది `OOPS_Telugu.pdf` కి జోడీ – అది objects మీద దృష్టి, ఇది భాష మొత్తం మీద. React / Node ఏదైనా – పునాది ఇదే.

తెలుగు + ENGLISH

YASWANTH · REFERENCE

విషయ సూచిక (Table of Contents)

Part 1 – పునాదులు (Foundations)

1. JavaScript అంటే ఏమిటి, ఎలా run అవుతుంది (ECMAScript, V8/JS engines, JIT, interpreted+compiled, browser/Node/Deno/Bun, JS≠Java)
2. Setup & Running JS (browser console, Node REPL, script tags, "use strict")
3. Variables – var, let, const (scope, hoisting, TDZ, redeclare/reassign)
4. Data Types – 7 primitives + object, typeof, value vs reference, null vs undefined
5. Type Coercion & Conversion (== vs ===, truthy/falsy, implicit/explicit, wat cases)
6. Operators (arithmetic, comparison, logical, nullish ??, optional chaining ?., spread/rest, ternary, bitwise, **)
7. Strings (immutability, methods, template literals, Unicode/code points)
8. Numbers & Math (IEEE-754, floating-point gotcha, BigInt, NaN, Infinity, Math)
9. Control Flow (if/else, switch, loops for/for-of/for-in/while/do-while, break/continue/labels)

Part 2 – Functions, Scope & Closures

10. Functions (declaration vs expression, arrow, default/rest params, arguments)
11. Scope & Scope Chain (global/function/block, lexical scope)
12. Hoisting deep (var/function/let/const, execution context phases, TDZ)
13. Closures (deep, private state, loop gotcha, memory, uses)
14. this keyword (default/implicit/explicit/new, arrow this, lost this)
15. call, apply, bind (borrowing, partial application, polyfills)
16. Higher-Order Functions & Callbacks
17. Functional programming (pure functions, immutability, currying, composition, IIFE, memoization)

Part 3 – Objects, Prototypes & Classes

18. Objects (literals, property descriptors, getters/setters, computed/shorthand)
19. Object utilities (keys/values/entries/assign/freeze/seal/create, spread, structuredClone)
20. Arrays (mutating vs non-mutating, map/filter/reduce deep, flat, sort gotchas)
21. Destructuring (array, object, nested, defaults, params)
22. Prototypes & Prototypal Inheritance (prototype chain, **proto** vs prototype, Object.create, shadowing)
23. Classes (ES6 constructor/methods/static/#private/getters-setters/inheritance/super/instanceof; note OOPS_Telugu.md for deeper OOP)
24. Map, Set, WeakMap, WeakSet (when vs object/array)
25. Symbols & Well-known Symbols (Symbol.iterator, uniqueness, metadata)
26. JSON (parse/stringify, replacer/reviver, deep clone, gotchas)

Part 4 – Asynchronous JavaScript

27. Event Loop deep (call stack, Web APIs, task vs microtask queue, rendering, starvation)
28. Callbacks & Callback Hell
29. Promises (states, then/catch/finally, chaining, Promise.all/allSettled/race/any)
30. async/await (try/catch, sequential vs parallel, top-level await, mistakes)
31. Generators & Iterators (function*, yield, custom iterables, async generators, for-await-of)

Part 5 – Advanced Language & Meta

32. ES Modules (import/export, dynamic import, CommonJS vs ESM, tree-shaking)
33. Error Handling (try/catch/finally, throw, Error types, custom errors, async errors)
34. Regular Expressions (flags, groups, lookahead/behind, methods)
35. Proxy & Reflect (meta-programming, traps, reactivity/validation)
36. Memory Management & Garbage Collection (reachability, mark-sweep, leaks, WeakRef/FinalizationRegistry)
37. Execution Context & Call Stack deep (creation/execution phase, variable environment)

Part 6 – Runtime Environments (Browser & Node)

38. The DOM (selection, traversal, manipulation, nodes, attributes/classes)
39. Events (capturing/bubbling, delegation, addEventListener, custom events, preventDefault)
40. Browser Web APIs (fetch, Storage/cookies, timers, observers, Web Workers)
41. Node.js essentials (runtime, modules, process, EventEmitter, streams, fs, non-blocking I/O)

Part 7 – Modern JS, Tooling & Quality

42. ES version-by-version features (ES6 → ES2024 highlights)
43. Tooling ecosystem (npm/package.json, bundlers Vite/webpack/esbuild, Babel, TypeScript intro)
44. Testing (unit testing, Jest/Vitest, mocking, TDD)
45. Performance (debounce/throttle, memoization, lazy loading, reflows, Big-O)
46. Security (XSS, CSRF, prototype pollution, CSP)

Part 8 – Patterns, Gotchas & Interview

47. Design Patterns in JS (module, singleton, factory, observer/pub-sub, strategy, decorator)
48. Tricky Parts & Gotchas (hoisting, this, closures-in-loops, coercion, floating point, NaN, async order)
49. Must-know Polyfills (bind, debounce, throttle, deepClone, Promise.all, curry, flatten, EventEmitter)
50. Interview Q&A (rapid-fire senior questions + crisp answers)
51. Memory Tips + Common Mistakes cheat-sheet

Part 1 – పునాదులు (Foundations)

> JavaScript ని నిజంగా అర్థం చేసుకోవాలంటే మొదట అది ఎలా *run* అవుతుంది అనేది తెలియాలి – engine, JIT, single-threaded event loop. ఈ Part లో language పుట్టుక, setup, variables, types, coercion, operators, strings, numbers, control flow వరకు గట్టి పునాది వేస్తాం. ఈ పునాది సరిగ్గా ఉంటేనే closures, `this`, event loop లాంటి తర్వాతి deep topics సులభం అవుతాయి.

---

1. JavaScript అంటే ఏమిటి, ఎలా run అవుతుంది

వివరణ

JavaScript (JS) అనేది ఒక **high-level, dynamically-typed, single-threaded, garbage-collected, multi-paradigm** (imperative + functional + object-oriented) programming language. దీన్ని **Brendan Eich** 1995లో Netscape లో కేవలం **10 రోజుల్లో** తయారు చేశాడు – మొదట `Mocha`, తర్వాత `LiveScript`, చివరికి marketing కారణంగా `JavaScript` అని పేరు పెట్టారు (అప్పుడు Java hot గా ఉండేది).

ఇక్కడే మొదటి పెద్ద confusion: **JavaScript ≠ Java.** ఈ రెండింటి మధ్య సంబంధం "car" మరియు "carpet" మధ్య ఉన్నంత మాత్రమే. పేరు marketing gimmick. Java = statically-typed, class-based, JVM మీద. JavaScript = dynamically-typed, prototype-based, engine మీద.

ECMAScript (ES) అనేది JavaScript యొక్క **official specification (standard).** అంటే – "JavaScript" అనేది ఆ standard ని అమలు చేసే language; **ECMA-262** అనే document లో ఆ rules ఉంటాయి, **TC39** అనే committee దాన్ని maintain చేస్తుంది. కొత్త versions: ES6 (2015) – పెద్ద మార్పు (let/const, classes, arrow, promises), తర్వాత ప్రతి సంవత్సరం ES2016, ES2017... ES2024.

> **గుర్తుంచుకో:** ECMAScript = *specification* (rulebook). JavaScript = ఆ rulebook ని follow అయ్యే *language/implementation.* V8, SpiderMonkey = ఆ language ని నిజంగా run చేసే *engines.*

Real-life Scenario

> **ECMAScript = క్రికెట్ నియమావళి (rulebook)** – ICC రాసిన rules. అది ఆడదు, కేవలం నియమాలు నిర్దేశిస్తుంది.
>
> **JavaScript = క్రికెట్ ఆట** – ఆ rules ప్రకారం ఆడే నిజమైన ఆట.
>
> **V8 / SpiderMonkey (engines) = నిజమైన ఆటగాళ్ళు** – ఆ ఆటని maidan లో అమలు చేసేవాళ్ళు. వేర్వేరు teams (Chrome, Firefox) వేర్వేరు ఆటగాళ్ళతో అదే rules ప్రకారం ఆడతాయి.

JS ఎలా run అవుతుంది – Engine, JIT, "interpreted + compiled"

చాలామంది "JavaScript is interpreted" అంటారు – ఇది **సగం నిజం మాత్రమే.** ఆధునిక JS engines **JIT (Just-In-Time) compilation** వాడతాయి – అంటే interpreted + compiled రెంటి కలయిక.

ఒక **JS engine** (ఉదా. Google V8 – Chrome & Node లో వాడతారు) లోపల ఏం జరుగుతుంది:

1. **Parser** – source code ని చదివి **tokens** గా విడగొట్టి, **AST (Abstract Syntax Tree)** అనే tree structure తయారు చేస్తుంది.
2. **Interpreter (V8 లో "Ignition")** – AST ని తీసుకుని **bytecode** గా మార్చి, వెంటనే execute చేయడం మొదలుపెడుతుంది. దీనివల్ల startup **వేగం** (compile కోసం ఆగనవసరం లేదు).
3. **Profiler / Monitor** – execute అవుతున్నప్పుడు, ఏ code మళ్ళీ మళ్ళీ run అవుతోందో (**"hot" code**) గమనిస్తుంది.
4. **Optimizing Compiler (V8 లో "TurboFan")** – ఆ hot code ని highly-optimized **native machine code** గా compile చేస్తుంది (assumptions ఆధారంగా – ఉదా. "ఈ variable ఎప్పుడూ number").
5. **Deoptimization** – ఆ assumption తప్పైతే (ఉదా. ఇప్పుడు అది string వచ్చింది), engine optimized code ని పడేసి, తిరిగి bytecode కి వెళ్తుంది.

ఇదే **"interpreted + compiled"** అనేదానికి అర్థం: మొదట interpret చేసి వేగంగా మొదలుపెడుతుంది, తర్వాత hot paths ని compile చేసి వేగం పెంచుతుంది. Best of both worlds.

లోపల ఏం జరుగుతుంది (Runtime = Engine + APIs + Event Loop)

ఒక్క engine మాత్రం JS ని పూర్తిగా run చేయలేదు. మనం వాడే `setTimeout`, `fetch`, `document`, `console` – ఇవేవీ ECMAScript spec లో లేవు! ఇవి **runtime environment** ఇచ్చే **APIs.**

```
┌─────────────────────────────────────────────┐
│         JS Runtime (Browser / Node)          │
│                                              │
│   ┌──────────────┐   ┌───────────────────┐   │
│   │  JS Engine   │   │  Runtime APIs     │   │
│   │  (V8)        │   │  (Web APIs / Node)│   │
│   │              │   │  setTimeout, fetch│   │
│   │ Call Stack   │   │  DOM, fs, http    │   │
│   │ Heap (memory)│   └───────────────────┘   │
│   └──────────────┘                           │
│         |                                    │
│   ┌─────────────────────────────────────┐    │
│   │  Event Loop + Task/Microtask Queues │    │  ← engine లో ఉండదు!
│   └─────────────────────────────────────┘    │
└─────────────────────────────────────────────┘
```

- **Engine** = Call Stack + Heap + JIT. కేవలం భాషని execute చేస్తుంది.
- **Runtime** = Engine + platform APIs + Event Loop. అసలు useful పని ఇది చేస్తుంది.

అందుకే ఒకే భాష (JS) వేర్వేరు చోట్ల వేర్వేరు capabilities ఇస్తుంది – browser లో `document`, Node లో `fs`.

వేర్వేరు JS Engines & Runtimes

Runtime	Engine	ఎక్కడ	ప్రత్యేకత
Chrome / Edge	V8	Browser	అత్యంత widely-used engine
Firefox	SpiderMonkey	Browser	మొదటి JS engine (Eich రాసింది)
Safari	JavaScriptCore	Browser	Apple, WebKit లో భాగం
Node.js	V8	Server / CLI	server-side JS ని popular చేసింది (2009)
Deno	V8	Server / CLI	Node creator (Ryan Dahl) రాసింది; secure-by-default, TS native
Bun	JavaScriptCore	Server / CLI	అత్యంత వేగం, all-in-one (runtime+bundler+test)

Node, Deno, Bun – మూడూ browser వెలుపల JS ని run చేయడానికి. Node = de-facto industry standard.

Code – same JS, వేర్వేరు runtimes

```
// ఈ pure JS ప్రతిచోటా పని చేస్తుంది (engine feature)
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6]

// కానీ ఇది runtime-specific:
// Browser లో మాత్రమే:
// document.querySelector("h1").textContent = "Hi";

// Node లో మాత్రమే:
// const fs = require("fs");
// fs.readFileSync("data.txt", "utf8");
```

JavaScript యొక్క ముఖ్య లక్షణాలు

- **Dynamically typed** – variable కి type declare చేయనవసరం లేదు; runtime లో type మారవచ్చు (`let x = 5; x = "hi";`).
- **Single-threaded** – ఒకే call stack, ఒక సమయంలో ఒకే పని. కానీ event loop వల్ల **non-blocking** async.
- **Prototype-based** – classes కూడా లోపల prototypes మీద ఆధారపడి ఉంటాయి (Topic 22, 23).
- **First-class functions** – functions ని variables లా pass చేయవచ్చు, return చేయవచ్చు (Topic 16).
- **Garbage-collected** – memory ని మనం manually free చేయనవసరం లేదు (Topic 36).

Gotchas (సాధారణ తప్పులు)

- **"JS is just interpreted, so it's slow"** – తప్పు. JIT వల్ల hot code native speed కి దగ్గరగా run అవుతుంది.
- **JavaScript = Java అనుకోవడం** – పూర్తిగా వేర్వేరు భాషలు.
- `setTimeout`**,** `fetch` **JS features అనుకోవడం** – కాదు, అవి runtime APIs. Spec లో లేవు.
- **"single-threaded కాబట్టి parallel పని చేయలేదు"** – పాక్షికంగా నిజం; kernel/runtime background threads (I/O), Web Workers వల్ల నిజమైన parallelism సాధ్యం.

Key Points

- JavaScript = ECMAScript spec ని అమలు చేసే dynamic, single-threaded language. TC39 committee, ECMA-262 standard.
- **JS ≠ Java** – పేరు marketing మాత్రమే.
- Engine (V8) = Parser → AST → Ignition (interpreter/bytecode) → TurboFan (JIT optimizer) → deopt.
- **JIT = interpreted (fast start) + compiled (hot paths native).**
- Runtime = Engine + platform APIs + Event Loop. Engine ఒక్కటే `setTimeout` / `fetch` / `DOM` ఇవ్వదు.
- Runtimes: Browser (V8/SpiderMonkey/JSC), Node (V8), Deno (V8), Bun (JSC).

Interview దృష్టి

Q: JavaScript interpreted భాషా, compiled భాషా? A: రెండూ. ఆధునిక engines JIT compilation వాడతాయి – మొదట interpreter (V8 లో Ignition) code ని bytecode గా interpret చేసి వేగంగా మొదలుపెడుతుంది, తర్వాత profiler "hot" code ని గుర్తించి optimizing compiler (TurboFan) దాన్ని native machine code గా compile చేస్తుంది. Assumptions తప్పైతే deoptimize అవుతుంది.

Q: ECMAScript కి, JavaScript కి తేడా? A: ECMAScript అనేది specification (ECMA-262 standard, TC39 maintain చేస్తుంది). JavaScript అనేది ఆ specification ని అమలు చేసే language. V8, SpiderMonkey అనేవి ఆ language ని run చేసే engines.

Q: JS single-threaded అయితే async ఎలా చేస్తుంది? A: Engine single call stack. కానీ runtime (browser/Node) background లో async operations (timers, network, I/O) handle చేసి, పూర్తయ్యాక callbacks ని queue లో పెడుతుంది. Event loop ఆ callbacks ని stack ఖాళీ అయినప్పుడు తీసుకొస్తుంది – దీనివల్ల thread block అవ్వకుండా concurrency వస్తుంది (Topic 27 లో deep).

Q: Node.js ఒక భాషా? A: కాదు. Node.js అనేది V8 engine ని browser వెలుపల run చేసే **runtime** – దానికి file system, network, OS access ఇచ్చే C++ APIs జోడించారు.

---

2. Setup & Running JS

వివరణ

JavaScript ని run చేయడానికి compiler install చేయనవసరం లేదు – నీ దగ్గర ఒక **browser** ఉంటే చాలు (అందులో engine built-in). కానీ serious development కి **Node.js** install చేసుకోవాలి. JS ని run చేయడానికి 4 ప్రధాన మార్గాలు ఉన్నాయి.

1. Browser Console (అత్యంత వేగవంతమైన మార్గం)

ఏ browser లోనైనా `F12` (లేదా Right-click → Inspect) → **Console** tab. ఇక్కడ నేరుగా JS type చేసి Enter నొక్కవచ్చు. చిన్న experiments కి perfect.

```
// Console లో నేరుగా:
2 + 2                    // 4
"hello".toUpperCase()    // "HELLO"
[1,2,3].map(x => x*x)    // [1, 4, 9]
```

2. `<script>` Tag (browser లో real projects)

HTML page లో JS ని embed చేయడానికి `<script>` tag వాడతాం. మూడు మార్గాలు:

```
<!DOCTYPE html>
<html>
<head>
  <!-- ❌ చెడు: head లో పెడితే DOM ఇంకా load అవ్వకముందే run అవుతుంది -->
  <!-- <script src="app.js"></script> -->
</head>
<body>
  <h1>Hello</h1>

  <!-- ✅ మంచి మార్గం 1: body చివర్లో (DOM ready అయ్యాక) -->
  <script src="app.js"></script>

  <!-- ✅ మంచి మార్గం 2: defer (head లోనే పెట్టవచ్చు, HTML parse అయ్యాక run) -->
  <!-- <script src="app.js" defer></script> -->

  <!-- inline script కూడా చెల్లుతుంది -->
  <script>
    console.log("inline JS running");
  </script>
</body>
</html>
```

`async` **vs** `defer` – script load చేసేటప్పుడు HTML parsing block అవ్వకుండా ఉండటానికి:

Attribute	Download	Execute ఎప్పుడు	Order guarantee
(ఏదీ లేదు)	HTML parsing ఆగుతుంది	వెంటనే (blocking)	Order maintain

Attribute	Download	Execute ఎప్పుడు	Order guarantee
`async`	parallel (parse ఆగదు)	download అయిన వెంటనే (parse మధ్యలోనే)	**Order guarantee లేదు**
`defer`	parallel (parse ఆగదు)	HTML పూర్తిగా parse అయ్యాక	**Order maintain**

> **గుర్తుంచుకో:** `defer` = "నేను download అవుతాను, కానీ HTML మొత్తం చదివాక మాత్రమే run అవుతాను, order తప్పకుండా." Analytics లాంటి independent scripts కి `async`, app logic కి `defer`.

3. Node.js – REPL & Files

Node install అయ్యాక terminal లో:

```
node                  # REPL (Read-Eval-Print-Loop) — browser console లాంటిది
> 2 + 2
4
> .exit               # బయటకి రావడానికి

node app.js           # ఒక file ని run చేయడానికి
```

REPL అంటే interactive shell – line by line type చేసి వెంటనే result చూడవచ్చు. Quick testing కి బాగుంటుంది.

4. Code – ఒక file

```
// app.js — node app.js తో run చేయవచ్చు
function greet(name) {
  return `నమస్తే, ${name}!`;
}
console.log(greet("Surya")); // నమస్తే, Surya!
```

`"use strict"` – Strict Mode

ES5 (2009) లో వచ్చిన feature. File లేదా function మొదట్లో `"use strict";` రాస్తే, JS **"sloppy mode"** యొక్క పాత తప్పులని errors గా మారుస్తుంది – silent bugs ని బయటపెడుతుంది.

```
"use strict"; // file మొదటి line లో ఉండాలి

x = 10; // ❌ ReferenceError! (strict లేకపోతే silent గా global x create అయ్యేది)

// strict mode లో ఏం మారుతుంది:
// 1. Undeclared variable కి assign = error (accidental globals ఆగుతాయి)
// 2. Read-only property కి write = error (silent fail కాదు)
// 3. Duplicate parameter names = error → function f(a, a) {}
// 4. this in normal function call = undefined (window/global కాదు)
// 5. delete of variable/function = error
// 6. Reserved keywords (implements, interface, private...) వాడలేం
```

ముఖ్యం: ES **Modules** (`import` / `export`) మరియు ES6 **classes** లోపల **ఎప్పుడూ automatic strict mode** – `"use strict"` రాయనవసరం లేదు. అందుకే modern code లో ఇది చాలా అరుదుగా కనిపిస్తుంది; కానీ దాని behavior మనకి ఎప్పుడూ ఉంటుంది.

Gotchas (సాధారణ తప్పులు)

- **Script ని `<head>` లో `defer` లేకుండా పెట్టడం** – DOM ఇంకా లేనప్పుడే `document.querySelector` run అయ్యి `null` వస్తుంది.
- **`"use strict"` ని file మధ్యలో రాయడం** – పని చేయదు; ఇది మొదటి statement గా ఉండాలి.
- **`async` scripts order మీద ఆధారపడటం** – `async` execution order guarantee ఇవ్వదు; dependency ఉన్న scripts కి `defer` వాడు.
- **REPL లో `const` redeclare** – పాత Node REPL లో error వచ్చేది (session persists); files లో scope-dependent.

Key Points

- Run చేయడానికి 4 మార్గాలు: Browser console, `<script>` tag, Node REPL, `node file.js`.
- `<script>`: `defer` = parse ఆగదు + HTML తర్వాత + order maintain (ఉత్తమం). `async` = order లేదు (independent scripts).
- **Node REPL** = interactive, `node file.js` = script run.
- `"use strict"` = పాత silent bugs ని errors చేస్తుంది; **modules & classes లో default.**

Interview దృష్టి

Q: `async` vs `defer` తేడా? A: రెండూ HTML parsing ని block చేయకుండా script ని parallel download చేస్తాయి. తేడా execution: `async` – download అయిన వెంటనే run (parse మధ్యలోనైనా), order guarantee లేదు. `defer` – HTML పూర్తిగా parse అయ్యాక, DOM ready అయిన తర్వాత, declared order లో run. Dependency ఉన్న app scripts కి `defer` వాడాలి.

Q: Strict mode ఎందుకు, ఏం మారుస్తుంది? A: పాత JS యొక్క forgiving "sloppy" behavior వల్ల వచ్చే silent bugs ని catch చేయడానికి. Undeclared variable assign, read-only write, duplicate params, `this` being global – ఇవన్నీ errors అవుతాయి. ES modules & classes లో ఇది automatic.

---

3. Variables – var, let, const

వివరణ

JavaScript లో variable declare చేయడానికి 3 keywords: `var` (పాతది, ES1), `let` మరియు `const` (రెండూ ES6/2015). ఇవి 4 ముఖ్యమైన విషయాల్లో తేడా చూపిస్తాయి: **scope, hoisting behavior, redeclaration, reassignment.** ఈ తేడాలు అర్థం కాకపోతే bugs తప్పవు.

సూత్రం (modern best practice): డిఫాల్ట్ గా `const` వాడు. Value మార్చాల్సి వస్తేనే `let`. `var` అస్సలు వాడకు.

Real-life Scenario

> ఊహించుకో – ఇది ఒక **hostel room కేటాయింపు.**
>
> - `var` **= పాత hostel** – ఏ గది అయినా మొత్తం floor (function) కి కనిపిస్తుంది. ఒకే గదిని రెండుసార్లు కేటాయించవచ్చు (redeclare). Warden రాకముందే (declaration ముందే) "గది ఉంది" అని board మీద రాసేస్తారు (hoisting).
> - `let` **= కొత్త hostel** – గది కేవలం ఆ `{}` block కి మాత్రమే. ఒకసారి కేటాయించిన గదిని మళ్ళీ కేటాయించలేరు (no redeclare), కానీ లోపల వేరే వ్యక్తిని పెట్టవచ్చు (reassign).
> - `const` **= permanent allotment** – గది ఒక్కరికే, మార్చలేరు (no reassign). కానీ ఆ వ్యక్తి తన గదిలో furniture (object properties) మార్చుకోవచ్చు!

పోలిక Table

అంశం	`var`	`let`	`const`
Scope	Function	Block `{}`	Block `{}`
Hoisting	✅ (`undefined` గా init)	✅ (కానీ TDZ)	✅ (కానీ TDZ)
Redeclare (అదే scope)	✅ అనుమతి	❌ SyntaxError	❌ SyntaxError
Reassign	✅	✅	❌ TypeError
Global లో `window` కి attach	✅ అవుతుంది	❌ కాదు	❌ కాదు
Init లేకుండా declare	✅	✅	❌ SyntaxError (value తప్పనిసరి)

Code – Scope తేడా

```
// ── var = function-scoped (block ని పట్టించుకోదు) ──
function testVar() {
  if (true) {
    var x = 10;
  }
  console.log(x); // 10 ✅ — block దాటి బయటకి leak అయింది!
}

// ── let/const = block-scoped ──
function testLet() {
  if (true) {
    let y = 20;
    const z = 30;
  }
  console.log(y); // ❌ ReferenceError: y is not defined
}

// ── Classic loop gotcha ──
```

```
for (var i = 0; i < 3; i++) { /* ... */ }
console.log(i); // 3 — var leak అయ్యి loop బయట కూడా ఉంది!

for (let j = 0; j < 3; j++) { /* ... */ }
console.log(j); // ❌ ReferenceError — let loop కి పరిమితం (correct)
```

Code – Redeclare & Reassign

```
var a = 1;
var a = 2;       // ✅ OK (var redeclare అనుమతిస్తుంది — bug-prone!)

let b = 1;
// let b = 2;    // ❌ SyntaxError: Identifier 'b' has already been declared
b = 2;           // ✅ reassign OK

const c = 1;
// c = 2;        // ❌ TypeError: Assignment to constant variable

// ⚠️ const అంటే variable "binding" మారదు, value "content" కాదు!
const user = { name: "Surya" };
user.name = "Palsingh"; // ✅ OK! object లోపల మార్చవచ్చు
user.age = 30;          // ✅ OK! property add చేయవచ్చు
// user = {};           // ❌ TypeError — కొత్త object assign చేయలేం

const arr = [1, 2];
arr.push(3);            // ✅ [1, 2, 3] — mutate OK
// arr = [9];           // ❌ reassign కాదు
```

లోపల ఏం జరుగుతుంది – TDZ (Temporal Dead Zone)

చాలామంది "let/const hoist అవ్వవు" అంటారు – ఇది **తప్పు.** అవి **hoist అవుతాయి,** కానీ `var` లా `undefined` తో initialize అవ్వవు. Declaration line వచ్చేవరకు అవి **"Temporal Dead Zone" (TDZ)** అనే స్థితిలో ఉంటాయి – access చేస్తే `ReferenceError`.

```
console.log(myVar); // undefined ✅ (var hoist + undefined init)
console.log(myLet); // ❌ ReferenceError — TDZ లో ఉంది!

var myVar = 1;
let myLet = 2;
```

TDZ = "variable ఉంది అని engine కి తెలుసు (hoist అయింది), కానీ దాన్ని ముట్టుకోవడం నిషిద్ధం – declaration line దాటేవరకు." Scope మొదలైన క్షణం నుండి declaration line వరకు ఉన్న ఆ ప్రాంతమే TDZ.

TDZ ఎందుకు మంచిది? ఇది programming తప్పుని (declare చేయకముందే వాడటం) silent `undefined` కాకుండా, స్పష్టమైన error గా చూపిస్తుంది. `var` యొక్క `undefined` bug ని పరిష్కరిస్తుంది.

```
// TDZ యొక్క tricky example:
let temp = "outer";
{
  // ఇక్కడ నుండి TDZ మొదలు (block scope లో temp hoist అయింది)
  console.log(temp); // ❌ ReferenceError! (outer temp కాదు, inner TDZ)
```

```
  let temp = "inner";
}
```

Gotchas (సాధారణ తప్పులు)

- **`const` = immutable value అనుకోవడం.** కాదు! Binding మాత్రమే constant. Object/array content మారవచ్చు. నిజంగా freeze చేయాలంటే `Object.freeze()` (Topic 19).
- **TDZ ని మర్చిపోవడం** – "let hoist అవ్వదు" అని తప్పుగా చెప్పడం. Hoist అవుతుంది, కానీ TDZ వల్ల access కుదరదు.
- **`var` global scope లో `window` కి attach అవ్వడం** – `var x = 1` browser లో `window.x = 1` అవుతుంది, `let` / `const` కాదు.
- **Loop లో `var` వాడి closure bug** – `setTimeout` లో అన్నీ చివరి value print చేస్తాయి (Topic 13 లో deep).

Key Points

- **Default `const` → అవసరమైతే `let` → `var` never.**
- `var` = function-scoped + hoisted (undefined) + redeclarable + `window` కి attach.
- `let` / `const` = block-scoped + hoisted-but-TDZ + no redeclare.
- `const` = binding constant, **content కాదు** (object mutate చేయవచ్చు).
- **TDZ** = scope start నుండి declaration వరకు; access = ReferenceError.

Interview దృష్టి

Q: `var`, `let`, `const` మధ్య తేడాలు? A: (1) Scope: `var` function-scoped, `let` / `const` block-scoped. (2) Hoisting: మూడూ hoist అవుతాయి, కానీ `var` `undefined` గా init అవుతుంది, `let` / `const` TDZ లో ఉంటాయి. (3) Redeclare: `var` అనుమతిస్తుంది, మిగతా రెండూ కాదు. (4) Reassign: `var` / `let` OK, `const` కాదు. (5) `var` global scope లో `window` కి attach అవుతుంది.

Q: TDZ అంటే ఏమిటి? A: Temporal Dead Zone – `let` / `const` variable యొక్క scope మొదలైన క్షణం నుండి, దాని declaration line వరకు ఉన్న ప్రాంతం. ఈ zone లో variable hoist అయ్యి ఉంటుంది కానీ access చేస్తే `ReferenceError`. ఇది "declare చేయకముందే వాడటం" అనే bug ని `undefined` కాకుండా స్పష్టమైన error గా చూపిస్తుంది.

Q: `const` object ని modify చేయవచ్చా? A: అవును. `const` variable binding ని మాత్రమే lock చేస్తుంది – reference మార్చలేం. కానీ ఆ reference చూపించే object/array యొక్క properties/elements మార్చవచ్చు. పూర్తిగా immutable చేయాలంటే `Object.freeze()`.

---

4. Data Types

వివరణ

JavaScript లో types రెండు categories: **Primitives (7)** మరియు **Objects (1 category).** Primitives = simple, immutable values, **by value** copy అవుతాయి. Objects = collections, **by reference** copy

అవుతాయి. ఈ "value vs reference" తేడా JS లో అత్యంత ముఖ్యమైన మరియు bugs కి అత్యంత కారణమయ్యే concept.

7 Primitive Types

Type	ఉదాహరణ	వివరణ
string	`"hi"`, `` `t${x}` ``	text; immutable
number	`42`, `3.14`, `NaN`	IEEE-754 double (integer & float రెంటికీ ఒకటే)
boolean	`true`, `false`	logical
undefined	`undefined`	value ఇంకా assign అవ్వలేదు (default)
null	`null`	"ఏమీ లేదు" అని **ఉద్దేశపూర్వకంగా** పెట్టింది
bigint	`123n`	number safe limit దాటిన పెద్ద integers (ES2020)
symbol	`Symbol("id")`	unique identifier (Topic 25)

మిగతా అన్నీ – **objects:** `{}`, `[]` (array), `function`, `Date`, `RegExp`, `Map`, `Set` – అన్నీ object category లోనివే.

Real-life Scenario – Value vs Reference

> **Primitive (by value) = photocopy.** నీ document ని photocopy ఇచ్చాను. నువ్వు దాని మీద రాసుకున్నా, నా original మారదు. రెండూ వేర్వేరు కాగితాలు.
>
> **Object (by reference) = Google Doc link.** నేను నీకు నా doc **link** ఇచ్చాను. నువ్వు edit చేస్తే, నా doc లోనూ మారుతుంది – ఎందుకంటే ఇద్దరం **ఒకే** doc చూస్తున్నాం, రెండు copies కాదు.

Code – Value vs Reference

```
// ── Primitives: by value (copy) ──
let a = 10;
let b = a;    // b కి a యొక్క value (10) copy అయింది
b = 20;
console.log(a); // 10 ✅ – a మారలేదు (వేర్వేరు)

// ── Objects: by reference (shared) ──
let obj1 = { count: 1 };
let obj2 = obj1;      // reference copy – ఇద్దరూ ఒకే object చూస్తున్నారు!
obj2.count = 99;
console.log(obj1.count); // 99 ❗ – obj1 కూడా మారింది!

// ── Function arguments కూడా అలాగే ──
function mutate(o) { o.x = 100; }
const data = { x: 1 };
mutate(data);
console.log(data.x); // 100 – object pass by reference
```

```
function tryChange(n) { n = 500; }
let num = 5;
tryChange(num);
console.log(num); // 5 — primitive pass by value
```

> **JS లో అంతా "pass by value"** అని కచ్చితంగా చెప్పాలంటే – objects కి ఆ "value" అనేది **reference (address).** అందుకే reference ని copy చేసి pass చేస్తుంది, కానీ అది అదే object ని చూపిస్తుంది.

Equality – reference పోలిక

```
console.log({} === {});          // false — వేర్వేరు objects (వేర్వేరు addresses)
console.log([1] === [1]);        // false — content same అయినా reference వేరు
const x = { v: 1 };
console.log(x === x);            // true — అదే reference

console.log("hi" === "hi");      // true — primitives value తో పోలిక
```

`typeof` operator

```
typeof "hello"      // "string"
typeof 42           // "number"
typeof true         // "boolean"
typeof undefined    // "undefined"
typeof 10n          // "bigint"
typeof Symbol()     // "symbol"
typeof {}           // "object"
typeof [1,2]        // "object"  ⚠️ array కి కూడా "object"!
typeof function(){} // "function" ⚠️ ప్రత్యేకంగా!

typeof null         // "object"  ❗❗ చారిత్రక BUG! (కింద చూడు)
```

null vs undefined

రెండూ "ఏమీ లేదు" అని చెప్తాయి, కానీ ఉద్దేశం వేరు:

అంశం	`undefined`	`null`
అర్థం	Value **ఇంకా set అవ్వలేదు** (system default)	**ఉద్దేశపూర్వకంగా** "ఏమీ లేదు" (developer set)
ఎవరు set చేస్తారు	JS engine (automatic)	Developer (manual)
`typeof`	`"undefined"`	`"object"` (bug)
ఎప్పుడు వస్తుంది	declared కానీ unassigned var; missing param; no return; missing property	మనం explicitly `= null` పెట్టినప్పుడు

```
let x;                    // undefined (assign అవ్వలేదు)
let y = null;             // null (మనం పెట్టాం)

console.log(null == undefined);  // true  (loose — రెండూ "empty")
console.log(null === undefined); // false (strict — types వేరు)

console.log(typeof null);        // "object" — 1995 నుండి fix చేయని bug!
```

లోపల ఏం జరుగుతుంది — `typeof null === "object"` bug

ఇది JS యొక్క అత్యంత ప్రసిద్ధ **historical bug.** మొదటి JS implementation లో, values ని type tag తో store చేసేవారు. Objects యొక్క tag `0` (binary `000`). `null` ని machine code లో `NULL pointer` (అంటే all-zeros) గా represent చేశారు — దీని tag కూడా `000`. అందుకే `typeof null` object tag చూసి `"object"` return చేస్తుంది. దీన్ని fix చేస్తే పాత websites పాడవుతాయని ఇప్పటికీ ఉంచేశారు (backward compatibility).

Gotchas (సాధారణ తప్పులు)

- `typeof null === "object"` — null object కాదు, primitive. ఇది bug. Null check కి `x === null` వాడు.
- `typeof array === "object"` — array కోసం `Array.isArray(x)` వాడు, `typeof` కాదు.
- `typeof NaN === "number"` — NaN ("Not-a-Number") అయినా number type! (Topic 8).
- **Object copy అనుకుని reference share చేయడం** — `obj2 = obj1` copy కాదు; shallow copy కి `{...obj1}` వాడు.
- `==` **తో null/undefined** — `null == undefined` true, కానీ `null == 0` false. Confusing; `===` వాడు.

Key Points

- **7 primitives:** string, number, boolean, undefined, null, bigint, symbol. మిగతా అన్నీ objects.
- **Primitives = by value** (copy), **Objects = by reference** (shared).
- `{} === {}` → false (వేర్వేరు references).
- `typeof` bugs: `null` → `"object"`, array → `"object"`, function → `"function"`.
- **undefined = system default** (unassigned), **null = intentional empty** (developer).

Interview దృష్టి

Q: Primitive vs reference types తేడా? A: Primitives (string, number, boolean, null, undefined, bigint, symbol) by value copy అవుతాయి — copy చేస్తే స్వతంత్ర value. Objects (arrays, functions అన్నీ కలిపి) by reference — variable అనేది memory address ని పట్టుకుంటుంది; copy చేస్తే అదే object ని ఇద్దరూ share చేస్తారు, ఒకరు మార్చితే ఇద్దరికీ కనిపిస్తుంది.

Q: null vs undefined? A: `undefined` = value ఇంకా assign అవ్వలేదు (JS engine default — unassigned variable, missing return, missing param). `null` = developer ఉద్దేశపూర్వకంగా "ఏమీ లేదు" అని పెట్టింది. `null == undefined` true కానీ `null === undefined` false. చిక్కు: `typeof null === "object"` (historical bug), `typeof undefined === "undefined"`.

Q: `typeof null` ఎందుకు `"object"`? A: మొదటి JS implementation లో type ని tag bits తో store చేసేవారు; objects tag `000`, null కూడా NULL pointer (all zeros = `000`). అందుకే typeof దాన్ని object అనుకుంటుంది. Backward compatibility కోసం fix చేయలేదు.

---

5. Type Coercion & Conversion

TYPE COERCION · == యొక్క ఉచ్చులు

== ఎందుకు ప్రమాదకరం – coercion నియమాలు

`0 == "0"`	true	string → number
`0 == []`	true	[] → "" → 0
`"0" == []`	false	"0" vs "" – రెండూ strings
`null == undefined`	true	ప్రత్యేక నియమం
`null == 0`	false	null ఎప్పుడూ number కి మారదు
`NaN == NaN`	false	NaN తనకి తానే సమానం కాదు

నియమం ఒక్కటే

ఎప్పుడూ **===** వాడండి. == యొక్క నియమాలు ఎవరూ పూర్తిగా గుర్తుంచుకోలేరు.
ఒకే మినహాయింపు:

`x == null` – ఇది null మరియు undefined రెండింటినీ పట్టుకుంటుంది.NaN ని check చేయడానికి `Number.isNaN()` – == ఎప్పటికీ పని చేయదు.

వివరణ

Coercion అంటే JavaScript ఒక type ని automatic గా (implicit) మరో type కి మార్చడం. JS dynamically-typed కాబట్టి, `"5" + 3` లాంటివి చూస్తే engine ఏదో ఒక type కి convert చేసి పని కానిస్తుంది. ఇది convenient కానీ **JS లో అత్యంత confusing, bug-prone భాగం** – famous "wat" jokes అన్నీ దీనివల్లే.

- **Explicit conversion (type casting):** మనం స్వయంగా మారుస్తాం – `Number("5")`, `String(42)`, `Boolean(0)`.
- **Implicit coercion:** JS మనకి చెప్పకుండా మారుస్తుంది – `"5" * 2`, `1 + "1"`, `if ("hello")`.

Real-life Scenario

> **Coercion = ఒక అతి-ఉత్సాహపు translator (అనువాదకుడు).** నువ్వు తెలుగులో "5" అని, ఇంకొకరు English లో "three" అని మాట్లాడితే, ఇతను ఇద్దరినీ ఒకే భాషకి మార్చి "అర్థం అయ్యేలా" చేస్తాడు. కానీ కొన్నిసార్లు అతని అనువాదం **తప్పు** – `[] + []` ని "" అని, `[] + {}` ని "[object Object]" అని translate చేస్తాడు! అందుకే అతని translation ని గుడ్డిగా నమ్మకూడదు (`===` వాడాలి).

Truthy & Falsy

`if`, `&&`, `||`, `!` లాంటి boolean context లో values automatic గా boolean కి coerce అవుతాయి. **8 falsy values మాత్రమే ఉన్నాయి – మిగతా అన్నీ truthy.**

Falsy (8): `false`, `0`, `-0`, `0n` (BigInt zero), `""` (empty string), `null`, `undefined`, `NaN`.

```
// ఈ కింది అన్నీ truthy – ఆశ్చర్యకరమైనవి:
if ("0")        console.log("truthy!"); // ✅ non-empty string
if ("false")    console.log("truthy!"); // ✅ non-empty string
if ([])         console.log("truthy!"); // ✅ empty array కూడా truthy!
if ({})         console.log("truthy!"); // ✅ empty object కూడా truthy!
if (function(){}) console.log("truthy!"); // ✅

// అన్నీ print అవుతాయి!
```

> **గుర్తుంచుకో:** `[]` (empty array) మరియు `{}` (empty object) **truthy.** కానీ coercion లో `[] == false` → **true** (కింద wat cases చూడు)! ఇదే JS trap.

`==` vs `===`

- `===` **(strict equality)** – type convert చేయకుండా, type + value రెండూ same అయితేనే `true`. **ఎప్పుడూ దీన్నే వాడు.**
- `==` **(loose equality)** – types వేరైతే coerce చేసి పోల్చుతుంది. Rules గజిబిజి.

```
5 === "5"     // false – types వేరు (number vs string)
5 == "5"      // true  – "5" ని 5 కి coerce చేసింది

0 == false    // true  – false → 0
"" == false   // true  – రెండూ 0 కి
null == undefined  // true  – ప్రత్యేక rule (ఇద్దరే ఒకరికొకరు equal)
null == 0     // false – null 0 కి coerce అవ్వదు!
NaN === NaN   // false – NaN దేనితోనూ equal కాదు (తనతో కూడా!)
```

`==` coercion rules (సరళంగా):

1. types same → `===` లా.
2. `null == undefined` → true (ఇంకేదానితోనూ కాదు).
3. number vs string → string ని number కి.
4. boolean ఉంటే → boolean ని number కి (`true`→1, `false`→0).
5. object vs primitive → object ని primitive కి (`valueOf` / `toString`).

`+` operator – string concatenation vs addition

`+` కి రెండు పనులు: **addition** (numbers) మరియు **concatenation** (strings). ఏదైనా operand string అయితే, `+` **concatenation** చేస్తుంది – number ని string కి మారుస్తుంది.

```
1 + 2       // 3    (number addition)
"1" + 2     // "12" (2 → "2", concatenate)
```

```
1 + "2"      // "12"
1 + 2 + "3" // "33" — ఎడమ నుండి: (1+2)=3, తర్వాత 3+"3"="33"
"1" + 2 + 3 // "123" — "1"+2="12", "12"+3="123"

// మిగతా arithmetic operators (- * / %) ఎప్పుడూ number కి:
"5" - 2      // 3   (string → number)
"5" * "2"    // 10
"abc" - 1    // NaN (convert కాలేదు)
"5" - "2"    // 3
```

Trick: `+` ఒక్కటే string వైపు మొగ్గు చూపిస్తుంది. `- * / %` అన్నీ number వైపు. అందుకే `"5" + 1 = "51"` కానీ `"5" - 1 = 4`.

Explicit Conversion (సరైన మార్గం)

```
// String కి:
String(123)      // "123"
String(null)     // "null"
(123).toString() // "123"

// Number కి:
Number("123")    // 123
Number("")       // 0   ⚠️
Number("12px")   // NaN  (whole string number కావాలి)
Number(null)     // 0   ⚠️
Number(undefined)// NaN
parseInt("12px") // 12   (partial parse — leading number తీస్తుంది)
parseFloat("3.14abc") // 3.14
+"42"            // 42   (unary + = quick Number())

// Boolean కి:
Boolean(0)       // false
Boolean("")      // false
Boolean("hi")    // true
!!"hi"           // true  (double-NOT = quick Boolean())
!!0              // false
```

లోపల ఏం జరుగుతుంది — famous "wat" cases

```
[] + []         // ""              — రెండు arrays "" గా, "" + "" = ""
[] + {}         // "[object Object]" — [] → "", {} → "[object Object]"
{} + []         // 0 (కొన్ని చోట్ల) — {} ని empty block అనుకుంటుంది, +[] = 0
[] == ![]       // true !!         — ![] = false → 0; [] → "" → 0; 0==0 true
true + true     // 2               — true → 1, 1+1
"5" - - "2"     // 7               — -"2" = -2, "5" - (-2) = 7
0.1 + 0.2       // 0.30000000000000004 — floating point (Topic 8)
null + 1        // 1               — null → 0
undefined + 1   // NaN             — undefined → NaN
"" == 0         // true            — "" → 0
"  " == 0       // true            — whitespace string → 0
```

`[] == ![]` **ఎందుకు** `true` **?** ఇది నిజంగా mind-bending: `![]` first evaluate అవుతుంది → `[]` truthy కాబట్టి `![]` = `false`. ఇప్పుడు `[] == false`. `==` boolean ని number కి: `false` → `0`. తర్వాత `[]` ని primitive కి: `[].toString()` = `""`, `""` → `0`. చివరికి `0 == 0` → **true.**

Gotchas (సాధారణ తప్పులు)

- `==` **వాడటం** – దాదాపు ఎప్పుడూ `===` వాడాలి. Exception: `x == null` (null మరియు undefined రెంటినీ ఒకేసారి check చేయడానికి shortcut).
- `Number("")` → `0` అని మర్చిపోవడం (`NaN` కాదు). Empty input validation లో bug.
- `[] == false` **true కానీ** `if([])` **truthy** – coercion context బట్టి వేర్వేరు rules!
- `+` **తో numbers add చేయాలనుకుని strings concatenate అవ్వడం** – form inputs strings కాబట్టి `Number()` చేయకపోతే `"5"+"3" = "53"`.
- `parseInt` **లో radix మర్చిపోవడం** – `parseInt("08")` పాత engines లో problematic; ఎప్పుడూ `parseInt("08", 10)`.

Key Points

- **8 falsy values:** `false, 0, -0, 0n, "", null, undefined, NaN`. మిగతా అన్నీ truthy (`[]`, `{}` కూడా!).
- `===` **always** (type + value). `==` coerces – avoid (except `== null`).
- `+` : ఏదైనా string ఉంటే concatenate; `- * / %` ఎప్పుడూ number.
- Explicit: `Number()`, `String()`, `Boolean()`, `+x`, `!!x`, `parseInt/parseFloat`.
- `NaN === NaN` → **false** (దేనితోనూ equal కాదు; check కి `Number.isNaN`).

Interview దృష్టి

Q: `==` **vs** `===` **?** A: `===` strict – type convert చేయదు, type మరియు value రెండూ same అయితేనే true. `==` loose – types వేరైతే coerce చేసి పోల్చుతుంది (`5 == "5"` true). Coercion rules గజిబిజి & bug-prone; production లో ఎప్పుడూ `===`. మినహాయింపు: `x == null` – null మరియు undefined రెంటినీ ఒకేసారి catch చేస్తుంది.

Q: JS లో falsy values ఏవి? A: 8 – `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. మిగతా అన్నీ truthy, **empty array** `[]` **మరియు empty object** `{}` **కూడా truthy.**

Q: `"5" + 3` **మరియు** `"5" - 3` **results?** A: `"5" + 3 = "53"` – `+` కి ఒక operand string అయితే concatenation చేస్తుంది. `"5" - 3 = 2` – `-` string ని number కి coerce చేసి subtract చేస్తుంది. `+` ఒక్కటే dual-purpose (add/concat).

Q: `0.1 + 0.2 === 0.3` **?** A: `false`. IEEE-754 floating-point representation వల్ల `0.1 + 0.2 = 0.30000000000000004`. Fix: `Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON` (Topic 8).

---

6. Operators

వివరణ

Operators = values మీద operations చేసే symbols. చాలావరకు straightforward, కానీ కొన్ని modern operators (`??`, `?.`, `...`) మరియు logical operators యొక్క **short-circuit** behavior senior-level lo ముఖ్యం. వాటిమీద focus చేద్దాం.

Arithmetic Operators

```
5 + 2    // 7
5 - 2    // 3
5 * 2    // 10
5 / 2    // 2.5  (JS integer division ఇవ్వదు — ఎప్పుడూ float)
5 % 2    // 1    (remainder / modulo)
5 ** 2   // 25   (exponentiation, ES2016) — Math.pow(5,2) కి బదులు
2 ** 3 ** 2 // 512 — right-associative! (2 ** (3**2) = 2**9)

let x = 5;
x++      // post-increment: వాడాక పెంచుతుంది
++x      // pre-increment: పెంచాక వాడుతుంది
```

Logical Operators & Short-Circuit (ముఖ్యం!)

`&&`, `||` కేవలం boolean return చేయవు – అవి **operands లో ఒకదాన్ని** return చేస్తాయి, మరియు **short-circuit** అవుతాయి (అవసరం లేకపోతే కుడివైపు evaluate చేయవు).

```
// && — ఎడమది falsy అయితే దాన్నే return (కుడివైపు చూడదు); లేదంటే కుడిది
true && "hi"     // "hi"   (ఎడమ truthy → కుడి return)
0 && "hi"        // 0      (ఎడమ falsy → ఎడమ return, "hi" evaluate కాదు)
"a" && "b"       // "b"

// || — ఎడమది truthy అయితే దాన్నే return; లేదంటే కుడిది
"hi" || "bye"    // "hi"
"" || "default" // "default"  (fallback pattern!)
0 || 100         // 100

// ఆచరణలో:
const name = userInput || "Guest";     // userInput ఖాళీ అయితే "Guest"
user.isAdmin && showAdminPanel();      // isAdmin true అయితేనే call

// ⚠️ || యొక్క సమస్య: 0, "" లాంటి valid falsy values ని కూడా తోసేస్తుంది
const count = userCount || 10; // userCount = 0 అయితే → 10 (తప్పు!)
```

Nullish Coalescing `??` (ES2020) – `||` కి పరిష్కారం

`??` కేవలం `null` లేదా `undefined` అయితేనే కుడివైపు fallback ఇస్తుంది – `0`, `""`, `false` లాంటి valid falsy values ని గౌరవిస్తుంది.

```
const count = userCount ?? 10;  // userCount = 0 → 0 (సరైనది!)
0 ?? "fb"         // 0     (0 null/undefined కాదు)
"" ?? "fb"        // ""
```

```
null ?? "fb"     // "fb"
undefined ?? "fb"// "fb"
```

Expression	`userVal = 0`	`userVal = null`	`userVal = ""`
`userVal || "def"`	`"def"` ❌	`"def"` ✅	`"def"` ❌
`userVal ?? "def"`	`0` ✅	`"def"` ✅	`""` ✅

> **నియమం:** "value లేకపోతే" (null/undefined) fallback కావాలంటే `??`. "falsy అయితే" fallback కావాలంటే `||`. Config defaults, `0` / `""` valid ఉన్న చోట ఎప్పుడూ `??`.

Optional Chaining `?.` (ES2020)

Nested property access లో మధ్యలో `null` / `undefined` వస్తే error రాకుండా `undefined` return చేస్తుంది.

```
const user = { profile: { name: "Surya" } };

user.address.city           // ❌ TypeError: Cannot read 'city' of undefined
user.address?.city          // ✅ undefined (crash కాదు)
user.profile?.name          // "Surya"

user.getName?.()            // method ఉంటే call, లేకపోతే undefined (crash కాదు)
user.items?.[0]             // array access కూడా safe

// ?. తో ?? కలిపి — perfect combo:
const city = user.address?.city ?? "Unknown"; // "Unknown"
```

> **గుర్తుంచుకో:** `?.` = "ఉంటే ముందుకు వెళ్ళు, లేకపోతే `undefined` తో ఆగిపో — error వేయకు." కానీ **అతిగా వాడకు** — నిజంగా optional అయిన చోట మాత్రమే; లేకపోతే bugs దాక్కుంటాయి.

Spread `...` & Rest `...` (ఒకే symbol, రెండు పనులు)

```
// ── Spread: విడగొట్టడం (expand) ──
const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4];        // [1, 2, 3, 4]
const combined = [...arr1, ...arr2]; // merge
const copy = [...arr1];              // shallow copy

const obj1 = { a: 1 };
const obj2 = { ...obj1, b: 2 };      // { a: 1, b: 2 }
Math.max(...[3, 1, 4]);              // 4 (array → args)

// ── Rest: సేకరించడం (collect) ──
function sum(...nums) {              // అన్ని args ని array గా
  return nums.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3, 4);                     // 10
```

```
const [first, ...others] = [1, 2, 3]; // first=1, others=[2,3]
const { a, ...rest } = { a: 1, b: 2, c: 3 }; // a=1, rest={b:2,c:3}
```

తేడా: Spread = "విప్పు" (function call / literal లో). Rest = "కట్టు" (function params / destructuring left లో). Position చూసి తెలుసుకో.

Ternary Operator

```
const age = 20;
const type = age >= 18 ? "adult" : "minor"; // condition ? ifTrue : ifFalse

// nested (readability కోసం జాగ్రత్త):
const grade = score >= 90 ? "A" : score >= 80 ? "B" : "C";
```

Bitwise Operators

Numbers ని 32-bit integers గా చూసి bit-level operations. అరుదుగా వాడతారు, కానీ flags, low-level tricks కి.

```
5 & 3     // 1   (AND:  0101 & 0011 = 0001)
5 | 3     // 7   (OR:   0101 | 0011 = 0111)
5 ^ 3     // 6   (XOR:  0101 ^ 0011 = 0110)
~5        // -6  (NOT:  ~n = -(n+1))
5 << 1    // 10  (left shift = ×2)
5 >> 1    // 2   (right shift = ÷2 floor)
-1 >>> 0 // 4294967295 (unsigned right shift)

// Common tricks:
~~3.7     // 3  (double NOT = fast Math.trunc for 32-bit)
n & 1     // odd/even check (1 = odd)
```

Gotchas (సాధారణ తప్పులు)

- **`||` ని `0` / `""` valid ఉన్న defaults కి వాడటం** — `count || 10` bug; `count ?? 10` వాడు.
- **`?.` అతిగా వాడటం** — నిజమైన bugs ని దాచేస్తుంది. Required data కి కాదు.
- **`??` ని `||` / `&&` తో parentheses లేకుండా కలపడం** — `a ?? b || c` = SyntaxError. `(a ?? b) || c` రాయాలి.
- **`**` right-associative మర్చిపోవడం** — `2 ** 3 ** 2` = `512` (`2**9`), `64` కాదు.
- **`typeof x === "number" ? a : b` కి బదులు `x ? a : b`** — 0 falsy కాబట్టి bug.

Key Points

- **`&&` / `||` short-circuit** & operand return చేస్తాయి (boolean కాదు).
- `??` = null/undefined కి మాత్రమే fallback; `||` = ఏ falsy కి అయినా.
- `?.` = safe nested access (null/undefined → undefined, crash లేదు).
- `...` = spread (expand) లేదా rest (collect), position బట్టి.
- `**` = exponent, **right-associative.** `/` ఎప్పుడూ float ఇస్తుంది.

Interview దృష్టి

Q: `??` vs `||` తేడా, ఎప్పుడు ఏది? A: `||` ఎడమది **falsy** (0, "", false, null, undefined, NaN) అయితే కుడిది return చేస్తుంది. `??` ఎడమది **null లేదా undefined** అయితేనే. అందుకే `0` లేదా `""` valid values ఉన్న defaults కి `??` వాడాలి (`userCount ?? 10` – 0 అయితే 0 ఉంచుతుంది, `||` అయితే 10 తెచ్చేది).

Q: Short-circuit evaluation అంటే? A: `&&` / `||` అవసరమైనంత వరకే evaluate చేస్తాయి. `a && b()` – a falsy అయితే `b()` అసలు call అవ్వదు. `a || b()` – a truthy అయితే `b()` skip. ఇది conditional execution (`isValid && submit()`) మరియు defaults (`x || fallback`) కి వాడతారు, మరియు expensive/unsafe కుడివైపు call ని guard చేస్తుంది.

Q: Spread మరియు rest ఒకే `...` ఎలా వేర్వేరు? A: Position చూసి. Function call/array/object literal లో ఉంటే **spread** (విడగొడుతుంది – `[...arr]`, `f(...args)`). Function parameter list లో లేదా destructuring యొక్క ఎడమవైపు ఉంటే **rest** (సేకరిస్తుంది – `function f(...args)`, `const [a, ...rest]`).

---

7. Strings

వివరణ

String = characters యొక్క sequence. JavaScript లో strings **immutable** – ఒకసారి create అయ్యాక మార్చలేం. "మార్చినట్టు" అనిపించే ప్రతి method నిజానికి **కొత్త string** create చేస్తుంది, original ని touch చేయదు. ఇది చాలా methods, template literals, మరియు Unicode subtleties తో వస్తుంది.

Real-life Scenario

> **String = రాతి మీద చెక్కిన శాసనం (stone inscription).** ఒకసారి చెక్కాక, ఆ రాయి మీద ఒక అక్షరం మార్చలేవు. "మార్చాలంటే" కొత్త రాయి తీసుకుని మళ్ళీ చెక్కాలి (కొత్త string). పాత రాయి అలాగే ఉంటుంది. అందుకే `str[0] = "X"` పని చేయదు – శాసనాన్ని మార్చలేవు.

Immutability

```
let s = "hello";
s[0] = "H";        // ❌ silent fail (strict mode లో error) – మారదు
console.log(s);    // "hello" (unchanged)

s = s.toUpperCase(); // ✅ కొత్త string "HELLO" ని s కి assign చేశాం
console.log(s);      // "HELLO"

// ప్రతి "modification" కొత్త string:
const a = "abc";
const b = a.replace("a", "X"); // b = "Xbc", a = "abc" (unchanged)
```

Creating Strings & Template Literals

```
const single = 'hi';
const double = "hi";
const name = "Surya", age = 30;

// ── Template literals (backticks) — ES6 ──
const msg = `${name} is ${age} years old`;   // interpolation
const sum = `Total: ${10 + 20}`;              // expressions
const multi = `Line 1
Line 2`;                                       // multi-line (no \n needed)

// Nested + ternary:
const status = `User is ${age >= 18 ? "adult" : "minor"}`;

// Tagged templates (advanced) — function తో process:
function highlight(strings, ...values) {
  return strings.reduce((acc, str, i) =>
    `${acc}${str}${values[i] ? `<b>${values[i]}</b>` : ""}`, "");
}
highlight`Name: ${name}`; // "Name: <b>Surya</b>"
```

ముఖ్యమైన String Methods

```
const str = "Hello World";

// ── Access / search ──
str.length              // 11
str[0]                  // "H"
str.charAt(0)           // "H"
str.at(-1)              // "d"  (negative index! ES2022)
str.indexOf("o")        // 4    (మొదటిది; లేకపోతే -1)
str.lastIndexOf("o")    // 7
str.includes("World")   // true
str.startsWith("Hell")  // true
str.endsWith("rld")     // true

// ── Extract ──
str.slice(0, 5)         // "Hello"  (start, end) — negative OK
str.slice(-5)           // "World"
str.substring(0, 5)     // "Hello"  (negative ని 0 చేస్తుంది)

// ── Transform (అన్నీ కొత్త string) ──
str.toUpperCase()       // "HELLO WORLD"
str.toLowerCase()       // "hello world"
"  hi  ".trim()         // "hi"      (trimStart / trimEnd కూడా)
str.replace("o", "0")   // "Hell0 World" (మొదటిది మాత్రమే)
str.replaceAll("o", "0")// "Hell0 W0rld" (అన్నీ, ES2021)
"ab".repeat(3)          // "ababab"
"5".padStart(3, "0")    // "005"     (padEnd కూడా)

// ── Split / join ──
"a,b,c".split(",")      // ["a", "b", "c"]
"abc".split("")         // ["a", "b", "c"]
["a","b"].join("-")     // "a-b"
```

> `slice` **vs** `substring` **vs** `substr`**:** `slice` negative indices support చేస్తుంది (`-5` = చివరి నుండి) – దీన్నే వాడు. `substring` negatives ని 0 చేస్తుంది. `substr` deprecated – వాడకు.

లోపల ఏం జరుగుతుంది – Unicode & Code Points

JavaScript strings **UTF-16** లో store అవుతాయి – ప్రతి "unit" 16 bits. చాలా common characters (English, most languages) ఒక్క unit. కానీ emojis, కొన్ని Unicode characters **రెండు units** (surrogate pair) తీసుకుంటాయి – ఇక్కడే `.length` తప్పు చెప్తుంది!

```
"a".length          // 1
"😀".length         // 2 ❗ – ఒక emoji, కానీ length 2 (surrogate pair)
"😀"[0]             // "�" – విరిగిన సగం character!

// సరైన మార్గం – spread లేదా for-of (code points వాడతాయి):
[..."😀"].length    // 1 ✅
[..."a😀b"]         // ["a", "😀", "b"] ✅

// code point methods:
"A".charCodeAt(0)      // 65   (UTF-16 unit – 16-bit)
"😀".codePointAt(0)    // 128512 (పూర్తి code point – 21-bit safe)
String.fromCharCode(65)      // "A"
String.fromCodePoint(128512) // "😀"

// normalize – accented chars రెండు రూపాలుగా ఉండవచ్చు:
"é".normalize("NFC") === "é".normalize("NFC") // true (canonical)
```

> **గుర్తుంచుకో:** Emoji/Unicode ఉన్న strings లో `.length` = **UTF-16 units**, నిజమైన characters సంఖ్య కాదు. Character count కి `[...str].length` వాడు.

String comparison

```
"a" < "b"       // true  (Unicode code point పోలిక)
"Z" < "a"       // true  (uppercase Unicode < lowercase!)
"10" < "9"      // true  (string comparison – "1" < "9")

// locale-aware sorting (భాషలకి సరైనది):
["ä", "z", "a"].sort((x, y) => x.localeCompare(y)); // proper order
```

Gotchas (సాధారణ తప్పులు)

- `str[0] = "X"` **పని చేస్తుందనుకోవడం** – strings immutable; silent fail.
- `.length` **= character count అనుకోవడం** – emoji/Unicode తో UTF-16 units, `[...str].length` వాడు.
- `replace("o", ...)` **అన్ని** `o` **లని మారుస్తుందనుకోవడం** – మొదటిది మాత్రమే; `replaceAll` లేదా `/o/g` regex.
- **String concatenation loop లో** `+=` – పెద్ద data కి slow (కొత్త string ప్రతిసారి); `array.push` + `join` వాడు.

- `"10" < "9"` **true** – string comparison lexicographic, numeric కాదు; numbers ని `Number()` చేయాలి.

Key Points

- Strings **immutable** – methods కొత్త string return చేస్తాయి, original మారదు.
- **Template literals** (` ): interpolation `${}`, multi-line, expressions, tagged templates.
- `slice` (negative OK) > `substring` > ~~`substr`~~ (deprecated).
- `replace` = first only; `replaceAll` = all.
- UTF-16: emoji `.length` = 2. Character-safe iteration: `[...str]`, `for...of`, `codePointAt`.

Interview దృష్టి

Q: JS strings mutable నా? A: కాదు, immutable. ఒకసారి create అయ్యాక characters మార్చలేం (`str[0] = "X"` silent fail). ప్రతి "modifying" method (toUpperCase, replace, slice) కొత్త string return చేస్తుంది, original touch చేయదు. అందుకే loop లో `+=` తో string build చేయడం inefficient – array + join మంచిది.

Q: `"😀".length` ఎందుకు `2`? A: JS strings UTF-16 encoded. చాలా characters ఒక 16-bit unit, కానీ emoji లాంటి BMP-వెలుపలి characters **surrogate pair** – రెండు 16-bit units. `.length` units లెక్కిస్తుంది, characters కాదు. నిజమైన character count కి `[...str].length` లేదా `Array.from(str).length` (ఇవి code points ని iterate చేస్తాయి).

Q: `slice`, `substring`, `substr` తేడా? A: `slice(start, end)` – negative indices support (చివరి నుండి), most flexible, దీన్నే వాడు. `substring(start, end)` – negatives ని 0 చేస్తుంది, arguments swap చేస్తుంది. `substr(start, length)` – deprecated, వాడకూడదు.

---

8. Numbers & Math

వివరణ

చాలా భాషల్లా JS లో `int`, `float`, `double`, `long` వేర్వేరు types **లేవు.** ఒకే `number` type – అది **IEEE-754 double-precision (64-bit) floating point.** ఇది integers, decimals రెంటికీ ఒకటే. ఈ ఒకే-type approach convenient కానీ కొన్ని ఆశ్చర్యకరమైన gotchas తెస్తుంది – precision loss, `0.1 + 0.2` problem. పెద్ద integers కి **BigInt** అనే వేరే type ఉంది.

Real-life Scenario

> **Floating point = పరిమిత అరల ఉన్న సూట్‌కేస్‌లో అనంతమైన బట్టలు.** 64 bits అనే fixed సూట్‌కేస్‌లో అనంతమైన decimal numbers ని పెట్టాలి. కొన్ని numbers (`0.1`, `0.2`) binary లో అనంత digits – వాటిని కొంచెం **నలిపి (rounded)** పెట్టాలి. అందుకే `0.1 + 0.2` బయటకి తీసినప్పుడు కొంచెం **నలిగిపోయి** `0.30000000000000004` వస్తుంది. Suitcase తప్పు కాదు – infinity ని finite లో పెట్టడం అసాధ్యం.

లోపల ఏం జరుగుతుంది – IEEE-754 & floating point

కంప్యూటర్ binary (base-2) లో numbers store చేస్తుంది. `0.1` ని base-10 లో రాయగలం, కానీ base-2 లో అది `0.0001100110011...` (అనంతంగా repeat). 64 bits లో దీన్ని పెట్టడానికి round చేయాలి – అందుకే చిన్న error వస్తుంది.

```
0.1 + 0.2              // 0.30000000000000004 ❗
0.1 + 0.2 === 0.3      // false ❗❗
0.3 - 0.2              // 0.09999999999999998

// సరైన పోలిక – epsilon (tiny tolerance):
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON  // true ✅

// money (currency) కి – integers గా పని చేయి (cents/paise):
const price = 1010;  // ₹10.10 ని 1010 paise గా
// లేదా toFixed (display కి మాత్రమే – string return చేస్తుంది):
(0.1 + 0.2).toFixed(2)  // "0.30" (string!)
```

> **Money ని ఎప్పుడూ floating-point లో store చేయకు.** Cents/paise integers గా, లేదా decimal libraries (`decimal.js`, `big.js`) వాడు. Banking bug #1 ఇదే.

Safe Integers

64-bit double లో integer safe గా represent అయ్యే గరిష్ఠ పరిమితి `2^53 - 1`. దాన్ని దాటితే precision loss.

```
Number.MAX_SAFE_INTEGER   // 9007199254740991 (2^53 - 1)
Number.MIN_SAFE_INTEGER   // -9007199254740991

9007199254740991 + 1      // 9007199254740992 ✅
9007199254740991 + 2      // 9007199254740992 ❗ (should be ...993!)

Number.isSafeInteger(2 ** 53)  // false
```

BigInt (ES2020) – పెద్ద integers కి

Safe limit దాటిన integers కి `BigInt` – చివర `n` పెట్టడం ద్వారా.

```
const big = 9007199254740991n;   // n suffix
const huge = BigInt("123456789012345678901234567890");
big + 1n                         // 9007199254740992n ✅ (precise)
big + 2n                         // 9007199254740993n ✅ (correct!)

typeof 10n                       // "bigint"

// ⚠️ BigInt మరియు number ని కలపలేం:
// 10n + 5      // ❌ TypeError: Cannot mix BigInt and other types
10n + 5n         // 15n ✅
Number(10n) + 5  // 15  (explicit convert)
10n == 10        // true  (loose – coerce)
10n === 10       // false (strict – types వేరు)
```

NaN – "Not a Number"

`NaN` అనేది invalid numeric operation యొక్క ఫలితం. దాని అత్యంత విచిత్రమైన లక్షణం: **NaN దేనితోనూ equal కాదు, తనతో సహా.**

```
0 / 0             // NaN
Number("abc")     // NaN
Math.sqrt(-1)     // NaN
parseInt("xyz")   // NaN
undefined + 1     // NaN

typeof NaN        // "number" ❗ (NaN అయినా number type!)
NaN === NaN       // false ❗❗ (తనతో కూడా equal కాదు!)

// అందుకే NaN check ఇలా చేయలేం: x === NaN (ఎప్పుడూ false)
// సరైన మార్గం:
Number.isNaN(x)   // ✅ (best — coerce చేయదు)
isNaN(x)          // ⚠️ పాతది — coerce చేస్తుంది: isNaN("abc") = true!
Number.isNaN("abc") // false (string, NaN కాదు — correct)
```

Infinity

```
1 / 0             // Infinity
-1 / 0            // -Infinity
Infinity + 1      // Infinity
Number.MAX_VALUE * 2  // Infinity (overflow)
typeof Infinity   // "number"
Number.isFinite(x)    // finite number check
```

Number Methods & Math

```
// ── Number methods ──
(3.14159).toFixed(2)      // "3.14"  (string!)
(255).toString(16)        // "ff"    (hex)
(1234.5678).toLocaleString("en-IN") // "1,234.568" (formatting)
Number.parseInt("42px")   // 42
Number.parseFloat("3.14") // 3.14

// ── Math object ──
Math.round(4.5)     // 5   (.5 → up)
Math.round(-4.5)    // -4  (⚠️ -.5 → up కాదు, "toward +∞")
Math.floor(4.9)     // 4   (కిందకి)
Math.ceil(4.1)      // 5   (పైకి)
Math.trunc(4.9)     // 4   (decimal తీసేస్తుంది, no rounding)
Math.abs(-5)        // 5
Math.max(1, 5, 3)   // 5
Math.min(...[1,5,3]) // 1  (array → spread)
Math.pow(2, 10)     // 1024  (2 ** 10 కూడా)
Math.sqrt(16)       // 4
Math.random()       // [0, 1) random
Math.sign(-5)       // -1

// 1-6 random dice:
Math.floor(Math.random() * 6) + 1
```

Gotchas (సాధారణ తప్పులు)

- `0.1 + 0.2 !== 0.3` – floating point. Epsilon comparison లేదా integer cents వాడు.
- **Money ని float లో store చేయడం** – precision bugs. Paise/cents integers.
- `x === NaN` – ఎప్పుడూ false. `Number.isNaN(x)` వాడు.
- `isNaN()` **vs** `Number.isNaN()` – పాత `isNaN("abc")` = true (coerce చేస్తుంది); `Number.isNaN` safe.
- **Big IDs (>2^53) ని number గా parse చేయడం** – precision loss (Twitter/DB IDs). String లేదా BigInt గా ఉంచు.
- `parseInt` **radix మర్చిపోవడం** – `parseInt("0x10")` = 16; ఎప్పుడూ `parseInt(s, 10)`.

Key Points

- ఒకే `number` type = IEEE-754 64-bit double (int + float రెంటికీ).
- `0.1 + 0.2 = 0.30000...4` – floating point. Epsilon compare, money = integers.
- **Safe integer limit** `2^53 - 1` (`Number.MAX_SAFE_INTEGER`); దాటితే `BigInt` (`123n`).
- **NaN**: `typeof` = number, `NaN !== NaN`, check `Number.isNaN`.
- `Math.trunc` (cut) vs `floor` (down) vs `round` (nearest) vs `ceil` (up).

Interview దృష్టి

Q: `0.1 + 0.2 === 0.3` ఎందుకు false? A: JS numbers IEEE-754 64-bit floating point. `0.1` మరియు `0.2` binary లో అనంతంగా repeat అయ్యే fractions – 64 bits లో పెట్టడానికి round చేస్తుంది, చిన్న error వస్తుంది. Result `0.30000000000000004`. పోలిక కి `Math.abs(a - b) < Number.EPSILON`, money కి integer cents వాడాలి.

Q: NaN ని ఎలా check చేస్తావు? A: `NaN === NaN` ఎప్పుడూ false కాబట్టి equality పని చేయదు. `Number.isNaN(x)` వాడాలి – ఇది coerce చేయకుండా x నిజంగా NaN అయితేనే true. పాత global `isNaN()` coerce చేస్తుంది (`isNaN("abc")` = true, తప్పు), కాబట్టి avoid.

Q: BigInt ఎప్పుడు అవసరం? A: Integers `Number.MAX_SAFE_INTEGER` (2^53 - 1) దాటినప్పుడు – పెద్ద DB IDs, cryptography, precise big-integer math. `123n` లేదా `BigInt(x)`. కానీ BigInt మరియు number ని operations లో కలపలేం (TypeError); explicit convert చేయాలి, మరియు BigInt Math object తో పని చేయదు.

---

9. Control Flow

వివరణ

Control flow = code ఏ order లో execute అవుతుందో నిర్ణయించడం – conditions (`if` / `switch`) మరియు loops (`for` / `while`). Basics అందరికీ తెలుసు, కానీ JS లో loops కి **అనేక variants** (`for`, `for-of`, `for-in`, `forEach`, `while`) ఉన్నాయి – ఏది ఎప్పుడు వాడాలో తెలియడమే senior skill.

Conditionals – if / else / switch

```
// if / else if / else
const score = 75;
if (score >= 90)      grade = "A";
else if (score >= 75) grade = "B";
else                  grade = "C";

// switch — ఒకే value ని అనేక cases తో పోల్చినప్పుడు
switch (day) {
  case "Sat":
  case "Sun":                    // fall-through (multiple cases)
    console.log("Weekend");
    break;                       // ⚠️ break మర్చిపోతే కింది case కి పడిపోతుంది!
  case "Mon":
    console.log("Monday");
    break;
  default:
    console.log("Weekday");
}
```

> `switch` `===` **(strict) వాడుతుంది** — coercion లేదు. `case "1":` కి `1` (number) match అవ్వదు.

Real-life Scenario – break మర్చిపోవడం

> `switch` **= రైలు పట్టాలు.** ప్రతి `case` ఒక station. `break` = ఆ station లో ఆగే signal. `break` పెట్టకపోతే రైలు ఆగకుండా **తర్వాతి stations అన్నిటి గుండా దూసుకుపోతుంది** (fall-through). కొన్నిసార్లు ఇది కావాలి (multiple cases → ఒకే పని), కానీ చాలాసార్లు ఇది **bug** – `break` మర్చిపోవడం classic mistake.

Loops – పూర్తి పోలిక

Loop	దేనిమీద iterate	ఏం ఇస్తుంది	ఎప్పుడు వాడాలి
`for`	counter	index	సంఖ్య తెలిసినప్పుడు, index అవసరమైనప్పుడు
`for...of`	**iterable** (array, string, Map, Set)	**value**	array/string values కి (best)
`for...in`	object **keys** (enumerable)	**key** (string)	object properties కి
`forEach`	array	value + index	array కి (కానీ break లేదు)
`while`	condition	–	condition ఆధారంగా (సంఖ్య తెలియనప్పుడు)
`do...while`	condition	–	కనీసం ఒకసారి run కావాలి

```
const arr = ["a", "b", "c"];

// ── for (classic) ──
for (let i = 0; i < arr.length; i++) console.log(i, arr[i]); // 0 "a" ...

// ── for...of → VALUES (arrays, strings కి best) ──
for (const val of arr) console.log(val);       // "a", "b", "c"
for (const ch of "hi") console.log(ch);        // "h", "i"
for (const [i, v] of arr.entries()) console.log(i, v); // index + value

// ── for...in → KEYS (objects కి) ──
const obj = { x: 1, y: 2 };
for (const key of obj) {} // ❌ TypeError — object iterable కాదు!
for (const key in obj) console.log(key, obj[key]); // "x" 1, "y" 2

// ── while / do-while ──
let n = 0;
while (n < 3) { console.log(n); n++; }
do { console.log(n); n++; } while (n < 3); // కనీసం ఒకసారి
```

for...of vs for...in — అత్యంత common confusion

```
const arr = ["a", "b", "c"];

for (const x of arr) console.log(x); // "a", "b", "c" (VALUES) ✅
for (const i in arr) console.log(i); // "0", "1", "2" (INDICES as strings!) ⚠️

// for...in ని arrays కి వాడకూడదు ఎందుకంటే:
arr.customProp = "oops";
for (const i in arr) console.log(i); // "0","1","2","customProp" ❗
// → for...in inherited/added enumerable properties కూడా iterate చేస్తుంది!
```

> **నియమం:** `for...of` = **values** (arrays, strings, Maps, Sets — iterables). `for...in` = **keys** (plain objects మాత్రమే). Array కి ఎప్పుడూ `for...of`. "**o**f = **o**bjects values, **in** = **in**dices/keys."

break / continue / labels

```
// break — loop నుండి పూర్తిగా బయటకి
for (const x of arr) {
  if (x === "b") break;      // "b" వచ్చాక ఆగిపో
  console.log(x);            // "a"
}

// continue — ఈ iteration skip చేసి తర్వాతిది
for (let i = 0; i < 5; i++) {
  if (i % 2 === 0) continue; // even skip
  console.log(i);            // 1, 3
}

// labels — nested loops నుండి బయటకి (అరుదు)
outer:
```

```
for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i + j === 2) break outer; // రెండు loops నుండీ బయటకి!
  }
}
```

Gotchas (సాధారణ తప్పులు)

- `switch` లో `break` **మర్చిపోవడం** – fall-through bug. లేదా intentional fall-through ని comment తో గుర్తించు.
- **Array కి** `for...in` – indices ని strings గా ("0","1") ఇస్తుంది + inherited props. `for...of` వాడు.
- `forEach` **లో** `break` – పని చేయదు! `forEach` ఆపలేం. Early exit కావాలంటే `for...of` లేదా `some` / `every`.
- `forEach` **లో** `await` – పని చేయదు (async ని wait చేయదు); `for...of` + `await` వాడు (Topic 30).
- **Infinite loop** – `while(true)` లో exit condition మర్చిపోవడం, లేదా `i++` మర్చిపోవడం.
- `switch` **coercion అనుకోవడం** – `switch` `===` వాడుతుంది; `"1"` కి `1` match కాదు.

Key Points

- `switch` = strict `===`; `break` మర్చిపోతే fall-through.
- `for...of` **= values** (iterables), `for...in` **= keys** (objects). Array కి `for...of`.
- `forEach` – clean కానీ `break` / `await` **పని చేయవు.**
- `break` = loop exit, `continue` = skip iteration, **labels** = nested loops exit.
- `do...while` = కనీసం ఒకసారి guarantee.

Interview దృష్టి

Q: `for...of` **vs** `for...in` **తేడా?** A: `for...of` – iterables (arrays, strings, Maps, Sets) మీద **values** ని iterate చేస్తుంది. `for...in` – object యొక్క enumerable **keys** (arrays కి indices ని strings గా) ని, inherited properties తో సహా iterate చేస్తుంది. అందుకే arrays కి `for...of` వాడాలి; `for...in` ని plain objects కి మాత్రమే. Plain object `for...of` కి iterable కాదు (TypeError).

Q: `forEach` **లో loop ని ఎలా ఆపుతావు?** A: `forEach` ని ఆపలేం – `break` / `return` loop ని stop చేయవు (`return` కేవలం ఆ ఒక్క callback నుండి బయటకి వస్తుంది). Early exit కావాలంటే `for...of` (break support), లేదా `Array.some()` (true return చేస్తే ఆగుతుంది) / `Array.every()` వాడాలి.

Q: `switch` **coercion చేస్తుందా?** A: లేదు. `switch` cases ని strict equality (`===`) తో పోల్చుతుంది – type coercion లేదు. కాబట్టి `case "1":` కి number `1` match అవ్వదు.

Part 2 — Functions, Scope & Closures

> ఇక్కడే JavaScript యొక్క నిజమైన "ఆత్మ" ఉంది. Functions first-class citizens, lexical scope, closures, `this`, `call/apply/bind` — ఈ concepts అర్థమైతే JS master అయినట్టే. చాలా interview questions, tricky bugs ఇక్కడి నుండే వస్తాయి. ఈ Part ని నెమ్మదిగా, గట్టిగా చదువు.

---

10. Functions

వివరణ

JavaScript లో functions **first-class citizens** — వాటిని variables కి assign చేయవచ్చు, arguments గా pass చేయవచ్చు, return చేయవచ్చు, arrays/objects లో పెట్టవచ్చు. Function ని define చేయడానికి అనేక మార్గాలు (declaration, expression, arrow), ప్రతి దానికి తనదైన hoisting, `this`, `arguments` behavior. ఈ తేడాలు అర్థం కావడం చాలా ముఖ్యం.

Function Declaration vs Expression

```
// ── Declaration — పూర్తిగా hoist అవుతుంది (define ముందే call చేయవచ్చు) ──
sayHi();  // ✅ "Hi" — declaration hoisted with body
function sayHi() { console.log("Hi"); }

// ── Expression — variable hoisting rules వర్తిస్తాయి ──
sayBye(); // ❌ ReferenceError (const TDZ) / TypeError (var: undefined కి call)
const sayBye = function() { console.log("Bye"); };

// Named function expression (recursion/stack traces కి):
const fact = function factorial(n) {
  return n <= 1 ? 1 : n * factorial(n - 1); // లోపల పేరు వాడవచ్చు
};
```

అంశం	Declaration	Expression
Hoisting	పూర్తిగా (body తో)	variable rules (TDZ/undefined)
Define ముందు call	✅	❌
పేరు	తప్పనిసరి	optional (anonymous)

Arrow Functions (ES6)

Arrow functions కేవలం short syntax కాదు — వాటికి **తమదైన `this` లేదు, `arguments` లేదు, `new` తో వాడలేం.** ఇదే వాటి అతిపెద్ద తేడా.

```
const add = (a, b) => a + b;          // implicit return (no braces)
const square = x => x * x;            // ఒక param → () optional
const greet = () => "hi";             // no params → () తప్పనిసరి
const getObj = () => ({ a: 1 });      // object return → () చుట్టూ! (లేకపోతే {} = block)
const multi = (a, b) => {             // block body → return explicit
  const sum = a + b;
  return sum;
};
```

Arrow vs Regular – కీలక తేడాలు:

అంశం	Regular function	Arrow function
`this`	call ఎలా చేశారో బట్టి (dynamic)	**lexical** (outer scope నుండి)
`arguments`	✅ ఉంది	❌ లేదు (rest `...args` వాడు)
`new` తో	✅ constructor	❌ TypeError
hoisting	declaration hoist	expression rules
method గా	మంచిది	❌ (this bind కాదు)

```
// Arrow this = lexical (Topic 14 లో deep) — ఇదే arrow ముఖ్యమైన use:
const obj = {
  name: "Surya",
  hobbies: ["read", "code"],
  show() {
    // arrow ఇక్కడ this ని obj నుండి తీసుకుంటుంది (regular అయితే undefined అయ్యేది)
    this.hobbies.forEach(h => console.log(`${this.name}: ${h}`));
  }
};
obj.show(); // "Surya: read", "Surya: code"
```

Real-life Scenario – Arrow యొక్క `this`

> **Regular function = అద్దె ఇంట్లో ఉన్న వ్యక్తి** – ఎవరు పిలిస్తే వాళ్ళ ఇంటి address (this) వాడతాడు. Address ప్రతిసారి మారుతుంది (dynamic).
>
> **Arrow function = పుట్టిన ఇంటి address ని permanent గా పెట్టుకున్న వ్యక్తి** – ఎవరు ఎక్కడ పిలిచినా, తను **పుట్టిన చోటి (lexical) address** నే వాడతాడు. అందుకే `forEach`, `setTimeout` callbacks లో arrow వాడితే `this` సరిగ్గా ఉంటుంది.

Parameters – default, rest, arguments

```
// ── Default parameters (ES6) ──
function greet(name = "Guest", greeting = "Hello") {
  return `${greeting}, ${name}`;
}
greet();               // "Hello, Guest"
```

```
greet("Surya");       // "Hello, Surya"
greet(undefined, "Hi"); // "Hi, Guest" — undefined = default వాడు (null కాదు!)

// default లో ముందు params వాడవచ్చు:
function box(w, h = w) { return w * h; } // h default = w
box(5);               // 25

// ── Rest parameters — variable args ──
function sum(...nums) {          // అన్నీ array గా
  return nums.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3, 4);      // 10

function log(first, ...rest) {   // కొన్ని named + మిగతా
  console.log(first, rest);      // rest ఎప్పుడూ చివర్లో
}

// ── arguments object (regular functions లో మాత్రమే) ──
function old() {
  console.log(arguments);        // array-LIKE (real array కాదు!)
  console.log(arguments.length);
  const arr = [...arguments];    // real array కి convert
}
// arrow లో arguments లేదు — rest వాడు:
const modern = (...args) => args.reduce((a, b) => a + b);
```

> `arguments` **vs rest:** `arguments` పాత array-like object (map/filter లేవు), arrow లో లేదు. Rest `...args` **నిజమైన array**, arrow లో పని చేస్తుంది. Modern code లో ఎప్పుడూ rest.

Gotchas (సాధారణ తప్పులు)

- **Arrow ని object method గా వాడటం** – `this` object కి bind అవ్వదు (lexical = outer/global). Methods కి regular function.
- **Arrow ని event handler గా వాడి `this` = element అనుకోవడం** – arrow `this` element కాదు.
- **Arrow తో `new`** – TypeError; constructors కి regular functions/classes.
- **Default param కి `null` పంపడం** – `null` default ని trigger చేయదు (`undefined` మాత్రమే).
- **`arguments` ని arrow లో వాడటం** – undefined/outer నుండి తీసుకుంటుంది; rest వాడు.
- **Object return చేసే arrow లో `()` మర్చిపోవడం** – `() => {a:1}` block అనుకుంటుంది; `() => ({a:1})`.

Key Points

- Functions = **first-class** (pass, return, store).
- **Declaration** hoist (body తో), **expression** variable rules follow.
- **Arrow**: lexical `this`, no `arguments`, no `new`, short syntax. Methods/constructors కి కాదు.
- **Default params**: `undefined` trigger చేస్తుంది, `null` కాదు.
- **Rest `...args`** = నిజమైన array, arrow-friendly; `arguments` = పాత array-like, regular only.

Interview దృష్టి

Q: Arrow function మరియు regular function తేడా? A: (1) `this` – regular dynamic (call ఎలా చేశారో బట్టి), arrow lexical (outer scope నుండి, bind కాదు). (2) `arguments` – regular కి ఉంది, arrow కి లేదు. (3) `new` – regular constructor అవుతుంది, arrow TypeError. (4) hoisting – declaration hoist, arrow expression rules. కాబట్టి callbacks కి arrow (this preserve), object methods/constructors కి regular.

Q: Function declaration vs expression? A: Declaration (`function f(){}`) పూర్తిగా hoist అవుతుంది – define ముందే call చేయవచ్చు. Expression (`const f = function(){}`) variable hoisting rules follow అవుతుంది – `const` / `let` అయితే TDZ, `var` అయితే undefined కి call చేసి TypeError. Expression లో పేరు optional.

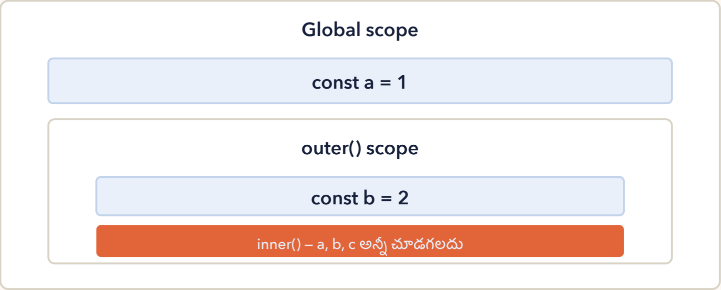
Q: `arguments` object అంటే? Arrow లో ఉందా? A: Regular functions లో ఉండే array-like object – pass అయిన అన్ని arguments ని పట్టుకుంటుంది (map/filter లేవు, real array కాదు). Arrow functions కి `arguments` లేదు – అవి outer scope నుండి తీసుకుంటాయి. Modern replacement: rest parameter `...args` (real array, arrow లో పని చేస్తుంది).

---

11. Scope & Scope Chain

SCOPE CHAIN · LEXICAL SCOPING

SCOPE CHAIN – లోపలి నుంచి బయటికి వెతకడం



వెతికే క్రమం

1 · నా సొంత scope
2 · నా parent scope
3 · ... global దాకా
4 · దొరకకపోతే ReferenceError

ఒక దిక్కు మాత్రమే

Inner outer ని చూడగలదు; outer inner ని **ఎప్పటికీ** చూడలేదు.
Scope chain **code ఎక్కడ రాశారో** దాని మీద ఆధారపడుతుంది (lexical), ఎక్కడి నుంచి పిలిచారో దాని మీద కాదు.
ఇదే closures పని చేయడానికి కారణం.

వివరణ

Scope అంటే – ఒక variable **ఎక్కడ నుండి access చేయగలం** అనే ప్రాంతం. JavaScript లో 3 rakala scopes: **Global, Function, Block.** ఒక variable కనిపించకపోతే, JS **scope chain** ద్వారా బయటి scopes లో వెతుకుతుంది. ఈ వెతికే విధానం **lexical (static)** – code ఎక్కడ **రాశారో** బట్టి నిర్ణయం అవుతుంది, ఎక్కడ **call చేశారో** బట్టి కాదు.

Real-life Scenario

> **Scope chain = ఒక కుటుంబ డబ్బు ఏర్పాటు.** నీకు (inner scope) ఏదైనా కావాలంటే – మొదట **నీ జేబు** (local variables) చూస్తావు. లేకపోతే **నాన్న దగ్గరకి** (outer function) వెళ్తావు. ఆయన దగ్గర లేకపోతే **తాత దగ్గరకి** (global). ఎక్కడా దొరక్కపోతే "లేదు" (ReferenceError). కానీ **ఒక్క నియమం**: నువ్వు పైకి (బయటికి) మాత్రమే అడగగలవు – **తాత నీ జేబులో వెతకలేడు** (outer inner ని access చేయలేదు). డబ్బు ఒకే దిశలో ప్రవహిస్తుంది.

3 రకాల Scopes

```
const globalVar = "నేను global";  // Global scope – ఎక్కడైనా access

function outer() {
  const funcVar = "నేను function scope";  // outer లో మాత్రమే

  if (true) {
    let blockVar = "నేను block scope";     // ఈ {} లో మాత్రమే (let/const)
    var notBlock = "నేను function లోకి leak"; // var block ని పట్టించుకోదు!
    console.log(globalVar, funcVar, blockVar); // ✅ మూడూ కనిపిస్తాయి
  }

  console.log(notBlock);   // ✅ "leak" (var function-scoped)
  console.log(blockVar);   // ❌ ReferenceError (let block-scoped)
}

console.log(funcVar);      // ❌ ReferenceError (బయటి నుండి inner కనిపించదు)
```

Scope Chain – lookup ఎలా జరుగుతుంది

```
const a = "global-a";

function grandparent() {
  const b = "gp-b";

  function parent() {
    const c = "p-c";

    function child() {
      const d = "child-d";
      // child ఇక్కడ లోపలి నుండి బయటికి వెతుకుతుంది:
      console.log(d); // "child-d"  (own scope)
      console.log(c); // "p-c"      (parent లో దొరికింది)
      console.log(b); // "gp-b"     (grandparent లో)
      console.log(a); // "global-a" (global లో)
      console.log(z); // ❌ ReferenceError (ఎక్కడా లేదు)
    }
    child();
  }
  parent();
}
grandparent();
```

Lookup ఎప్పుడూ లోపల → బయటికి, ఒక్క దిశలో. Inner scope outer variables ని చూడగలదు; outer inner ని చూడలేదు.

లోపల ఏం జరుగుతుంది – Lexical (Static) Scope

JavaScript **lexical scoping** వాడుతుంది – scope అనేది code ని **రాసిన చోటు (physically ఎక్కడ ఉంది)** బట్టి నిర్ణయం అవుతుంది, **call చేసిన చోటు** బట్టి కాదు. ఇది **compile time** లోనే fix అవుతుంది.

```
const x = "global";

function printX() {
  console.log(x);   // ఏ x? – printX రాసిన చోట (lexical) x = "global"
}

function run() {
  const x = "local"; // ఈ x printX కి కనిపించదు!
  printX();          // "global" – call ఎక్కడ చేసినా, రాసిన చోటి scope
}
run(); // "global" (కాదు "local"!)
```

> **ఇది క్లిష్టమైన interview trap.** `printX` ని `run` లోపల **call** చేసినా, `printX` **రాసింది** global level లో – కాబట్టి దాని outer scope global. Call site కాదు, **definition site** ముఖ్యం. ఇదే lexical scope. (దీనికి opposite = "dynamic scope", ఇది JS లో లేదు.)

Global Scope యొక్క ప్రమాదాలు

```
// Browser లో var/function declarations global object (window) కి attach:
var leaked = 1;
console.log(window.leaked); // 1 (browser) – pollution!

// let/const attach అవ్వవు:
let safe = 2;
console.log(window.safe);   // undefined ✅

// Accidental global (strict mode లేకపోతే):
function bad() {
  count = 5;  // var లేదు → global count create అవుతుంది! (bug)
}
```

Global scope ని కలుషితం చేయడం (global pollution) పెద్ద అప్లికేషన్‌లలో naming conflicts, memory leaks, hard-to-track bugs కి దారితీస్తుంది. అందుకే code ని functions/modules/IIFEs లో encapsulate చేస్తారు.

Gotchas (సాధారణ తప్పులు)

- **Lexical ని dynamic అనుకోవడం** – scope definition site బట్టి, call site బట్టి కాదు.
- `var` **block-scoped అనుకోవడం** – `var` function-scoped; block నుండి leak అవుతుంది.
- **Accidental globals** – `let` / `const` / `var` లేకుండా assign చేస్తే (non-strict) global create. `"use strict"` దీన్ని catch చేస్తుంది.

- **Inner shadowing** – inner scope లో అదే పేరు variable outer ని దాచేస్తుంది (shadowing) – accidental అయితే confusing.
- **Global pollution** – పెద్ద apps లో global variables conflicts. Modules/closures వాడు.

Key Points

- 3 scopes: **Global** (ఎక్కడైనా), **Function** (`var` కూడా), **Block** (`let` / `const` `{}` లో).
- **Scope chain** = inner → outer, ఒక్క దిశలో lookup; దొరక్కపోతే ReferenceError.
- **Lexical scope** = definition site (code రాసిన చోటు) బట్టి, compile-time లో fix – **call site కాదు.**
- `var` function-scoped (block leak); `let` / `const` block-scoped.
- Global pollution ప్రమాదకరం – encapsulate చేయి.

Interview దృష్టి

Q: Lexical scope అంటే? A: Function యొక్క scope అది code లో **ఎక్కడ రాశారో** (physically nested ఎక్కడ ఉంది) దాన్ని బట్టి compile-time లో నిర్ణయం అవుతుంది – ఎక్కడ call చేశారో బట్టి కాదు. కాబట్టి `printX` ని ఎక్కడ call చేసినా, అది తను రాసిన చోటి outer variables ని మాత్రమే access చేస్తుంది. Closures ఈ lexical scope మీదే ఆధారపడతాయి.

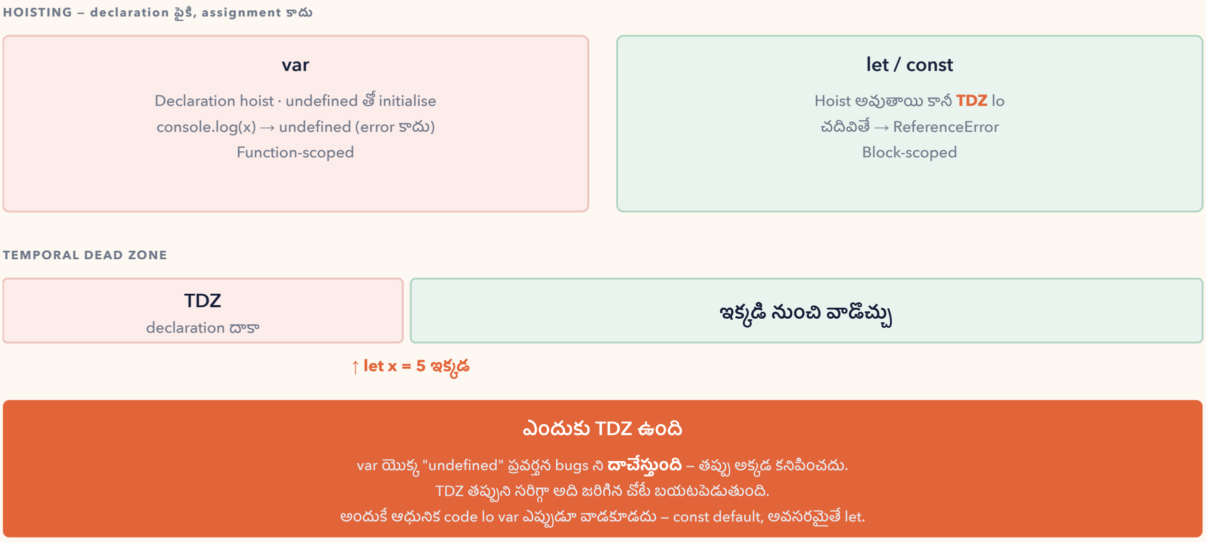
Q: Scope chain ఎలా పని చేస్తుంది? A: JS ఒక variable ని resolve చేయాలంటే మొదట current scope లో వెతుకుతుంది; లేకపోతే enclosing (outer) scope, తర్వాత దాని outer... global వరకు – ఇదే scope chain. దొరికిన మొదటి declaration వాడుతుంది; ఎక్కడా లేకపోతే ReferenceError. Lookup ఎప్పుడూ inner → outer, ఒక్క దిశలో – outer inner scope ని access చేయలేదు.

Q: `var` మరియు `let` scope తేడా? A: `var` function-scoped – ఏ block లో declare చేసినా, దగ్గరి function అంతటికీ కనిపిస్తుంది (blocks నుండి leak అవుతుంది). `let` / `const` block-scoped – వాటి `{}` block కి మాత్రమే పరిమితం. అందుకే loops, if-blocks లో `let` predictable.

---

12. Hoisting deep

HOISTING · VAR, LET, CONST మరియు TDZ



వివరణ

Hoisting అంటే – JavaScript code ని execute చేయడానికి ముందు, ఒక **"creation/compilation" phase** లో అన్ని `var`, `function`, `let`, `const` declarations ని scan చేసి memory లో register చేస్తుంది. దీనివల్ల declarations "పైకి లేచినట్టు" (hoisted) అనిపిస్తాయి – declare చేయకముందే వాడగలిగినట్టు. కానీ ఇది కేవలం భ్రమ; అసలు ఏం జరుగుతుందో execution context phases ద్వారా అర్థం చేసుకోవాలి.

లోపల ఏం జరుగుతుంది – Execution Context యొక్క 2 Phases

ప్రతి code (global లేదా function) run అయ్యేముందు ఒక **Execution Context** create అవుతుంది, అది **2 phases** లో నడుస్తుంది:

Phase 1 – Creation (Memory allocation):

- అన్ని `var` variables కి memory allocate చేసి `undefined` గా initialize.
- అన్ని `let` / `const` కి memory allocate కానీ **uninitialized** (TDZ – access చేస్తే error).
- అన్ని **function declarations** ని పూర్తి body తో memory లో పెట్టడం.

Phase 2 – Execution (Code run):

- Line by line code run అవుతుంది.
- Assignments ఇప్పుడు జరుగుతాయి (`x = 5`).

```
// మనం రాసేది:
console.log(a);   // undefined
console.log(fn);  // [Function: fn]
var a = 10;
function fn() {}

// JS "అర్థం చేసుకునేది" (conceptual):
// --- Creation phase ---
var a = undefined;      // hoisted, undefined
function fn() {}        // hoisted with body
// --- Execution phase ---
console.log(a);         // undefined
console.log(fn);        // [Function: fn]
a = 10;                 // ఇప్పుడు assign
```

Real-life Scenario

> **Hoisting = ఒక event కి guest list ముందే తయారు చేయడం.** Event మొదలవ్వకముందు (creation phase), organizer అందరి **పేర్లు** guest list లో రాసేస్తాడు – కానీ వాళ్ళు ఇంకా రాలేదు (`var` = "పేరు ఉంది, seat ఖాళీ = undefined").
>
> **Function declaration = VIP** – వాళ్ళ **పూర్తి details తో** ముందే register అయిపోతారు (body తో hoist). అందుకే event మొదలవ్వకముందే వాళ్ళని పిలవగలవు.
>
> `let` / `const` = **"పేరు రిజర్వ్ అయింది, కానీ badge ఇంకా ఇవ్వలేదు"** – declaration line వచ్చేవరకు వాళ్ళని లోపలికి రానివ్వరు (TDZ).

var vs let/const vs function — hoisting తేడా

```
// —— var: hoist + undefined ——
console.log(v); // undefined (crash కాదు)
var v = 1;

// —— let/const: hoist + TDZ (uninitialized) ——
console.log(l); // ❌ ReferenceError: Cannot access 'l' before initialization
let l = 1;

// —— function declaration: పూర్తిగా hoist (body తో) ——
hoisted();      // ✅ "works!" — పూర్తి function అందుబాటులో
function hoisted() { console.log("works!"); }

// —— function EXPRESSION: variable rules follow ——
notYet();       // ❌ TypeError: notYet is not a function (var = undefined కి call)
var notYet = function() {};

// arrow కూడా expression — అదే rule:
arrowFn();      // ❌ ReferenceError (const TDZ)
const arrowFn = () => {};
```

రకం	Hoist అవుతుందా	Initial value	Declare ముందు access
`var`	✅	`undefined`	`undefined` (no error)
`let`	✅	uninitialized (TDZ)	❌ ReferenceError
`const`	✅	uninitialized (TDZ)	❌ ReferenceError
function declaration	✅	పూర్తి function	✅ works
function/arrow expression	variable rules	variable బట్టి	❌ (TypeError/ReferenceError)

Function vs Variable — ఏది గెలుస్తుంది

```
console.log(typeof foo); // "function" — function declaration variable కంటే గెలుస్తుంది
var foo = "I am string";
function foo() {}
console.log(typeof foo); // "string" — execution phase లో assign అయ్యాక

// నియమం: function declarations, var declarations కంటే priority.
// అదే పేరు రెండు function declarations → చివరిది గెలుస్తుంది.
```

TDZ — మళ్ళీ deep

TDZ (Temporal Dead Zone) = scope మొదలైన క్షణం నుండి `let` / `const` declaration line వరకు. ఈ zone లో variable **exists** (hoisted) కానీ **uninitialized** — access = ReferenceError.

```
{
  // TDZ start (myLet కి)
  // console.log(myLet); // ❌ ReferenceError
  const before = "ok";   // ఇది వేరే variable, fine
```

```
  let myLet = 5;          // TDZ end — ఇక్కడ నుండి myLet safe
  console.log(myLet);     // 5 ✅
}

// typeof కూడా TDZ లో fail అవుతుంది (var తో fail అవ్వదు):
console.log(typeof undeclaredVar); // "undefined" (ఎప్పుడూ declare కాలేదు)
console.log(typeof tdzVar);        // ❌ ReferenceError (TDZ లో)
let tdzVar = 1;
```

Gotchas (సాధారణ తప్పులు)

- **"let/const hoist అవ్వవు" అనడం** – తప్పు. అవి hoist అవుతాయి కానీ TDZ లో ఉంటాయి (initialize అవ్వవు).
- **Function expression ని declaration లా ముందే call చేయడం** – TypeError (undefined కి call).
- **Function declaration variable కంటే గెలుస్తుందని మర్చిపోవడం** – same-name shadowing bugs.
- **TDZ లో `typeof` safe అనుకోవడం** – `let` / `const` కి TDZ లో `typeof` కూడా ReferenceError.
- **Best practice** – variables ని ఎప్పుడూ వాడేముందు, top లో declare చేయి; hoisting మీద ఆధారపడకు.

Key Points

- Hoisting = execution context **creation phase** లో declarations register అవ్వడం.
- `var` → `undefined`; **`let` / `const`** → TDZ (ReferenceError); **function declaration** → పూర్తి body.
- Function/arrow **expressions** hoist అవ్వవు (variable rules).
- Function declarations `var` కంటే **priority.**
- TDZ లో `typeof` కూడా error (`var` తో కాదు).

Interview దృష్టి

Q: Hoisting అంటే ఏమిటి, లోపల ఎలా జరుగుతుంది? A: Code execute అయ్యేముందు execution context యొక్క creation phase లో JS engine అన్ని declarations ని scan చేసి memory allocate చేస్తుంది – `var` కి `undefined`, `let` / `const` uninitialized (TDZ), function declarations పూర్తి body తో. అందుకే declaration లైన్ ముందే వాటిని (కొన్ని) reference చేయవచ్చు. Assignments మాత్రం execution phase లోనే జరుగుతాయి.

Q: `let` / `const` hoist అవుతాయా? A: అవును, hoist అవుతాయి – కానీ `var` లా `undefined` గా initialize అవ్వవు. Declaration వరకు అవి TDZ లో uninitialized గా ఉంటాయి; access చేస్తే ReferenceError. "hoist అవ్వవు" అనేది common misconception – నిజానికి hoist అవుతాయి, initialize అవ్వవు.

Q: ఈ output ఏమిటి – `console.log(typeof foo); var foo = "x"; function foo(){}`? A: `"function"`. Creation phase లో function declaration `foo` ని పూర్తి body తో hoist చేస్తుంది, `var foo` దాన్ని overwrite చేయదు (function priority). అందుకే execution phase మొదట్లో `foo` ఇంకా function; string assignment తర్వాత line లోనే జరుగుతుంది.

13. Closures

వివరణ

Closure అంటే – ఒక function తన **outer (lexical) scope లోని variables ని "గుర్తుపెట్టుకోవడం"** – ఆ outer function return అయిపోయినా కూడా. అంటే inner function, తను పుట్టిన environment ని తనతో పాటు మోసుకెళ్తుంది. ఇది JavaScript యొక్క **అత్యంత శక్తివంతమైన, అత్యంత అడిగే** concept. Closures = function + దాని lexical environment.

Real-life Scenario

> **Closure = ఒక backpack (వీపు సంచి).** ఒక function పుట్టినప్పుడు, తనకి చుట్టూ ఉన్న variables అన్నీ ఒక backpack లో సర్దుకుంటుంది. తర్వాత ఆ function ఎక్కడికెళ్ళినా – outer function చనిపోయినా (return అయినా) – ఆ backpack తనతోనే ఉంటుంది. అవసరమైనప్పుడు backpack తెరిచి ఆ variables వాడుకుంటుంది.
>
> ఉదా: outer function ఒక "school" (అది మూసేసినా), inner function ఆ school certificate (variable) ని తన backpack లో పెట్టుకుని జీవితాంతం వాడుకుంటుంది.

Code – మౌలిక closure

```
function makeCounter() {
  let count = 0;             // ఈ variable outer scope లోది

  return function () {       // inner function — count ని "గుర్తుపెట్టుకుంటుంది"
    count++;
    return count;
  };
}

const counter = makeCounter(); // makeCounter return అయిపోయింది...
console.log(counter()); // 1   ...కానీ count ఇంకా బతికే ఉంది!
console.log(counter()); // 2
console.log(counter()); // 3

const counter2 = makeCounter(); // కొత్త స్వతంత్ర closure
console.log(counter2()); // 1  — వేరే count (ఒకదానితో ఒకటి share అవ్వదు)
```

`makeCounter()` return అయ్యాక కూడా `count` garbage collect అవ్వలేదు – ఎందుకంటే returned function దాన్ని ఇంకా reference చేస్తోంది. ఇదే closure.

లోపల ఏం జరుగుతుంది

Function ని create చేసినప్పుడు, JS దానికి `[[Environment]]` అనే hidden property ఇస్తుంది – అది ఆ function పుట్టిన lexical scope ని point చేస్తుంది. Function ని call చేసినప్పుడు, ఒక variable ఆ function లో దొరక్కపోతే `[[Environment]]` ద్వారా outer scope లో వెతుకుతుంది. Outer function return అయినా, ఆ variables ని ఏదో ఒక inner function reference చేస్తున్నంత వరకు, memory లో అవి బతికే ఉంటాయి (garbage collect అవ్వవు).

> **ముఖ్యం:** Closure variable యొక్క **value** ని copy చేయదు – **variable (reference) నే** పట్టుకుంటుంది. అందుకే closure తర్వాత ఆ variable మారితే, closure కి కొత్త value కనిపిస్తుంది.

వాడకాలు (Uses) – closures ఎందుకు గొప్ప

1. Private state (data hiding / encapsulation):

```
function createBankAccount(initial) {
  let balance = initial;  // private! బయటి నుండి direct access లేదు
  return {
    deposit: (amt) => { balance += amt; return balance; },
    withdraw: (amt) => {
      if (amt > balance) return "సరిపోని balance";
      balance -= amt; return balance;
    },
    getBalance: () => balance
  };
}
const acc = createBankAccount(100);
acc.deposit(50);        // 150
acc.getBalance();       // 150
// acc.balance;         // undefined — direct access లేదు! (encapsulation)
```

2. Function factories (parameterized functions):

```
function multiplier(factor) {
  return (n) => n * factor;   // factor ని గుర్తుపెట్టుకుంటుంది
}
const double = multiplier(2);
const triple = multiplier(3);
double(5);  // 10
triple(5);  // 15
```

3. Memoization, debounce, once, event handlers – అన్నీ closures మీద ఆధారపడతాయి (Topic 17, 49).

The Loop Closure Gotcha (అత్యంత famous interview trap)

```
// ❌ var తో — అందరూ 3 print చేస్తారు!
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 3, 3, 3 ❗❗

// ఎందుకు? var function-scoped → ఒకే i అందరు closures share చేస్తారు.
// setTimeout callbacks loop అయ్యాక run అవుతాయి, అప్పటికి i = 3.

// ✅ పరిష్కారం 1: let (block-scoped → ప్రతి iteration కి కొత్త i)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 0, 1, 2 ✅
```

```
// ✅ పరిష్కారం 2: IIFE తో value capture (పాత ES5 మార్గం)
for (var i = 0; i < 3; i++) {
  ((j) => setTimeout(() => console.log(j), 100))(i);
}
// Output: 0, 1, 2 ✅
```

> **గుండె లోతున గుర్తుంచుకో:** `var` loop లో అందరు closures **ఒకే** variable ని share చేస్తారు (అది loop చివర్లో final value). `let` ప్రతి iteration కి **కొత్త binding** create చేస్తుంది – అందుకే ప్రతి closure తన iteration value ని పట్టుకుంటుంది. ఇదే `let` యొక్క గొప్ప advantage.

Closures & Memory

Closures అవసరమైన variables ని memory లో బతికేలా ఉంచుతాయి. దీన్ని జాగ్రత్తగా వాడకపోతే **memory leaks:** పెద్ద objects/DOM elements ని closures అనవసరంగా reference చేస్తే garbage collect అవ్వవు.

```
function leaky() {
  const bigData = new Array(1000000).fill("x"); // పెద్ద array
  return () => console.log("hi"); // bigData వాడకపోయినా closure దాన్ని పట్టుకోవచ్చు
}
// modern engines optimize చేస్తాయి, కానీ event listeners/timers లో జాగ్రత్త
```

Gotchas (సాధారణ తప్పులు)

- **Loop + `var` + async** – అన్నీ final value. `let` లేదా IIFE వాడు.
- **Closure value copy అనుకోవడం** – reference పట్టుకుంటుంది; తర్వాత variable మారితే కొత్త value కనిపిస్తుంది.
- **Memory leaks** – closures పెద్ద data/DOM/timers ని అనవసరంగా బతికించడం. అవసరం తీరాక `null` చేయి, listeners remove చేయి.
- **అన్ని inner functions closures** – technically ప్రతి nested function closure; కానీ "closure" term outer variable capture చేసినప్పుడు అర్థవంతం.

Key Points

- Closure = function + అది పుట్టిన **lexical environment** (outer variables ని గుర్తుపెట్టుకోవడం).
- Outer function return అయినా, referenced variables **బతికే** ఉంటాయి (GC అవ్వవు).
- Uses: **private state** (encapsulation), function factories, memoization, debounce, callbacks.
- **Loop trap:** `var` = shared single variable (final value); `let` = per-iteration binding.
- Closure variable యొక్క **reference** పట్టుకుంటుంది, value copy కాదు.

Interview దృష్టి

Q: Closure అంటే ఏమిటి? A: ఒక inner function తన outer (lexical) scope లోని variables ని access చేయగలగడం – outer function execution ముగిసిపోయినా. Function ని define చేసినప్పుడు అది తన surrounding scope యొక్క reference ని పట్టుకుంటుంది (backpack analogy). దీనివల్ల ఆ variables memory లో బతికే ఉంటాయి. Private state, factories, memoization, callbacks కి పునాది.

Q: `for (var i...) setTimeout(...console.log(i))` ఎందుకు `3,3,3` print చేస్తుంది? Fix? A: `var` function-scoped కాబట్టి అన్ని setTimeout closures **ఒకే** `i` variable ని share చేస్తాయి. Callbacks loop పూర్తయ్యాక (async) run అవుతాయి, అప్పటికి `i = 3`. Fix: `let` వాడు (ప్రతి iteration కి కొత్త block-scoped binding → `0,1,2`), లేదా IIFE తో ప్రతి iteration value ని capture చెయ్యి.

Q: Closures తో private variables ఎలా చేస్తావు? A: Outer function లో variable declare చేసి, దాన్ని access/modify చేసే inner functions ని return చేస్తాను. Variable బయటి scope కి directly కనిపించదు – కేవలం returned methods ద్వారానే. ఉదా: `createCounter` లో `count` private, `increment` / `get` methods మాత్రమే expose. ES6 `#private` fields రాకముందు encapsulation కి ప్రధాన pattern ఇదే.

---

14. this keyword

వివరణ

`this` = JavaScript లో అత్యంత గందరగోళ concept. చాలా భాషల్లో `this` అంటే "current object" – స్థిరం. కానీ JS లో `this` యొక్క value **function ని ఎలా call చేశారో** బట్టి నిర్ణయం అవుతుంది (call-time), function ని ఎక్కడ define చేశారో బట్టి కాదు. అదే variable `this` వేర్వేరు calls లో వేర్వేరు values పొందవచ్చు. దీన్ని అర్థం చేసుకోవడానికి **4 binding rules** ఉన్నాయి.

Real-life Scenario

> **`this` = "నేను" అనే పదం.** "నేను" అని ఎవరు మాట్లాడితే వాళ్ళని సూచిస్తుంది – Surya అంటే Surya, నువ్వు అంటే నువ్వు. పదం ఒక్కటే, కానీ **ఎవరు మాట్లాడారో** బట్టి అర్థం మారుతుంది. అలాగే `this` – **ఎవరు (ఎలా) call చేశారో** బట్టి మారుతుంది.
>
> **Arrow function `this` = ఒక recorded message** – ఇది "నేను" అని ఎప్పుడూ **రికార్డ్ చేసిన వ్యక్తినే** సూచిస్తుంది, ఎవరు play చేసినా. Fixed (lexical).

4 Binding Rules (priority order)

1. Default binding – plain function call. Non-strict = global (`window`), strict = `undefined`.

```
function show() { console.log(this); }
show();  // window (non-strict) / undefined (strict mode)
```

2. Implicit binding – object యొక్క method గా call. `this` = ఆ object (dot కి ఎడమవైపు).

```
const user = {
  name: "Surya",
  greet() { console.log(this.name); }  // this = call చేసిన object
};
user.greet();  // "Surya" (this = user — dot కి ఎడమవైపు)
```

3. Explicit binding – `call` / `apply` / `bind` తో మనం `this` ని force చేయడం (Topic 15).

```
function greet() { console.log(this.name); }
const person = { name: "Palsingh" };
greet.call(person);   // "Palsingh" (this = person)
greet.apply(person);  // "Palsingh"
const bound = greet.bind(person);
bound();              // "Palsingh"
```

4. `new` binding – constructor గా call. `this` = కొత్తగా create అయిన object.

```
function User(name) { this.name = name; } // this = కొత్త object
const u = new User("Surya");
console.log(u.name); // "Surya"
```

Priority: `new` > explicit (bind) > implicit (method) > default.

Arrow Functions – lexical `this` (rule లేదు!)

Arrow functions పైన 4 rules ని **పట్టించుకోవు.** వాటికి **తమదైన `this` లేదు** – అవి ఎప్పుడూ **enclosing (outer) lexical scope** యొక్క `this` ని వాడతాయి. ఇది call time లో మారదు.

```
const obj = {
  name: "Surya",
  regularMethod() {
    console.log(this.name);          // "Surya" (implicit – obj)

    setTimeout(function () {
      console.log(this.name);        // undefined ❗ (default – this =
window/undefined)
    }, 100);

    setTimeout(() => {
      console.log(this.name);        // "Surya" ✅ (arrow – outer this = obj)
    }, 100);
  }
};
obj.show?.();
```

అందుకే callbacks (`setTimeout` , `forEach` , event handlers, promises) లో arrow functions వాడితే `this` సరిగ్గా ఉంటుంది. ఇదే arrow యొక్క ప్రధాన use case.

The "Lost this" Problem (classic bug)

```
const user = {
  name: "Surya",
  greet() { console.log(this.name); }
};

const fn = user.greet;    // method ని extract చేశాం
fn();                     // undefined ❗ – implicit binding పోయింది!
// ఎందుకు? ఇప్పుడు plain function call – dot లేదు → default binding
```

```
// event handlers, callbacks లో ఇది common:
setTimeout(user.greet, 100);        // undefined (this lost)
setTimeout(() => user.greet(), 100); // "Surya" ✅ (arrow wraps)
setTimeout(user.greet.bind(user), 100); // "Surya" ✅ (bind)
```

> **గుండె లోతున:** `this` ని function యొక్క **call-site** నిర్ణయిస్తుంది. `user.greet()` – dot ఉంది → `this = user`. `const fn = user.greet; fn()` – dot లేదు → default (`this` lost). Method ని variable కి assign చేసినా, callback గా pass చేసినా – binding తెగిపోతుంది.

this in different contexts

```
console.log(this);          // module/global — Node: {}, browser: window

class Counter {
  count = 0;
  increment() { this.count++; }  // this = instance
}

// DOM event handler (regular function):
button.addEventListener("click", function () {
  console.log(this);  // this = button (the element) — regular function
});
button.addEventListener("click", () => {
  console.log(this);  // this = outer scope (NOT button) — arrow
});
```

Gotchas (సాధారణ తప్పులు)

- **Method ని extract/pass చేసి `this` పోవడం** – `const f = obj.method; f()` → this lost. `bind` లేదా arrow wrapper.
- **Callback లో regular function వాడి `this` = undefined** – arrow వాడు, లేదా bind.
- **Arrow ని object method గా వాడటం** – `this` object కి bind కాదు (lexical = outer/global).
- **Constructor లో `new` మర్చిపోవడం** – `User("x")` (new లేకుండా) → this = global, global pollution + undefined.
- **DOM handler లో `this` = element అనుకుని arrow వాడటం** – arrow లో this element కాదు.

Key Points

- `this` = **call-time** లో నిర్ణయం (call ఎలా చేశారో బట్టి), define time కాదు.
- **4 rules:** `new` > explicit (`bind` / `call` / `apply`) > implicit (method, dot ఎడమవైపు) > default (global/undefined).
- **Arrow** = lexical `this` (outer scope; call time లో మారదు; rules వర్తించవు). Callbacks కి ideal.
- **Lost this:** method ని extract/pass చేస్తే implicit binding తెగుతుంది → `bind`/arrow.
- Strict mode: default `this` = `undefined` (window కాదు).

Interview దృష్టి

Q: `this` value ఎలా నిర్ణయం అవుతుంది? A: Function ని **ఎలా call చేశారో** బట్టి (call-time), define time కాదు. 4 rules priority order లో: (1) `new Fn()` → కొత్త object; (2) `fn.call/apply/bind(obj)` → obj; (3) `obj.fn()` → obj (dot ఎడమవైపు); (4) plain `fn()` → global (non-strict) / undefined (strict). Arrow functions ఈ rules ని ignore చేసి outer lexical `this` వాడతాయి.

Q: Arrow function `this` ఎందుకు వేరు? A: Arrow functions కి తమదైన `this` binding లేదు. అవి define అయిన చోటి (lexical) outer scope యొక్క `this` ని capture చేస్తాయి, call time లో మారదు. అందుకే `setTimeout`, `forEach`, promises లాంటి callbacks లో arrow వాడితే surrounding object యొక్క `this` preserve అవుతుంది – regular function అయితే default binding వల్ల undefined/window అయ్యేది.

Q: "Lost this" problem అంటే? ఎలా fix? A: Object method ని variable కి assign చేసినా, callback గా pass చేసినా – dot-based implicit binding తెగిపోయి, plain function call అవుతుంది, `this` global/undefined అవుతుంది (`const f = obj.greet; f()` → undefined). Fixes: `obj.greet.bind(obj)`, arrow wrapper `() => obj.greet()`, లేదా class లో fields + arrow methods (auto-bind).

---

15. call, apply, bind

వివరణ

`call`, `apply`, `bind` – ఇవి మూడూ **explicit binding** methods. మనం function యొక్క `this` ని **మనమే force** చేయడానికి వాడతాం (Topic 14 లోని rule 3). తేడా: `call` / `apply` **వెంటనే call చేస్తాయి**, `bind` **కొత్త function return చేస్తుంది** (తర్వాత call కోసం). `call` vs `apply` తేడా కేవలం arguments ఎలా pass చేస్తామో.

Real-life Scenario

> `this` **= ఏ ఇంటి vessel (పాత్ర) తో వండాలో నిర్ణయించడం.** ఒక recipe (function) ఉంది. అది "ఈ ఇంటి పాత్రలతో వండు" అంటుంది (this).
>
> - `call` = "ఈ పాత్రతో, ఈ ingredients (comma-separated) తో – **ఇప్పుడే వండు**."
> - `apply` = అదే, కానీ ingredients ఒక **basket (array) లో** ఇస్తా – ఇప్పుడే వండు.
> - `bind` = "ఈ పాత్రని permanent గా ఈ recipe కి **జోడించి పెట్టు** (కొత్త recipe card ఇవ్వు) – వండటం తర్వాత చేద్దాం."

call vs apply vs bind

```
function introduce(city, country) {
  console.log(`${this.name}, ${city}, ${country}`);
}
const person = { name: "Surya" };
```

```
// —— call: this + arguments (comma-separated), వెంటనే run ——
introduce.call(person, "Hyderabad", "India");
// "Surya, Hyderabad, India"

// —— apply: this + arguments (array గా), వెంటనే run ——
introduce.apply(person, ["Hyderabad", "India"]);
// "Surya, Hyderabad, India"   ("A"pply = "A"rray గుర్తుంచుకో)

// —— bind: this ని bind చేసి కొత్త function return (వెంటనే run కాదు) ——
const boundFn = introduce.bind(person, "Hyderabad");
boundFn("India");  // "Surya, Hyderabad, India" (తర్వాత call, remaining args)
```

Method	Arguments	Execution	Return
`call`	comma-separated	వెంటనే	function result
`apply`	array	వెంటనే	function result
`bind`	comma-separated	తర్వాత (manual call)	కొత్త bound function

వాడకం 1 – Function Borrowing (method అరువు తెచ్చుకోవడం)

ఒక object యొక్క method ని మరో object కోసం అరువు తెచ్చుకోవడం:

```
const arr = { 0: "a", 1: "b", length: 2 }; // array-LIKE object

// Array methods ని అరువు తెచ్చుకోవడం:
const real = Array.prototype.slice.call(arr); // ["a", "b"] — real array!
Array.prototype.forEach.call(arr, x => console.log(x)); // a, b

// arguments (array-like) ని array చేయడానికి classic pattern:
function old() {
  return Array.prototype.slice.call(arguments); // [...arguments] modern
}

// Math.max కి array pass చేయడం (apply):
Math.max.apply(null, [3, 1, 4]); // 4 (Math.max(...[3,1,4]) modern)
```

వాడకం 2 – Partial Application (bind తో)

`bind` కొన్ని arguments ని ముందే fix చేసి, మిగతా వాటిని తర్వాత తీసుకునే కొత్త function ఇస్తుంది:

```
function multiply(a, b) { return a * b; }

const double = multiply.bind(null, 2); // a = 2 fix (this వాడనప్పుడు null)
double(5);   // 10
double(10);  // 20

const triple = multiply.bind(null, 3);
triple(5);   // 15
```

వాడకం 3 – `this` ని stable గా ఉంచడం (React/callbacks)

```
class Button {
  constructor() {
    this.label = "Click";
    // handler ని bind చేయకపోతే callback లో this lost అవుతుంది:
    this.handleClick = this.handleClick.bind(this);
  }
  handleClick() { console.log(this.label); }
}
// element.addEventListener("click", btn.handleClick); // bind లేకపోతే this undefined
```

లోపల ఏం జరుగుతుంది – Polyfills (interview favorite)

`call`, `apply`, `bind` ని manually implement చేయడం classic interview question:

```
// —— myCall ——
Function.prototype.myCall = function (context, ...args) {
  context = context || globalThis;
  const fnKey = Symbol();            // collision-free temp key
  context[fnKey] = this;             // this = ఏ function మీద call చేశామో
  const result = context[fnKey](...args); // method గా call → this = context
  delete context[fnKey];
  return result;
};

// —— myApply (args array గా) ——
Function.prototype.myApply = function (context, args = []) {
  context = context || globalThis;
  const fnKey = Symbol();
  context[fnKey] = this;
  const result = context[fnKey](...args);
  delete context[fnKey];
  return result;
};

// —— myBind (కొత్త function return) ——
Function.prototype.myBind = function (context, ...preset) {
  const fn = this;
  return function (...later) {
    return fn.apply(context, [...preset, ...later]); // partial application
  };
};

// test:
function greet(g) { return `${g}, ${this.name}`; }
greet.myCall({ name: "Surya" }, "Hi"); // "Hi, Surya"
```

> **Polyfill core idea:** `call`/`apply` – function ని context object మీద **temporary property** గా పెట్టి, method గా call చేస్తే `this` automatic గా context అవుతుంది, తర్వాత property delete. `bind` – closure లో context + preset args ని capture చేసి కొత్త function return.

Gotchas (సాధారణ తప్పులు)

- **`bind` వెంటనే call అవుతుందనుకోవడం** – కాదు; కొత్త function return చేస్తుంది, మనం call చేయాలి.
- **`call` vs `apply` గందరగోళం** – `call` = comma args, `apply` = array. ("A"pply = "A"rray).
- **Arrow function ని bind చేయడం** – arrow `this` lexical & fixed; `bind` / `call` / `apply` దాన్ని మార్చవు (silently ignore).
- **Double bind** – ఒకసారి bind అయిన function ని మళ్ళీ bind చేయలేం; మొదటి binding స్థిరం.
- **`bind(null)` non-strict లో** – this = global; strict mode లో null గానే ఉంటుంది.

Key Points

- **explicit `this` binding:** `call` (comma args, now), `apply` (array args, now), `bind` (new fn, later).
- **"A"pply = "A"rray** – గుర్తుంచుకునే trick.
- Uses: **function borrowing** (array-like → array), **partial application** (bind), stable callback `this`.
- **Arrow functions** ఈ మూడింటినీ ignore చేస్తాయి (lexical this fixed).
- Polyfill: context మీద temp property (call/apply); closure తో preset args (bind).

Interview దృష్టి

Q: `call`, `apply`, `bind` తేడా? A: మూడూ function యొక్క `this` ని explicitly set చేస్తాయి. `call(thisArg, arg1, arg2)` – వెంటనే call, args comma-separated. `apply(thisArg, [args])` – వెంటనే call, args array గా. `bind(thisArg, ...args)` – వెంటనే call చేయదు; `this` (మరియు optional preset args) fix చేసిన కొత్త function return చేస్తుంది, దాన్ని తర్వాత call చేయాలి.

Q: Function borrowing అంటే? A: ఒక object కి లేని method ని మరో object/prototype నుండి `call` / `apply` తో అరువు తెచ్చుకోవడం. Classic ఉదాహరణ: `arguments` లేదా array-like objects (NodeList) మీద `Array.prototype.slice.call(arrayLike)` వాడి real array చేయడం – వాటికి array methods లేకపోయినా.

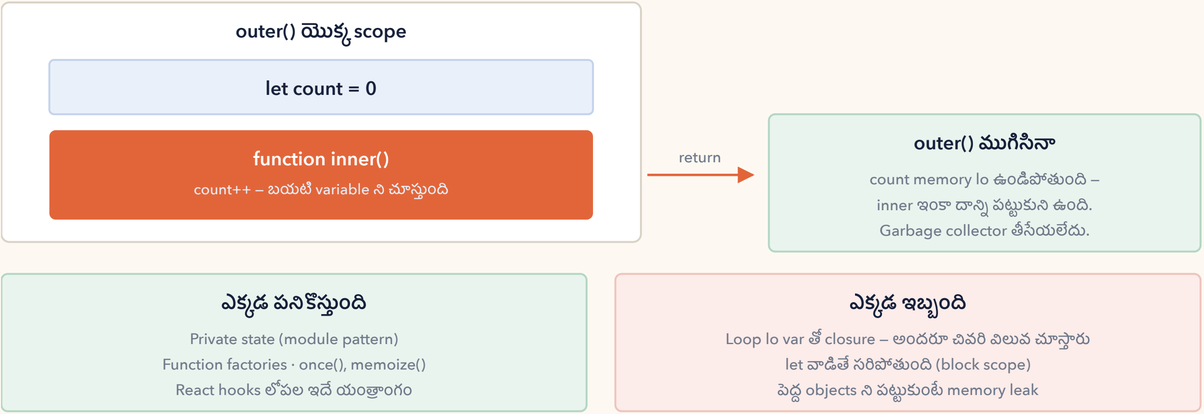
Q: `bind` ని polyfill ఎలా రాస్తావు? A: `Function.prototype.myBind = function(context, ...preset) { const fn = this; return function(...later) { return fn.apply(context, [...preset, ...later]); }; }` – original function (`this`), target context, preset args ని closure లో capture చేసి, call అయినప్పుడు preset + later args కలిపి original ని apply చేసే కొత్త function return చేస్తుంది (partial application support).

---

16. Higher-Order Functions & Callbacks

CLOSURE · SCOPE ని మోసుకెళ్ళే FUNCTION

CLOSURE – function తన పుట్టిన scope ని మోసుకెళ్తుంది



వివరణ

JavaScript లో functions **first-class citizens** కాబట్టి – వాటిని values లా వాడవచ్చు. దీనివల్ల వచ్చే రెండు కీలక concepts:

- **Callback function** = మరో function కి **argument గా pass చేసే** function (తర్వాత "call back" చేయబడుతుంది).
- **Higher-Order Function (HOF)** = ఒక function ని **argument గా తీసుకునే** లేదా function ని **return చేసే** function.

JS లోని `map`, `filter`, `reduce`, `setTimeout`, `addEventListener` – అన్నీ HOFs. Functional programming, async, event handling అన్నిటికీ ఇదే పునాది.

Real-life Scenario

> **Callback = restaurant లో "order ready అయ్యాక నా pager మోగించండి" అని చెప్పడం.** నువ్వు order ఇచ్చి (function call), "అయిపోయాక ఇది చేయ్" అని ఒక pager (callback) ఇస్తావు. Kitchen (HOF) పని పూర్తయ్యాక ఆ pager ని **మోగిస్తుంది** (calls back). నువ్వు అక్కడ నిలబడి wait చేయనవసరం లేదు (non-blocking).
>
> **HOF = ఒక manager** – పనిని ఎలా చేయాలో (callback) నువ్వు చెప్తావు, manager ఆ పనిని ఎప్పుడు/ఎన్నిసార్లు చేయాలో నిర్ణయిస్తాడు.

Callbacks – basics

```
// callback = argument గా pass అయిన function
function greet(name, callback) {
  console.log(`Hi ${name}`);
  callback();  // పని అయ్యాక callback ని "call back"
}
greet("Surya", () => console.log("Done!"));
// "Hi Surya" → "Done!"
```

```
// Synchronous callback (వెంటనే):
[1, 2, 3].forEach(n => console.log(n)); // forEach ప్రతి element కి callback call

// Asynchronous callback (తర్వాత):
setTimeout(() => console.log("2 sec later"), 2000);
button.addEventListener("click", () => console.log("clicked"));
```

Higher-Order Functions – function ని return చేయడం

```
// HOF: function ని return చేస్తుంది
function multiplier(factor) {
  return (n) => n * factor;   // కొత్త function return
}
const double = multiplier(2);
double(5);  // 10

// HOF: function ని argument గా తీసుకుంటుంది
function repeat(n, action) {
  for (let i = 0; i < n; i++) action(i);
}
repeat(3, i => console.log(i)); // 0, 1, 2
```

Array HOFs – daily bread (deep dive Topic 20)

```
const nums = [1, 2, 3, 4, 5];

nums.map(n => n * 2);           // [2,4,6,8,10] — transform (కొత్త array)
nums.filter(n => n % 2 === 0);  // [2, 4] — select
nums.reduce((acc, n) => acc + n, 0); // 15 — accumulate
nums.forEach(n => console.log(n));   // side effects (return లేదు)
nums.find(n => n > 3);          // 4 — మొదటిది
nums.some(n => n > 4);          // true — కనీసం ఒకటి
nums.every(n => n > 0);         // true — అన్నీ
nums.sort((a, b) => b - a);     // [5,4,3,2,1] — comparator callback
```

Custom HOFs – నిజమైన శక్తి

```
// compose — functions ని కుడి నుండి ఎడమకి కలపడం
const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x);
const addOne = x => x + 1;
const double = x => x * 2;
const process = compose(double, addOne); // addOne మొదట, తర్వాత double
process(5); // (5+1)*2 = 12

// pipe — ఎడమ నుండి కుడికి
const pipe = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);
const process2 = pipe(addOne, double); // addOne మొదట, తర్వాత double
process2(5); // 12

// once — function ని ఒక్కసారే run అయ్యేలా (HOF wrapper)
function once(fn) {
  let called = false, result;
  return (...args) => {
    if (!called) { called = true; result = fn(...args); }
```

```
    return result;
  };
}
const init = once(() => console.log("initialized"));
init(); init(); // "initialized" ఒక్కసారే
```

Sync vs Async callbacks

```
// Synchronous — వెంటనే, order లో run
[1, 2].forEach(n => console.log(n));  // 1, 2 (వెంటనే)
console.log("after");                 // after

// Asynchronous — తర్వాత (event loop ద్వారా, Topic 27)
console.log("start");
setTimeout(() => console.log("timeout"), 0);
console.log("end");
// Output: start → end → timeout (timeout 0 అయినా చివర్లో!)
```

> **ముఖ్యం:** అన్ని callbacks async కాదు. `map` / `filter` / `forEach` **synchronous** (వెంటనే run). `setTimeout` / `fetch` /event handlers **asynchronous** (event loop ద్వారా తర్వాత). ఈ తేడా బాగా అర్థం చేసుకో (Topic 27, 28).

Gotchas (సాధారణ తప్పులు)

- **Callback ని `()` తో pass చేయడం** – `setTimeout(fn(), 100)` = `fn` ని వెంటనే call చేసి, దాని **result** ని pass చేస్తుంది! `setTimeout(fn, 100)` (parens లేకుండా) లేదా `setTimeout(() => fn(), 100)`.
- **`this` lost callbacks లో** – method ని callback గా pass చేస్తే this తెగుతుంది (Topic 14); arrow/bind.
- **`forEach` async కాదు** – దానిలో `await` పని చేయదు; `for...of` వాడు (Topic 30).
- **Callback hell** – nested callbacks (Topic 28); Promises/async-await తో పరిష్కారం.
- **Synchronous అనుకుని async result వాడటం** – `let x; fetch(cb => x = data); use(x)` → x undefined.

Key Points

- **Callback** = argument గా pass చేసే function (తర్వాత called). **HOF** = function ని తీసుకునే/return చేసే function.
- First-class functions వల్ల సాధ్యం – pass, return, store.
- Array HOFs: `map` (transform), `filter` (select), `reduce` (accumulate), `find`, `some`, `every`.
- Custom HOFs: `compose`, `pipe`, `once`, `debounce`, `memoize`.
- **Callbacks sync (map/forEach) లేదా async (setTimeout/fetch)** – తేడా తెలుసుకో.

Interview దృష్టి

Q: Higher-order function అంటే? ఉదాహరణ? A: ఒక function ని argument గా తీసుకునే లేదా function ని return చేసే function. JS లో functions first-class కాబట్టి సాధ్యం. ఉదాహరణలు: `map`,

`filter`, `reduce`, `setTimeout`, `addEventListener` (callback తీసుకుంటాయి); `bind`, `multiplier(factor)` (function return చేస్తాయి). ఇవి abstraction, reusability, functional composition కి పునాది.

Q: Callback అంటే? Sync అవుతుందా, async అవుతుందా? A: మరో function కి argument గా pass చేసి, తర్వాత execute చేయబడే function. రెండూ కావచ్చు: `array.map` / `forEach` callbacks synchronous (వెంటనే, order లో run). `setTimeout` / `fetch` /event handlers callbacks asynchronous (event loop ద్వారా, current code పూర్తయ్యాక run). "అన్ని callbacks async" అనేది common misconception.

Q: `setTimeout(fn(), 1000)` లో ఏం తప్పు? A: `fn()` వెంటనే execute అయ్యి, దాని **return value** ని setTimeout కి పంపుతుంది (function reference కాదు). Callback గా function నే pass చేయాలి: `setTimeout(fn, 1000)` (parens లేకుండా) లేదా arguments అవసరమైతే `setTimeout(() => fn(arg), 1000)`.

---

17. Functional Programming (FP)

వివరణ

Functional Programming = software ని **pure functions** compose చేయడం ద్వారా నిర్మించడం – shared mutable state ని, side effects ని తగ్గించడం. JavaScript multi-paradigm కాబట్టి FP ని బాగా support చేస్తుంది (first-class functions, closures, HOFs). Core ideas: **pure functions, immutability, currying, composition, memoization.** ఇవి code ని predictable, testable, bug-resistant చేస్తాయి.

Real-life Scenario

> **Pure function = vending machine.** అదే coin (input) వేస్తే ఎప్పుడూ అదే chocolate (output) వస్తుంది. మధ్యలో ఎవరూ లేరు, బయటి ప్రపంచాన్ని మార్చదు (no side effects). Predictable, testable.
>
> **Impure function = మనసున్న వంటవాడు** – అదే ingredients ఇచ్చినా, మూడ్ (external state), fridge లో మిగిలింది (global) బట్టి వేర్వేరు వంటలు వస్తాయి. ఇంకా వంట చేస్తూ kitchen ని కూడా మారుస్తాడు (side effect). Debug చేయడం కష్టం.

1. Pure Functions

Pure function యొక్క 2 నియమాలు:

1. **అదే input → ఎప్పుడూ అదే output** (deterministic).
2. **No side effects** – బయటి state మార్చదు (global, DOM, network, console, arguments).

```
// ✅ PURE
function add(a, b) { return a + b; }          // అదే in → అదే out, side effect లేదు
```

```
const double = x => x * 2;

// ❌ IMPURE — external state మీద ఆధారం
let count = 0;
function increment() { return ++count; }     // బయటి count మారుతుంది
function now() { return Date.now(); }         // అదే input → వేర్వేరు output
function log(x) { console.log(x); return x; } // side effect (I/O)

// ❌ IMPURE — argument ని mutate చేస్తుంది
function addItem(arr, item) { arr.push(item); return arr; } // input మారింది!
// ✅ PURE version
function addItemPure(arr, item) { return [...arr, item]; }  // కొత్త array
```

Pure functions ఎందుకు మంచివి: testable (mock అవసరం లేదు), cacheable (memoize), parallelizable, predictable, easy to reason about.

2. Immutability

Data ని మార్చకుండా, **కొత్త copy** create చేయడం. FP లో అత్యంత ముఖ్యం.

```
// ❌ Mutating (original మారుతుంది)
const arr = [1, 2, 3];
arr.push(4);          // arr మారింది
arr.sort();           // arr మారింది

// ✅ Immutable (కొత్త array/object)
const newArr = [...arr, 4];             // add
const filtered = arr.filter(x => x > 1); // remove
const mapped = arr.map(x => x * 2);      // transform
const sorted = [...arr].sort();          // copy తర్వాత sort

const user = { name: "Surya", age: 30 };
const updated = { ...user, age: 31 };    // కొత్త object (user unchanged)

// నిజంగా freeze చేయాలంటే (Topic 19):
const frozen = Object.freeze({ a: 1 });
```

> **గుర్తుంచుకో:** mutating methods (`push`, `pop`, `splice`, `sort`, `reverse`) original ని మారుస్తాయి. Non-mutating (`map`, `filter`, `slice`, `concat`, spread) కొత్త array ఇస్తాయి. FP లో non-mutating వాడు (Topic 20 లో పూర్తి list).

3. Currying

ఒక multi-argument function ని, **ఒక్కో argument తీసుకునే functions chain** గా మార్చడం.

```
// Normal:
const add = (a, b, c) => a + b + c;
add(1, 2, 3); // 6

// Curried:
const addCurry = a => b => c => a + b + c;
addCurry(1)(2)(3);  // 6
const add5 = addCurry(5);      // partial: a=5 fixed
```

```
const add5and10 = add5(10);     // a=5, b=10
add5and10(100);                 // 115

// Generic curry helper (interview favorite):
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn.apply(this, args);
    return (...next) => curried(...args, ...next);
  };
}
const sum = curry((a, b, c) => a + b + c);
sum(1)(2)(3);    // 6
sum(1, 2)(3);    // 6
sum(1)(2, 3);    // 6  — flexible!
```

ఎందుకు? Reusable specialized functions, configuration ముందే fix చేయడం, point-free composition.

4. Composition

చిన్న functions ని కలిపి పెద్ద function చేయడం. FP యొక్క గుండె.

```
const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x);
const pipe = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);

const trim = s => s.trim();
const lower = s => s.toLowerCase();
const dashify = s => s.replaceAll(" ", "-");

const slugify = pipe(trim, lower, dashify); // ఎడమ→కుడి
slugify("  Hello World  "); // "hello-world"

// compose = కుడి→ఎడమ (math లా: f(g(x)))
const slugify2 = compose(dashify, lower, trim);
```

5. IIFE (Immediately Invoked Function Expression)

Define అయిన వెంటనే run అయ్యే function. ES6 modules రాకముందు **private scope** create చేయడానికి వాడేవారు.

```
(function () {
  const secret = "hidden";  // ఈ scope బయటికి leak అవ్వదు
  console.log("runs immediately");
})();

// ఆధునిక use: async IIFE (top-level await లేని చోట)
(async () => {
  const data = await fetch("/api");
})();

// module pattern (Topic 47):
const counter = (function () {
  let count = 0;                        // private (closure)
```

```
  return { inc: () => ++count, get: () => count };
})();
```

6. Memoization

Pure function results ని cache చేసి, అదే inputs కి మళ్ళీ compute చేయకుండా.

```
function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);  // cache hit
    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}

const slowSquare = n => { /* heavy */ return n * n; };
const fastSquare = memoize(slowSquare);
fastSquare(5); // compute → 25
fastSquare(5); // cache నుండి → 25 (compute కాదు)
```

> Memoization pure functions తోనే safe – impure అయితే stale cache bugs. (Performance Topic 45).

Gotchas (సాధారణ తప్పులు)

- **"Immutable" అనుకుని `const` వాడటం** – `const` object mutate చేయవచ్చు; నిజమైన immutability కి spread/ `Object.freeze`.
- **Argument mutation** – functions లో passed objects/arrays ని mutate చేయడం hidden side effect.
- **Curry లో `fn.length` rest params తో fail** – rest/default params `length` count అవ్వవు.
- **Memoize impure functions** – `Date`/random/state-dependent → stale results.
- **Deep immutability** – spread shallow copy మాత్రమే; nested objects ఇంకా shared (Topic 19).

Key Points

- **Pure function** = same input → same output + no side effects. Testable, cacheable.
- **Immutability** = mutate చేయకుండా కొత్త copy. Non-mutating methods + spread.
- **Currying** = multi-arg → single-arg chain (`a => b => c`). Partial application.
- **Composition** (`compose` / `pipe`) = చిన్న functions కలపడం. `compose` కుడి→ఎడమ, `pipe` ఎడమ→కుడి.
- **IIFE** = private scope; **Memoization** = pure results cache.

Interview దృష్టి

Q: Pure function అంటే? ఎందుకు ముఖ్యం? A: (1) అదే input కి ఎప్పుడూ అదే output (deterministic), (2) side effects లేవు (external state, DOM, I/O మార్చదు; arguments mutate చేయదు). ముఖ్యం ఎందుకంటే: testable (mocks అవసరం లేదు), memoizable (cacheable), predictable, concurrent-safe, refactor చేయడం సులభం. Impure functions (Date, random, global mutation) reason about చేయడం కష్టం.

Q: Currying అంటే? ఎందుకు వాడతారు? A: Multi-argument function ని ఒక్కో argument తీసుకునే nested functions గా మార్చడం (`f(a,b,c)` → `f(a)(b)(c)`). ఉపయోగం: partial application (కొన్ని args ముందే fix చేసి specialized functions – `add5 = add(5)`), point-free composition, reusability, configuration.

Q: Memoization ఎలా implement చేస్తావు? A: HOF wrapper – arguments నుండి unique key (`JSON.stringify(args)`) తయారు చేసి Map cache లో చూస్తా; cache hit అయితే stored result, లేకపోతే original function call చేసి, result ని cache చేసి return. Pure functions తోనే safe (impure అయితే stale). Fibonacci, expensive computations కి ideal.

Part 3 – Objects, Prototypes & Classes

> JavaScript "objects అన్నిటి మీదా నడుస్తుంది" – arrays, functions, classes అన్నీ objects. ఈ Part లో object internals, property descriptors, arrays deep, destructuring, మరియు JS యొక్క గుండె అయిన **prototype chain** (classes కూడా దీని మీదే నడుస్తాయి) చూస్తాం. `OOPS_Telugu.md` లో OOP, patterns, SOLID ఇంకా లోతుగా ఉన్నాయి – ఇక్కడ language mechanics మీద focus.

---

18. Objects

వివరణ

Object = key-value pairs యొక్క collection. JavaScript లో objects **అన్నిటి పునాది** – arrays, functions, dates, classes అన్నీ లోపల objects. Keys (properties) strings లేదా symbols, values ఏదైనా type. Objects కేవలం data hold చేయవు – property descriptors, getters/setters, computed keys లాంటి శక్తివంతమైన features ఇస్తాయి.

Object create చేయడం

```
// ── Object literal (అత్యంత common) ──
const user = {
  name: "Surya",           // property
  age: 30,
  "full name": "Surya P",  // spaces ఉంటే quotes (bracket access మాత్రమే)
  greet() { return `Hi ${this.name}`; }, // method shorthand (ES6)
};

// ── Access ──
user.name             // "Surya" (dot notation)
user["name"]          // "Surya" (bracket – dynamic keys, spaces కి)
user["full name"]     // "Surya P" (dot పని చేయదు ఇక్కడ)
const key = "age";
user[key]             // 30 (dynamic – variable తో)

// ── Add / update / delete ──
user.email = "s@x.com";   // add
user.age = 31;            // update
delete user.age;          // remove
"name" in user            // true (key ఉందా?)
```

Shorthand & Computed Properties (ES6)

```
const name = "Surya", age = 30;

// Property shorthand — variable పేరు = key అయితే
const user = { name, age };  // { name: "Surya", age: 30 }

// Computed property names — key ని dynamic గా
const key = "score";
const obj = {
  [key]: 100,              // { score: 100 }
  [`${key}_max`]: 200,     // { score_max: 200 }
  ["a" + "b"]: 1,          // { ab: 1 }
};

// Method shorthand
const calc = {
  add(a, b) { return a + b; },     // ES6 (function keyword అవసరం లేదు)
};
```

Getters & Setters

Property లా access అయ్యే, కానీ లోపల function run అయ్యే special properties. Computed values, validation కి.

```
const circle = {
  radius: 5,
  get area() {                    // getter — property లా access
    return Math.PI * this.radius ** 2;
  },
  set diameter(d) {               // setter — property లా assign
    this.radius = d / 2;
  }
};

circle.area;         // 78.53... (parens లేవు! property లా)
circle.diameter = 20; // setter run → radius = 10
circle.area;         // 314.15...
```

> **గుర్తుంచుకో:** getter/setter ని **method లా `()` తో call చేయవు** — property లా access చేస్తావు (`circle.area`, `circle.diameter = 20`). లోపల function run అవుతుంది. API ని property లా simple గా ఉంచి, లోపల logic పెట్టడానికి ఉపయోగం.

లోపల ఏం జరుగుతుంది — Property Descriptors

ప్రతి property కి కేవలం value కాదు — **hidden metadata (descriptor)** ఉంటుంది: `writable`, `enumerable`, `configurable`. ఇవి property behavior ని control చేస్తాయి.

```
const obj = { name: "Surya" };
Object.getOwnPropertyDescriptor(obj, "name");
// { value: "Surya", writable: true, enumerable: true, configurable: true }
```

Descriptor	అర్థం	default (literal)
`value`	property value	(ఇచ్చినది)
`writable`	value మార్చవచ్చా?	`true`
`enumerable`	`for...in` / `Object.keys` లో కనిపిస్తుందా?	`true`
`configurable`	delete/redefine చేయవచ్చా?	`true`

```
// defineProperty — descriptor ని manually control
const config = {};
Object.defineProperty(config, "API_KEY", {
  value: "secret123",
  writable: false,       // read-only
  enumerable: false,     // for...in/keys లో దాక్కుంటుంది
  configurable: false,   // delete/redefine కుదరదు
});
config.API_KEY = "hack"; // silent fail (strict = error)
config.API_KEY;          // "secret123" (మారలేదు)
Object.keys(config);     // [] — enumerable: false కాబట్టి hidden!
```

> Literal తో create చేసిన properties అన్నీ default `true` (writable/enumerable/configurable). `defineProperty` వాడితేనే వీటిని lock చేయవచ్చు – libraries, frameworks internal properties దాచడానికి వాడతాయి.

Objects పోలిక (reference – Topic 4)

```
const a = { x: 1 };
const b = { x: 1 };
a === b;              // false (వేర్వేరు references)
a === a;              // true

// content పోలిక కి — manual లేదా JSON (limited):
JSON.stringify(a) === JSON.stringify(b); // true (key order same అయితే)
```

Gotchas (సాధారణ తప్పులు)

- **Getter ని `()` తో call చేయడం** – `circle.area()` = TypeError (area number, function కాదు).
- **Object keys ఎప్పుడూ strings/symbols** – `obj[1]` మరియు `obj["1"]` ఒకటే (number key → string).
- **`in` vs `hasOwnProperty`** – `in` inherited properties కూడా చూస్తుంది; own కి `Object.hasOwn(obj, key)` (ES2022).
- **`delete` slow + arrays లో holes** – arrays కి `splice` / `filter` వాడు, `delete` కాదు.
- **Non-existent property = `undefined`** (error కాదు) – typos silent bugs; optional chaining `?.` deep access కి.

Key Points

- Object = key(string/symbol)-value pairs; JS లో **అన్నిటి పునాది.**
- Access: dot (static) vs bracket (dynamic/spaces). Keys → strings.
- **Shorthand** `{name}`, **computed** `{[key]: v}`, **method** `{fn(){}}` (ES6).
- **Getters/setters** = property లా access, లోపల function.
- **Descriptors** (`writable`/`enumerable`/`configurable`) – `defineProperty` తో control; literal defaults `true`.

Interview దృష్టి

Q: Dot vs bracket notation ఎప్పుడు? A: Dot (`obj.name`) – key ముందే తెలిసి, valid identifier అయినప్పుడు (clean, common). Bracket (`obj[key]`) – key dynamic (variable), spaces/special chars ఉన్నప్పుడు (`obj["full name"]`), లేదా computed. Object keys ఎప్పుడూ strings/symbols గా coerce అవుతాయి (`obj[1]` === `obj["1"]`).

Q: Property descriptors అంటే? `writable` vs `enumerable` vs `configurable`? A: ప్రతి property కి ఉండే hidden metadata. `writable` – value మార్చవచ్చా. `enumerable` – `for...in`/`Object.keys`/spread లో కనిపిస్తుందా. `configurable` – delete/redefine చేయవచ్చా. Literal properties default అన్నీ `true`; `Object.defineProperty` తో read-only/hidden/locked properties create చేయవచ్చు (frameworks internal state దాచడానికి వాడతాయి).

Q: Getters/setters ఎందుకు? A: Property లా simple access ఇస్తూ, లోపల logic (computed values, validation, side effects) run చేయడానికి. `obj.fullName` – getter లోపల `firstName + lastName` compute చేస్తుంది; `obj.age = -5` – setter validation చేస్తుంది. API ని property-based గా clean గా ఉంచుతుంది, method calls (`getFullName()`) లేకుండా.

---

19. Object Utilities

వివరణ

`Object` global పై అనేక static methods ఉన్నాయి – iteration (`keys`/`values`/`entries`), copy/merge (`assign`, spread), immutability (`freeze`/`seal`), inheritance (`create`). వీటిని రోజూ వాడతాం. దీనితో పాటు **shallow vs deep copy** అనే కీలక concept – bugs కి పెద్ద కారణం.

Iteration Methods

```
const user = { name: "Surya", age: 30, city: "Hyd" };

Object.keys(user);     // ["name", "age", "city"]
Object.values(user);   // ["Surya", 30, "Hyd"]
Object.entries(user);  // [["name","Surya"], ["age",30], ["city","Hyd"]]

// entries → iterate (for...of తో):
for (const [key, value] of Object.entries(user)) {
  console.log(`${key}: ${value}`);
```

```
}

// entries → object మార్చి, తిరిగి object:
const upper = Object.fromEntries(
  Object.entries(user).map(([k, v]) => [k, String(v).toUpperCase()])
);

// Object ని Map చేయడం & వెనక్కి:
const map = new Map(Object.entries(user));
const obj = Object.fromEntries(map);
```

Copy & Merge

```
// ── Spread (shallow copy + merge) — modern ──
const copy = { ...user };                    // shallow copy
const merged = { ...user, age: 31, x: 1 };   // merge + override
const combined = { ...obj1, ...obj2 };       // obj2 గెలుస్తుంది conflicts లో

// ── Object.assign (target ని mutate చేస్తుంది!) ──
Object.assign(target, source1, source2);     // target కి copy (mutates target)
const copy2 = Object.assign({}, user);       // {} target → shallow copy
```

లోపల ఏం జరుగుతుంది – Shallow vs Deep Copy (అత్యంత ముఖ్యం)

Spread మరియు `Object.assign` **shallow copy** మాత్రమే – top-level primitives copy అవుతాయి, కానీ **nested objects/arrays reference share** అవుతాయి!

```
const original = {
  name: "Surya",
  address: { city: "Hyd" }   // nested object
};

// ── Shallow copy ──
const shallow = { ...original };
shallow.name = "Palsingh";        // ✅ original.name మారలేదు (primitive)
shallow.address.city = "Delhi";   // ❗ original.address.city కూడా "Delhi"!
console.log(original.address.city); // "Delhi" — nested shared!

// ── Deep copy — పరిష్కారాలు ──

// 1. structuredClone (ES2022 — best, built-in)
const deep = structuredClone(original);
deep.address.city = "Mumbai";
original.address.city; // unchanged ✅ (Delhi)

// 2. JSON (limited — functions, undefined, Date, Symbol పోతాయి)
const deep2 = JSON.parse(JSON.stringify(original));

// 3. libraries: lodash cloneDeep
```

> **గుండె లోతున:** `{...obj}` = **shallow** (top-level మాత్రమే స్వతంత్రం; nested shared). Nested data ని safely copy చేయాలంటే `structuredClone(obj)` (ES2022, built-in, best) వాడు. JSON trick పని చేస్తుంది కానీ functions, `undefined`, `Date`, `Map`, `Symbol`, circular refs ని పోగొడుతుంది.

structuredClone vs JSON – పోలిక

అంశం	`structuredClone`	`JSON.parse(JSON.stringify())`
Nested deep copy	✅	✅
Functions	❌ DataCloneError (clone చేయలేదు)	❌ silently పోతాయి
`undefined` values	✅	❌ పోతాయి
`Date`	✅ (Date object)	❌ string అవుతుంది
`Map` / `Set`	✅	❌ `{}` అవుతుంది
Circular references	✅	❌ Error
Performance	వేగం	నెమ్మది

Immutability – freeze & seal

```
// ── Object.freeze – పూర్తిగా lock (no add/delete/modify) ──
const config = Object.freeze({ api: "x.com", timeout: 5000 });
config.api = "hack";      // silent fail (strict = error)
config.newProp = 1;       // silent fail
delete config.api;        // silent fail
Object.isFrozen(config);  // true

// ⚠️ freeze SHALLOW – nested objects freeze అవ్వవు!
const obj = Object.freeze({ nested: { x: 1 } });
obj.nested.x = 99;        // ✅ works! (nested frozen కాదు)
// deep freeze కి recursion అవసరం:
function deepFreeze(o) {
  Object.keys(o).forEach(k => {
    if (typeof o[k] === "object" && o[k] !== null) deepFreeze(o[k]);
  });
  return Object.freeze(o);
}

// ── Object.seal – add/delete కుదరదు, కానీ existing modify OK ──
const sealed = Object.seal({ x: 1 });
sealed.x = 2;      // ✅ modify OK
sealed.y = 3;      // ❌ add కుదరదు
delete sealed.x;   // ❌ delete కుదరదు
```

Method	Modify existing	Add new	Delete
`freeze`	❌	❌	❌

Method	Modify existing	Add new	Delete
`seal`	✅	❌	❌
`preventExtensions`	✅	❌	✅

Object.create – prototype తో object (Topic 22)

```
const proto = { greet() { return `Hi ${this.name}`; } };
const obj = Object.create(proto);   // proto ని prototype గా
obj.name = "Surya";
obj.greet();                        // "Hi Surya" (prototype నుండి)
Object.getPrototypeOf(obj) === proto; // true

// null prototype (pure dictionary, no inherited methods):
const dict = Object.create(null);   // toString/hasOwnProperty కూడా లేవు
```

Gotchas (సాధారణ తప్పులు)

- **Spread/assign ని deep copy అనుకోవడం** – shallow మాత్రమే; nested shared → `structuredClone`.
- `Object.freeze` **deep అనుకోవడం** – shallow; nested mutate అవుతుంది; `deepFreeze`.
- `Object.assign(target, ...)` **target ని mutate చేస్తుంది** – copy కి `Object.assign({}, src)`.
- **JSON deep clone లో data loss** – Date → string, functions/undefined పోతాయి, circular → error.
- `Object.keys` **order** – integer keys ascending, తర్వాత string keys insertion order (subtle).

Key Points

- Iterate: `keys` / `values` / `entries` + `fromEntries` (entries ↔ object).
- Copy: spread `{...o}` / `Object.assign` = **shallow.** Deep = **`structuredClone`** (best).
- **Nested = shared** in shallow copy – top-level ఒక్కటే స్వతంత్రం.
- **`freeze`** (fully lock, shallow) vs **`seal`** (modify OK, no add/delete).
- `Object.create(proto)` = prototype-based object; `Object.create(null)` = pure dict.

Interview దృష్టి

Q: Shallow copy vs deep copy? `{...obj}` ఏది? A: Shallow copy – top-level properties మాత్రమే copy; nested objects/arrays reference share అవుతాయి (ఒకటి మారితే రెండూ మారతాయి). `{...obj}` మరియు `Object.assign` shallow. Deep copy – nested levels అన్నీ స్వతంత్రంగా copy. Deep కి `structuredClone(obj)` (ES2022, built-in, best), లేదా limited `JSON.parse(JSON.stringify(obj))`, లేదా lodash `cloneDeep`.

Q: `Object.freeze` deep గా freeze చేస్తుందా? A: లేదు, shallow మాత్రమే – top-level properties ని lock చేస్తుంది, కానీ nested objects mutate అవుతాయి (`frozen.nested.x = 99` పని చేస్తుంది). Deep freeze కి nested objects మీద recursively `Object.freeze` apply చేయాలి (`deepFreeze` helper).

Q: `structuredClone` vs `JSON.parse(JSON.stringify())`? A: రెండూ deep copy. `structuredClone` — built-in, వేగం, `Date` / `Map` / `Set` /circular references/ `undefined` ని సరిగ్గా handle చేస్తుంది (కానీ functions clone చేయలేదు). JSON trick – `Date` ని string చేస్తుంది, functions/ `undefined` / `Symbol` పోగొడుతుంది, circular references కి error. Modern code లో `structuredClone` preferred.

---

20. Arrays

వివరణ

Array = ordered collection. JS లో arrays నిజానికి **special objects** (integer keys + `length`). చాలా built-in methods ఉన్నాయి – వాటిని **mutating** (original మారుస్తాయి) vs **non-mutating** (కొత్త array ఇస్తాయి) అని విభజించడం అత్యంత ముఖ్యం. `map` / `filter` / `reduce` deep, `sort` యొక్క పెద్ద trap ని చూద్దాం.

Mutating vs Non-mutating (గుండె లోతున గుర్తుంచుకో)

Mutating (original మారుస్తాయి)	Non-mutating (కొత్త array)
`push` / `pop` (చివర)	`map`
`shift` / `unshift` (మొదట)	`filter`
`splice` (add/remove మధ్యలో)	`slice`
`sort` ⚠️	`concat`
`reverse` ⚠️	`flat` / `flatMap`
`fill` / `copyWithin`	spread `[...arr]`

```
const arr = [3, 1, 2];

// ── Mutating — original మారుతుంది ──
arr.push(4);        // [3,1,2,4], returns new length (4)
arr.pop();          // [3,1,2], returns removed (4)
arr.sort();         // arr మారింది! [1,2,3]
arr.reverse();      // arr మారింది!
arr.splice(1, 1);   // index 1 నుండి 1 element తీసేసింది (mutates)

// ── Non-mutating — కొత్త array ──
const doubled = arr.map(x => x * 2);     // arr unchanged
const sorted = [...arr].sort();          // copy తర్వాత sort (safe)
const sliced = arr.slice(0, 2);          // కొత్త array
```

> **FP/React లో non-mutating వాడు.** State ని mutate చేస్తే re-render, bugs. `arr.sort()` కి బదులు `[...arr].sort()`.

map / filter / reduce — deep

```
const nums = [1, 2, 3, 4, 5];

// ── map: ప్రతి element ని transform (అదే length కొత్త array) ──
nums.map(x => x * 2);              // [2,4,6,8,10]
nums.map((x, i) => `${i}:${x}`);  // index కూడా: ["0:1","1:2",...]

// ── filter: condition true అయిన elements (subset) ──
nums.filter(x => x % 2 === 0);    // [2, 4]
nums.filter(Boolean);             // falsy తీసేయడం trick

// ── reduce: array → ఒకే value (accumulate) ──
nums.reduce((acc, x) => acc + x, 0);  // 15 (sum; 0 = initial)
nums.reduce((acc, x) => acc * x, 1);  // 120 (product)
nums.reduce((max, x) => x > max ? x : max); // 5 (initial లేకపోతే [0])

// reduce శక్తి — దాదాపు అన్నీ reduce తో చేయవచ్చు:
// group by:
const people = [{age:20,name:"a"},{age:20,name:"b"},{age:30,name:"c"}];
people.reduce((groups, p) => {
  (groups[p.age] ??= []).push(p.name);  // ??= nullish assign
  return groups;
}, {}); // { 20: ["a","b"], 30: ["c"] }

// count occurrences:
["a","b","a"].reduce((c, x) => { c[x]=(c[x]||0)+1; return c; }, {});
// { a: 2, b: 1 }

// chaining (map + filter + reduce):
nums.filter(x => x % 2 === 0).map(x => x * 10).reduce((a, b) => a + b, 0);
// [2,4] → [20,40] → 60
```

> **reduce లో initial value ఎప్పుడూ ఇవ్వు** (`, 0` / `, {}`). లేకపోతే empty array మీద `reduce` **TypeError** throws, మరియు మొదటి element ని accumulator గా వాడుతుంది (subtle bugs).

Search & Check methods

```
const arr = [10, 20, 30, 20];
arr.indexOf(20);              // 1 (మొదటిది; లేకపోతే -1)
arr.lastIndexOf(20);          // 3
arr.includes(20);             // true (NaN కూడా match — indexOf కాదు!)
arr.find(x => x > 15);        // 20 (మొదటి match value)
arr.findIndex(x => x > 15);   // 1 (index)
arr.findLast(x => x > 15);    // 20 (ES2023, చివరి నుండి)
arr.some(x => x > 25);        // true (కనీసం ఒకటి)
arr.every(x => x > 5);        // true (అన్నీ)

// includes vs indexOf తో NaN:
```

```
[NaN].includes(NaN);          // true ✅
[NaN].indexOf(NaN);           // -1   ❌ (indexOf === వాడుతుంది, NaN!==NaN)
```

flat / flatMap

```
[1, [2, [3, [4]]]].flat();      // [1, 2, [3, [4]]] (1 level default)
[1, [2, [3, [4]]]].flat(2);     // [1, 2, 3, [4]]
[1, [2, [3, [4]]]].flat(Infinity); // [1,2,3,4] (పూర్తిగా)

[1, 2, 3].flatMap(x => [x, x*2]); // [1,2,2,4,3,6] (map + flat(1))
```

లోపల ఏం జరుగుతుంది — `sort` యొక్క పెద్ద trap

`sort` default గా elements ని **strings గా మార్చి**, Unicode order లో sort చేస్తుంది — numbers కి కూడా! ఇది classic bug.

```
[10, 1, 2, 20, 3].sort();
// [1, 10, 2, 20, 3] ❗❗ — "10" < "2" (string comparison!)

// ✅ సరైన మార్గం — comparator function:
[10, 1, 2, 20, 3].sort((a, b) => a - b);  // [1,2,3,10,20] (ascending)
[10, 1, 2, 20, 3].sort((a, b) => b - a);  // [20,10,3,2,1] (descending)

// comparator: negative → a ముందు, positive → b ముందు, 0 → same
// objects sort:
users.sort((a, b) => a.age - b.age);       // age ascending
users.sort((a, b) => a.name.localeCompare(b.name)); // strings proper

// ⚠️ sort mutates! కొత్త array కావాలంటే:
const sorted = [...arr].sort((a, b) => a - b);
// లేదా toSorted (ES2023 — non-mutating!):
const sorted2 = arr.toSorted((a, b) => a - b);
```

> **ఎప్పుడూ `sort` కి comparator ఇవ్వు** (`(a,b) => a-b`). Default string-based sort numbers ని పాడు చేస్తుంది. ES2023 లో non-mutating `toSorted`, `toReversed`, `toSpliced`, `with` వచ్చాయి.

Array create & convert

```
Array.of(1, 2, 3);            // [1,2,3]
Array.from("abc");            // ["a","b","c"] (iterable → array)
Array.from({length: 3}, (_, i) => i); // [0,1,2] (map తో)
Array.from(new Set([1,1,2])); // [1,2] (dedupe)
[...new Set([1,1,2])];        // [1,2] (spread dedupe)
new Array(3).fill(0);         // [0,0,0]
Array(5).keys();              // iterator 0..4
```

Gotchas (సాధారణ తప్పులు)

- `sort()` **comparator లేకుండా numbers** — string sort ([1,10,2]). ఎప్పుడూ `(a,b) => a-b`.

- `sort` / `reverse` / `splice` **mutate** – React state లో bug; `[...arr]` copy లేదా `toSorted`.
- `reduce` **initial value లేకపోవడం** – empty array TypeError; ఎప్పుడూ initial ఇవ్వు.
- `map` ని `forEach` **కి బదులు side-effects కి వాడటం** – map కొత్త array create చేస్తుంది (waste); side effects కి forEach/for-of.
- `indexOf(NaN)` = -1 ఎప్పుడూ; NaN కి `includes` వాడు.
- `delete arr[2]` – hole వదిలేస్తుంది (length అలాగే); `splice` వాడు.

Key Points

- Arrays = special objects. **Mutating** (`push/pop/splice/sort/reverse`) vs **non-mutating** (`map/filter/slice/concat/flat`).
- `map` transform, `filter` select, `reduce` accumulate (initial value ఇవ్వు!).
- `sort` **default = string sort** – numbers కి `(a,b) => a-b` comparator తప్పనిసరి; `sort` mutates.
- `includes` NaN-safe, `indexOf` కాదు. `find` / `some` / `every` / `flat` / `flatMap`.
- ES2023 non-mutating: `toSorted`, `toReversed`, `toSpliced`, `with`.

Interview దృష్టి

Q: `map`, `filter`, `reduce` తేడా? A: `map` – ప్రతి element ని transform చేసి **అదే length** కొత్త array. `filter` – condition true అయిన elements తో **subset** కొత్త array. `reduce` – array ని ఒక accumulator ద్వారా **ఒకే value** (sum, object, ఏదైనా) కి కుదించడం. మూడూ non-mutating, callback-based. `reduce` అత్యంత powerful – map/filter కూడా reduce తో చేయవచ్చు.

Q: `[10, 1, 2].sort()` ఎందుకు `[1, 10, 2]`? A: Default `sort` elements ని strings గా convert చేసి Unicode order లో పోల్చుతుంది. `"10" < "2"` (మొదటి char "1" < "2"). Numbers ని సరిగ్గా sort చేయాలంటే comparator ఇవ్వాలి: `sort((a, b) => a - b)` (ascending). ఇంకా `sort` original array ని mutate చేస్తుంది – safe కి `[...arr].sort(...)` లేదా `toSorted`.

Q: Array ని mutate చేసే vs చేయని methods? A: Mutating (original మారుస్తాయి): `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`. Non-mutating (కొత్త array): `map`, `filter`, `slice`, `concat`, `flat`, `flatMap`, spread. Immutable state (React, FP) లో non-mutating వాడాలి; ES2023 లో mutating వాటికి non-mutating variants (`toSorted`, `toReversed`, `toSpliced`, `with`) వచ్చాయి.

---

21. Destructuring

వివరణ

Destructuring = arrays/objects నుండి values ని విడగొట్టి, వేర్వేరు variables లోకి ఒకే line లో తీయడం. ES6 feature. Code ని concise, readable చేస్తుంది – function params, API responses, swapping అన్నిటిలో రోజూ వాడతాం.

Real-life Scenario

> **Destructuring = ఒక parcel (డబ్బా) విప్పి, లోపలి వస్తువులని నేరుగా అరల్లో పెట్టడం.** పాత పద్ధతి – డబ్బా తీసుకుని, "ఇది phone, ఇది charger, ఇది cable" అని ఒక్కొక్కటి తీయడం (`const phone = box.phone`). Destructuring – డబ్బా విప్పగానే `const { phone, charger, cable } = box` – అన్నీ ఒకేసారి సరైన అరల్లోకి!

Array Destructuring

```
const arr = [1, 2, 3, 4, 5];

const [a, b] = arr;              // a=1, b=2
const [first, , third] = arr;    // skip: first=1, third=3 (2 skip)
const [x, ...rest] = arr;        // x=1, rest=[2,3,4,5] (rest – చివర్లో)
const [p = 10, q = 20] = [1];    // defaults: p=1, q=20 (missing → default)

// Swapping (temp variable లేకుండా!):
let m = 1, n = 2;
[m, n] = [n, m];                 // m=2, n=1

// Function return (multiple values):
function minMax(nums) { return [Math.min(...nums), Math.max(...nums)]; }
const [min, max] = minMax([3, 1, 4]); // min=1, max=4
```

Object Destructuring

```
const user = { name: "Surya", age: 30, city: "Hyd" };

const { name, age } = user;              // name="Surya", age=30
const { city, country = "India" } = user; // default: country="India" (missing)

// Rename (rare కానీ ముఖ్యం):
const { name: userName, age: userAge } = user;
// userName="Surya", userAge=30 (name/age variables లేవు!)

// Rename + default కలిపి:
const { role: userRole = "guest" } = user; // userRole="guest"

// Rest:
const { name: n, ...others } = user;     // n="Surya", others={age:30, city:"Hyd"}
```

Nested Destructuring

```
const data = {
  user: {
    name: "Surya",
    address: { city: "Hyd", pin: 500001 }
  },
  scores: [90, 85, 88]
};

const {
```

```
  user: { name, address: { city } },   // deep nested
  scores: [firstScore]                  // array లోపల
} = data;
console.log(name, city, firstScore);    // "Surya" "Hyd" 90

// ⚠️ user, address ఇక్కడ variables కావు — కేవలం "path"!
// console.log(user);    // ❌ ReferenceError
```

Nested trap: `const { user: { name } }` లో `user` ఒక variable **కాదు** – అది కేవలం "పథం" (path to reach name). చివరి పేరు (`name`) మాత్రమే variable. మధ్యలో ఏదైనా `null` / `undefined` అయితే → TypeError (deep default లేదా `?.` అవసరం).

Parameter Destructuring (అత్యంత powerful)

Function parameters ని నేరుగా destructure చేయడం – named/options arguments కి ideal.

```
// ── Object params (named arguments pattern) ──
function createUser({ name, age = 18, role = "user" }) {
  return `${name}, ${age}, ${role}`;
}
createUser({ name: "Surya", age: 30 });  // "Surya, 30, user"
createUser({ name: "A" });               // "A, 18, user" (defaults)

// ── Default = {} — argument missing అయినా crash కాకుండా ──
function config({ timeout = 5000 } = {}) { return timeout; }
config();          // 5000 ✅ (= {} లేకపోతే: destructuring undefined → TypeError!)
config({ timeout: 1000 }); // 1000

// ── Array params ──
function dist([x1, y1], [x2, y2]) {
  return Math.hypot(x2 - x1, y2 - y1);
}
dist([0, 0], [3, 4]); // 5

// ── React లో common ──
function Button({ label, onClick, disabled = false }) { /* ... */ }
```

వాడకాలు (real-world)

```
// API response:
const { data, status } = await fetch(url).then(r => r.json());

// Import (named):
// import { useState, useEffect } from "react";

// Loop with entries:
for (const [key, value] of Object.entries(obj)) { }

// Default params with destructuring:
const { page = 1, limit = 10 } = req.query;
```

Gotchas (సాధారణ తప్పులు)

- **Object destructuring statement start లో `{}` ** – `{ a } = obj` (let/const లేకుండా) → block అనుకుంటుంది; `({ a } = obj)` parens.
- **Nested లో మధ్య variable అనుకోవడం** – `{ user: { name } }` లో `user` variable కాదు (path).
- **Default `= {}` మర్చిపోవడం param లో** – `function f({ x }) {}` కి `f()` → TypeError (undefined destructure). `f({ x } = {})`.
- **Default trigger – `undefined` మాత్రమే** – `null` default ని trigger చేయదు (`const { x = 5 } = { x: null }` → x=null).
- **Rename గందరగోళం** – `{ name: userName }` – `userName` variable, `name` కాదు.

Key Points

- **Array** = position-based (`[a, b]`), **Object** = key-based (`{name, age}`).
- **Defaults** (`= val`) missing/ `undefined` కి; **rename** (`name: newName`); **rest** (`...others`).
- **Nested**: మధ్య పేర్లు paths, variables కావు; deep null → TypeError.
- **Param destructuring** = named/options args pattern; `= {}` **default** తప్పనిసరి (missing arg safety).
- Swapping: `[a, b] = [b, a]` (no temp).

Interview దృష్టి

Q: Destructuring అంటే? ఉపయోగాలు? A: Arrays/objects నుండి values ని విడగొట్టి variables లోకి తీసే syntax. Array position-based (`const [a, b] = arr`), object key-based (`const { name } = obj`). Defaults, renaming, rest, nesting support. Uses: function parameters (named/options args), API responses, imports, swapping (`[a,b]=[b,a]`), `Object.entries` loops. Code ని concise & readable చేస్తుంది.

Q: `function f({ x } = {})` లో `= {}` ఎందుకు? A: Argument పంపకపోతే (`f()`), `x` ని `undefined` నుండి destructure చేయాలి – అది TypeError throws. `= {}` default parameter argument missing అయినప్పుడు empty object ఇస్తుంది, దాని నుండి `x` = undefined (లేదా inner default) సురక్షితంగా వస్తుంది, crash కాదు. Options-object pattern కి essential.

Q: `const { a: b } = obj` ఏం చేస్తుంది? A: Rename – `obj.a` value ని `b` అనే variable లోకి తీస్తుంది. `a` variable create అవ్వదు; `b` మాత్రమే. Colon ఎడమవైపు = object key, కుడివైపు = కొత్త variable పేరు. Naming conflicts తప్పించడానికి లేదా clearer names కి ఉపయోగం.

---

22. Prototypes & Prototypal Inheritance

వివరణ

ఇది JavaScript యొక్క **అత్యంత core, అత్యంత misunderstood** concept. చాలా భాషలు **class-based inheritance** వాడతాయి. JavaScript **prototype-based** – ప్రతి object కి ఒక **hidden link** (`[[Prototype]]`) ఉంటుంది, అది మరో object ని point చేస్తుంది. ఒక property/method object లో

దొరక్కపోతే, JS ఈ link ద్వారా **prototype chain** పైకి వెతుకుతుంది. ES6 classes కూడా లోపల ఈ mechanism మీదే నడుస్తాయి (Topic 23).

Real-life Scenario

> **Prototype chain = అప్పు అడిగే గొలుసు.** నీకు ₹100 కావాలి (property lookup). నీ జేబు చూస్తావు (own property) – లేదు. అన్నయ్య దగ్గరకి వెళ్తావు (`__proto__`) – లేదు. నాన్న దగ్గరకి (అన్నయ్య `__proto__`) – ఉంది! తీసుకుంటావు. ఎక్కడా లేకపోతే (`null` = chain చివర), "డబ్బు లేదు" (`undefined`).
>
> **ముఖ్యం:** నాన్న డబ్బు నీ జేబులోకి **copy అవ్వదు** – నువ్వు ప్రతిసారి నాన్న దగ్గరకి వెళ్తావు (shared, live link). నాన్న డబ్బు మారిస్తే, నీకూ ఆ కొత్త amount కనిపిస్తుంది.

`__proto__` vs `prototype` – అత్యంత గందరగోళం

ఈ రెండూ వేర్వేరు! ఇది clear అయితే prototypes 80% అర్థమైనట్టే:

- `prototype` – కేవలం **functions** మీద ఉండే property. `new` తో objects create చేసినప్పుడు, ఆ objects యొక్క `__proto__` గా ఇది set అవుతుంది. అంటే "నా instances కి ఏ prototype ఇవ్వాలి" అనే **blueprint.**
- `__proto__` (లేదా `[[Prototype]]`) – **ప్రతి object** మీద ఉండే actual link – దాని prototype కి. Lookup ఈ link వెంట జరుగుతుంది.

```
function Dog(name) { this.name = name; }
Dog.prototype.bark = function () { return `${this.name}: Woof`; };

const d = new Dog("Tommy");

d.__proto__ === Dog.prototype;   // true ✅ – object లింక్ = function యొక్క prototype
Dog.prototype.constructor === Dog; // true
d.bark();                        // "Tommy: Woof" (own లో లేదు → prototype లో దొరికింది)

// Modern (preferred over __proto__):
Object.getPrototypeOf(d) === Dog.prototype; // true
```

> **సూత్రం:** `prototype` = functions మీద (blueprint for instances). `__proto__` = objects మీద (actual link used for lookup). `d.__proto__ === Dog.prototype`.

Prototype Chain – lookup

```
const arr = [1, 2, 3];

// arr.map ఎక్కడ నుండి వస్తుంది?
arr.map;  // arr own లో లేదు
// arr.__proto__ === Array.prototype (map ఇక్కడ ఉంది) ✅
// Array.prototype.__proto__ === Object.prototype (toString, hasOwnProperty)
// Object.prototype.__proto__ === null (chain చివర)

arr.__proto__ === Array.prototype;              // true
```

```
arr.__proto__.__proto__ === Object.prototype;    // true
arr.__proto__.__proto__.__proto__ === null;      // true (top)

// hasOwnProperty — own vs inherited తేడా:
arr.hasOwnProperty("map");   // false (inherited)
arr.hasOwnProperty(0);       // true  (own — index)
Object.hasOwn(arr, 0);       // true (ES2022, modern)
```

Object.create – direct prototypal inheritance

```
const animal = {
  eat() { return `${this.name} తింటోంది`; },
  sleep() { return `${this.name} నిద్రపోతోంది`; }
};

// animal ని prototype గా పెట్టి object create:
const dog = Object.create(animal);
dog.name = "Tommy";
dog.eat();   // "Tommy తింటోంది" (animal నుండి inherited)
Object.getPrototypeOf(dog) === animal; // true

// Chain build చేయడం:
const puppy = Object.create(dog);
puppy.name = "Chinnu";
puppy.eat(); // "Chinnu తింటోంది" (dog → animal వరకు వెతికింది)
```

Constructor Functions (classes రాకముందు pattern)

```
function Person(name, age) {
  this.name = name;        // own properties (ప్రతి instance కి వేరు)
  this.age = age;
}
// methods ని prototype మీద — అన్ని instances share (memory efficient!)
Person.prototype.greet = function () {
  return `Hi, I'm ${this.name}`;
};

const p1 = new Person("Surya", 30);
const p2 = new Person("Palsingh", 25);
p1.greet === p2.greet;  // true — ఒకే shared method (prototype లో)!
```

> **ఎందుకు methods ని prototype మీద పెడతారు?** Constructor లో పెడితే (`this.greet = ...`), ప్రతి instance కి ఒక **copy** – 1000 objects = 1000 copies (memory waste). Prototype మీద పెడితే **ఒక్కటే** shared. ఇదే classes లో methods automatic గా prototype మీద ఉండటానికి కారణం.

లోపల ఏం జరుగుతుంది – `new` operator

`new Person("Surya")` చేసినప్పుడు 4 steps:

```
// new Person("Surya") ≈
function myNew(Constructor, ...args) {
  const obj = {};                                   // 1. కొత్త empty object
  Object.setPrototypeOf(obj, Constructor.prototype);// 2. __proto__ link
  const result = Constructor.apply(obj, args);      // 3. this = obj, run
  return (typeof result === "object" && result) ? result : obj; // 4. return obj
}
```

Shadowing

Own property, prototype property ని "కప్పేస్తుంది" (shadow) – own మొదట దొరుకుతుంది.

```
const proto = { greet: () => "prototype greet" };
const obj = Object.create(proto);
obj.greet = () => "own greet";    // shadows prototype
obj.greet();                      // "own greet" (own మొదట)
delete obj.greet;
obj.greet();                      // "prototype greet" (మళ్ళీ prototype)
```

Gotchas (సాధారణ తప్పులు)

- **`prototype` vs `__proto__` గందరగోళం** – `prototype` functions మీద, `__proto__` objects మీద.
- **`__proto__` ని production లో set చేయడం** – slow, deprecated; `Object.create` / `Object.setPrototypeOf` / `getPrototypeOf`.
- **Methods ని constructor లో పెట్టడం** – ప్రతి instance copy (memory); prototype మీద పెట్టు.
- **Prototype ని mutate చేయడం (Object.prototype కి add)** – అన్ని objects ప్రభావితం (prototype pollution, Topic 46).
- **`for...in` inherited properties** – prototype properties కూడా iterate; own కి `hasOwnProperty` / `Object.keys`.

Key Points

- JS = **prototype-based** inheritance. ప్రతి object కి `[[Prototype]]` link.
- `prototype` = functions మీద (instances కి blueprint); `__proto__` = objects మీద (actual lookup link). `d.__proto__ === Dog.prototype`.
- **Prototype chain**: own → proto → proto → … → `Object.prototype` → `null`.
- Methods ని **prototype మీద** పెడితే అన్ని instances **share** (memory efficient).
- `Object.create(proto)` = direct prototypal inheritance; `new` = 4 steps (create, link, apply, return).

Interview దృష్టి

Q: `prototype` మరియు `__proto__` తేడా? A: `prototype` – functions (constructors) మీద ఉండే property; `new` తో create అయిన instances కి ఇది వాటి prototype అవుతుంది. `__proto__` (`[[Prototype]]`) – ప్రతి object మీద ఉండే actual link, దాని prototype కి; property lookup ఈ link వెంట జరుగుతుంది. సంబంధం: `instance.__proto__ === Constructor.prototype`. Modern access: `Object.getPrototypeOf(obj)`.

Q: Prototype chain ఎలా పని చేస్తుంది? A: Object మీద property/method access చేసినప్పుడు, JS మొదట own properties లో చూస్తుంది; లేకపోతే `__proto__` link ద్వారా prototype లో, తర్వాత దాని prototype లో... `Object.prototype` (chain top, దీని `__proto__` = null) వరకు. దొరికిన మొదటిది వాడుతుంది; ఎక్కడా లేకపోతే `undefined`. Values copy అవ్వవు – live shared links.

Q: Methods ని prototype మీద ఎందుకు పెడతారు, constructor లో కాదు? A: Prototype మీద పెడితే అన్ని instances ఒకే method copy ని **share** చేస్తాయి – memory efficient. Constructor లో (`this.method = ...`) పెడితే ప్రతి instance కి వేరే copy – 1000 objects = 1000 function copies (waste), మరియు identity పోతుంది (`a.m !== b.m`). అందుకే ES6 classes methods ని automatic గా prototype మీద పెడతాయి.

---

23. Classes

వివరణ

ES6 (2015) `class` syntax వచ్చింది. కానీ **ముఖ్యమైన నిజం:** JavaScript classes అనేవి **"syntactic sugar"** – లోపల అవి ఇప్పటికీ **prototypes + constructor functions** (Topic 22). అంటే classes కొత్త inheritance model కాదు – పాత prototypal mechanism కి అందమైన, సులభమైన syntax మాత్రమే. ఈ topic language mechanics మీద focus; **deep OOP (SOLID, patterns, polymorphism, abstraction) `OOPS_Telugu.md` లో ఉంది.**

Real-life Scenario

> **Class = సినిమా టిక్కెట్ machine యొక్క కొత్త touch-screen interface.** లోపల అదే పాత mechanism (prototype), కానీ బయట clean, సులభమైన buttons (class syntax). పాత రోజుల్లో manual గా `Function.prototype.x = ...` రాయాల్సి వచ్చేది – ఇప్పుడు `class { x() {} }`. పని ఒక్కటే, అనుభవం మెరుగు.

Class Basics

```
class Person {
  // instance field (ES2022)
  species = "human";

  constructor(name, age) {   // new తో automatic call
    this.name = name;        // own properties
    this.age = age;
  }

  greet() {                  // method – automatic గా prototype మీద!
    return `Hi, I'm ${this.name}`;
  }

  // getter / setter
  get info() { return `${this.name}, ${this.age}`; }
  set nickname(n) { this._nick = n; }
```

```
  // static method — instance కాదు, class మీద
  static create(name) { return new Person(name, 0); }
  static species = "Homo sapiens"; // static field
}

const p = new Person("Surya", 30);
p.greet();              // "Hi, I'm Surya"
p.info;                 // "Surya, 30" (getter — no parens)
Person.create("A");     // static — class మీద call
Person.species;         // "Homo sapiens"

// proof: methods prototype మీద ఉన్నాయి:
p.greet === Person.prototype.greet; // true
```

Private Fields (`#`) – ES2022 (నిజమైన privacy)

`#` prefix = నిజమైన private – class వెలుపల **అస్సలు** access కుదరదు (closures/ `_` convention కంటే బలం).

```
class BankAccount {
  #balance = 0;              // private field
  #pin;                     // private

  constructor(pin) { this.#pin = pin; }

  #validate(pin) {           // private method
    return pin === this.#pin;
  }

  deposit(amt) { this.#balance += amt; return this.#balance; }
  getBalance(pin) {
    return this.#validate(pin) ? this.#balance : "Access denied";
  }
}

const acc = new BankAccount(1234);
acc.deposit(100);
acc.getBalance(1234);   // 100
// acc.#balance;        // ❌ SyntaxError — వెలుపల access అసాధ్యం!
acc.#balance;           // even typeof/in fails

// brand check (Topic 33-style):
class C { #x; static has(o) { return #x in o; } } // ES2022 `#x in o`
```

Inheritance – extends & super

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name);             // ⚠️ parent constructor — this ముందు తప్పనిసరి!
```

```
    this.breed = breed;
  }
  speak() {                    // method overriding
    return `${super.speak()}, barks`; // super.method() — parent version
  }
}

const d = new Dog("Tommy", "Lab");
d.speak();               // "Tommy makes a sound, barks"
d instanceof Dog;       // true
d instanceof Animal;    // true (chain)
```

`super` **నియమాలు:** (1) subclass constructor లో `this` వాడేముందు `super()` call చేయాలి (లేకపోతే ReferenceError). (2) `super.method()` = parent version ని call (override చేసినా parent ని కూడా వాడటానికి).

instanceof & చెక్‌లు

```
d instanceof Dog;       // true (prototype chain లో Dog.prototype ఉందా?)
d instanceof Animal;    // true
d instanceof Object;    // true (అన్నీ చివర్లో Object)

// instanceof లోపల: obj.__proto__ chain లో Constructor.prototype ఉందా చెక్

// constructor property:
d.constructor === Dog;  // true
d.constructor.name;     // "Dog"
```

లోపల ఏం జరుగుతుంది — Class = sugar over prototype

```
// ఈ class...
class Cat {
  constructor(name) { this.name = name; }
  meow() { return "Meow"; }
}

// ...దాదాపు దీనితో సమానం:
function Cat(name) { this.name = name; }
Cat.prototype.meow = function () { return "Meow"; };

// తేడాలు (classes strict గా ఉంటాయి):
// 1. Classes hoist అవుతాయి కానీ TDZ (function declarations పూర్తిగా hoist)
// 2. Class body ఎప్పుడూ strict mode
// 3. new లేకుండా class call = TypeError (function call అవుతుంది)
// 4. Class methods non-enumerable (for...in లో కనిపించవు)
```

Static Blocks (ES2022)

```
class Config {
  static settings;
  static {                    // static initialization block
```

```
    this.settings = loadConfig();  // complex static setup
  }
}
```

Gotchas (సాధారణ తప్పులు)

- **Classes hoist అవ్వవు (usable గా)** – TDZ; declare ముందు `new` → ReferenceError.
- **`super()` మర్చిపోవడం** – subclass constructor లో `this` ముందు `super()` తప్పనిసరి.
- **`new` లేకుండా class call** – TypeError (functions లా silent కాదు).
- **Method లో `this` lost** – method ని callback గా pass చేస్తే this తెగుతుంది (Topic 14); arrow field లేదా bind.
- **`#private` ని `_private` అనుకోవడం** – `_` convention మాత్రమే (access అవుతుంది); `#` నిజమైన private.
- **Arrow methods vs prototype methods** – arrow class field (`fn = () => {}`) ప్రతి instance కి copy (auto-bind కానీ memory); regular method prototype-shared.

Key Points

- Class = **syntactic sugar** over prototypes + constructor functions.
- Methods → **prototype** (shared); fields → **instance** (own).
- `#private` = నిజమైన privacy (ES2022); `static` = class-level.
- `extends` + `super()` (this ముందు) + `super.method()` (parent call).
- Class differences: TDZ (hoist కానీ unusable), always strict, `new` mandatory, non-enumerable methods.
- Deep OOP/patterns → `OOPS_Telugu.md`.

Interview దృష్టి

Q: JS classes నిజమైన classes నా? A: కాదు – syntactic sugar over prototype-based inheritance. లోపల `class` constructor function + prototype methods గా translate అవుతుంది (`class C { m() {} }` ≈ `function C(){}; C.prototype.m = ...`). కానీ కొన్ని తేడాలు: classes TDZ లో ఉంటాయి (hoist కానీ unusable), always strict mode, `new` లేకుండా call చేస్తే TypeError, methods non-enumerable.

Q: `super` ఎప్పుడు, ఎందుకు? A: రెండు రూపాలు: (1) `super(...)` – subclass constructor లో parent constructor ని call; `this` వాడేముందు తప్పనిసరి (లేకపోతే ReferenceError), ఎందుకంటే parent ముందు `this` ని initialize చేయాలి. (2) `super.method()` – parent యొక్క method version ని call, override చేసిన method లోపల నుండి కూడా (parent behavior extend చేయడానికి).

Q: `#private` fields మరియు `_private` convention తేడా? A: `_private` – కేవలం naming convention; property నిజానికి public, ఎవరైనా `obj._x` access చేయవచ్చు. `#private` (ES2022) – language-enforced నిజమైన privacy; class వెలుపల `obj.#x` = SyntaxError, `in`/reflection లో కూడా కనిపించదు. Encapsulation కి `#` వాడాలి.

24. Map, Set, WeakMap, WeakSet

వివరణ

ES6 లో 4 కొత్త collection types వచ్చాయి. **Map** = key-value (object లా, కానీ better). **Set** = unique values (array లా, duplicates లేకుండా). **WeakMap/WeakSet** = memory-friendly variants (garbage collection ని allow చేస్తాయి). Objects/arrays బదులు వీటిని ఎప్పుడు వాడాలో తెలియడం senior skill.

Map – better key-value store

Object కి పోలిస్తే: **ఏ type key అయినా** (objects కూడా!), **insertion order preserve**, **size** direct, **iterate సులభం**, prototype pollution risk లేదు.

```
const map = new Map();
map.set("name", "Surya");
map.set(1, "number key");       // number key!
const objKey = { id: 1 };
map.set(objKey, "object key!"); // object key — objects తో అసాధ్యం!

map.get("name");     // "Surya"
map.get(objKey);     // "object key!"
map.has(1);          // true
map.size;            // 3 (Object.keys().length అవసరం లేదు)
map.delete(1);
map.forEach((v, k) => console.log(k, v));
for (const [k, v] of map) { }   // iterable directly

// init from entries:
const m = new Map([["a", 1], ["b", 2]]);
```

Map vs Object – ఎప్పుడు ఏది

అంశం	Map	Object
Key types	ఏదైనా (object, function కూడా)	string / symbol మాత్రమే
Order	insertion order guarantee	integer keys sorted (subtle)
Size	`map.size`	`Object.keys(o).length`
Iterate	directly iterable	`Object.entries` అవసరం
Performance (frequent add/delete)	better	slower
Default keys (prototype)	pure (no inherited)	`toString` etc. inherited
JSON	direct కాదు	native

> **నియమం:** keys dynamic/unknown, non-string, లేదా frequent add/delete → **Map.** Structured record, JSON, fixed known keys → **Object.**

Set – unique values

```
const set = new Set([1, 2, 2, 3, 3, 3]);
set;              // Set(3) {1, 2, 3} — duplicates తీసేసింది!
set.add(4);
set.has(2);       // true (O(1) lookup — array.includes O(n) కంటే వేగం)
set.delete(1);
set.size;         // 3
for (const v of set) { }  // iterable

// ── అత్యంత common use: array dedupe ──
const arr = [1, 1, 2, 3, 3];
const unique = [...new Set(arr)];  // [1, 2, 3] ✅

// set operations:
const a = new Set([1, 2, 3]), b = new Set([2, 3, 4]);
const intersection = [...a].filter(x => b.has(x)); // [2, 3]
const union = [...new Set([...a, ...b])];          // [1,2,3,4]
```

లోపల ఏం జరుగుతుంది – WeakMap & WeakSet

"Weak" అంటే – keys ని **weakly** hold చేస్తాయి. అంటే ఒక object కి WeakMap లోని reference **తప్ప** వేరే reference లేకపోతే, garbage collector దాన్ని **free చేయవచ్చు** (Map అయితే free చేయదు – memory leak). ఇది memory-sensitive metadata కి perfect.

```
const wm = new WeakMap();
let obj = { id: 1 };
wm.set(obj, "metadata");   // key తప్పనిసరిగా object
wm.get(obj);               // "metadata"
obj = null;                // ← ఇప్పుడు obj కి వేరే reference లేదు
// → GC ఆ object ని (మరియు WeakMap entry ని) free చేయవచ్చు!

// WeakMap limitations:
// - keys objects మాత్రమే (primitives కాదు)
// - iterate కుదరదు (no .size, no forEach, no keys())
// - ఎందుకు? entries ఎప్పుడు GC అవుతాయో తెలియదు → deterministic iteration అసాధ్యం
```

Use cases: DOM element metadata (element remove అయితే auto-cleanup), private data (`WeakMap` per instance), caching (object gone → cache entry gone), memoization.

```
// private data pattern (pre-# fields):
const privateData = new WeakMap();
class User {
  constructor(name) { privateData.set(this, { name }); }
  getName() { return privateData.get(this).name; }
}
// instance GC అయితే, దాని private data కూడా auto-free
```

పోలిక Table

Collection	Keys/Values	Duplicates	Iterable	GC-friendly
Map	ఏ key అయినా	keys unique	✅	❌ (strong refs)
Set	values	❌ unique	✅	❌
WeakMap	object keys	keys unique	❌	✅
WeakSet	object values	unique	❌	✅

Gotchas (సాధారణ తప్పులు)

- **Object ని Map లా వాడి key collisions** – object keys strings కి coerce (`obj[1] === obj["1"]`); Map distinguishes.
- **Set duplicates NaN/objects** – `Set` NaN ని unique గా (ఒక్కటే NaN OK), కానీ `{}` `{}` వేర్వేరు (reference).
- **WeakMap iterate చేయాలనుకోవడం** – కుదరదు (no size/keys/forEach); needed అయితే Map వాడు.
- **WeakMap key primitive** – TypeError; objects మాత్రమే.
- **Map ని JSON.stringify** – `{}` వస్తుంది (empty)! `[...map]` గా convert చేయాలి.

Key Points

- **Map** = ఏ-type keys, order, `size`, iterable, frequent add/delete కి better than object.
- **Set** = unique values; **dedupe:** `[...new Set(arr)]`; O(1) `has`.
- **WeakMap/WeakSet** = object keys/values only, **not iterable**, **GC-friendly** (weak refs → no leaks).
- WeakMap uses: metadata, private data, caches – object gone → entry auto-cleaned.
- Map vs Object: dynamic/non-string keys, frequent mutation → Map; JSON/records → Object.

Interview దృష్టి

Q: Map vs Object ఎప్పుడు? A: **Map** – keys ఏ type అయినా (objects, functions), insertion order guarantee, `.size`, directly iterable, frequent add/delete లో faster, prototype pollution risk లేదు. **Object** – string/symbol keys, JSON serialization, fixed known structure (records). Dynamic/unknown/non-string keys లేదా frequent mutation → Map; structured data/JSON → Object.

Q: `Set` ఎందుకు, common use? A: Unique values collection – duplicates automatic గా తీసేస్తుంది, O(1) `has` lookup (array `includes` O(n)). అత్యంత common use: array deduplication `[...new Set(arr)]`. Set operations (union/intersection), fast membership checks కి కూడా.

Q: WeakMap అంటే? Map కంటే ఎప్పుడు మంచిది? A: WeakMap keys (objects మాత్రమే) ని weakly hold చేస్తుంది – ఆ object కి వేరే reference లేకపోతే garbage collector దాన్ని (మరియు entry ని) free చేయవచ్చు, memory leak లేదు. Map strong reference పట్టుకుని GC ని ఆపుతుంది. WeakMap not

iterable (no size/keys). Use: object metadata, private instance data, caches – object lifecycle తో entry auto-cleanup కావాలంటే WeakMap.

---

25. Symbols & Well-known Symbols

వివరణ

Symbol = ES6 లో వచ్చిన 7వ primitive type. దీని ఏకైక లక్షణం: **ప్రతి Symbol పూర్తిగా unique** – అదే description ఇచ్చినా రెండు symbols ఎప్పటికీ equal కావు. దీనివల్ల **collision-free object keys** (ఇతరుల code తో conflict అవ్వవు) create చేయవచ్చు. **Well-known Symbols** (Symbol.iterator లాంటివి) JavaScript యొక్క అంతర్గత behaviors ని customize చేయనిస్తాయి.

Real-life Scenario

> **Symbol = ప్రతి ఒక్కరికీ unique అయిన Aadhaar number.** ఇద్దరి పేరు "Surya" అయినా (అదే description), వాళ్ళ Aadhaar numbers వేరు – ఎప్పటికీ ఒకటి కావు. ఒక object లో "id" అనే key ని నువ్వూ, ఇంకో library వాడూ పెడితే conflict. కానీ Symbol("id") key వాడితే – నీ symbol, library symbol పూర్తిగా వేరు, collision లేదు.

Uniqueness – core property

```
const s1 = Symbol("id");
const s2 = Symbol("id");   // అదే description!

s1 === s2;          // false ❗ – ఎప్పటికీ equal కాదు (unique)
s1.description;     // "id" (కేవలం debugging label)
typeof s1;          // "symbol"

// Symbol ని new తో create చేయలేం:
// new Symbol();    // ❌ TypeError (constructor కాదు)
```

Symbols as Object Keys – collision-free

```
const ID = Symbol("id");
const user = {
  name: "Surya",
  [ID]: 12345          // symbol key (computed syntax)
};

user[ID];              // 12345
user.name;             // "Surya"

// ── Symbols "hidden" from normal iteration ──
Object.keys(user);              // ["name"] – symbol కనిపించదు!
Object.entries(user);           // [["name", "Surya"]]
for (const k in user) { }       // "name" మాత్రమే
JSON.stringify(user);           // {"name":"Surya"} – symbol skip!
```

```
// symbols ని పొందడానికి ప్రత్యేక method:
Object.getOwnPropertySymbols(user); // [Symbol(id)]
Reflect.ownKeys(user);              // ["name", Symbol(id)] (అన్నీ)
```

> **Symbols "semi-private."** అవి normal iteration (`keys`, `for...in`, `JSON`) లో కనిపించవు – accidental access/overwrite నుండి రక్షణ. కానీ `getOwnPropertySymbols` తో పొందవచ్చు, కాబట్టి నిజమైన privacy కాదు (అది `#` fields, Topic 23).

Symbol.for – global symbol registry

```
// Symbol.for — global registry లో share చేయగల symbols:
const a = Symbol.for("shared");
const b = Symbol.for("shared");
a === b;              // true! (registry నుండి అదే symbol)
Symbol.keyFor(a);     // "shared"

// Symbol() vs Symbol.for():
Symbol("x") === Symbol("x");         // false (ప్రతిసారి కొత్తది)
Symbol.for("x") === Symbol.for("x"); // true (registry shared)
```

లోపల ఏం జరుగుతుంది – Well-known Symbols

JavaScript లోపల కొన్ని behaviors ని control చేసే predefined symbols. వీటిని object లో implement చేస్తే, ఆ object యొక్క built-in behavior మారుతుంది.

`Symbol.iterator` – object ని iterable చేయడం (అత్యంత ముఖ్యం):

```
const range = {
  start: 1,
  end: 5,
  [Symbol.iterator]() {           // ఈ method ఉంటే for...of పని చేస్తుంది!
    let current = this.start;
    const end = this.end;
    return {
      next() {                    // iterator protocol
        return current <= end
          ? { value: current++, done: false }
          : { value: undefined, done: true };
      }
    };
  }
};

for (const n of range) console.log(n); // 1, 2, 3, 4, 5 ✅
[...range];                            // [1,2,3,4,5] (spread కూడా!)
Array.from(range);                     // [1,2,3,4,5]
```

`for...of`, spread `...`, destructuring – ఇవన్నీ లోపల `Symbol.iterator` ని వాడతాయి. Arrays, strings, Maps, Sets కి ఇది built-in; plain objects కి లేదు (అందుకే `for...of` object మీద fail).

ఇతర well-known symbols:

```
class Temp {
  constructor(c) { this.c = c; }
  [Symbol.toPrimitive](hint) {     // coercion ని control
    if (hint === "number") return this.c;
    return `${this.c}°C`;
  }
  get [Symbol.toStringTag]() { return "Temperature"; } // Object.prototype.toString
  static [Symbol.hasInstance](inst) { return inst.c !== undefined; } // instanceof
}
const t = new Temp(25);
+t;                       // 25 (number hint)
`${t}`;                   // "25°C" (string hint)
Object.prototype.toString.call(t); // "[object Temperature]"
```

Well-known Symbol	ఏం control చేస్తుంది
`Symbol.iterator`	`for...of`, spread, destructuring
`Symbol.asyncIterator`	`for await...of` (Topic 31)
`Symbol.toPrimitive`	object → primitive coercion
`Symbol.toStringTag`	`Object.prototype.toString` output
`Symbol.hasInstance`	`instanceof` behavior

Gotchas (సాధారణ తప్పులు)

- **Symbols equal అవుతాయనుకోవడం** – `Symbol("x") !== Symbol("x")`; shared కావాలంటే `Symbol.for`.
- **`JSON.stringify` symbols ని skip చేస్తుంది** – symbol keys/values serialize అవ్వవు (silent data loss).
- **Symbols ని నిజమైన private అనుకోవడం** – `getOwnPropertySymbols` తో access; privacy కి `#` fields.
- **Symbol ని string context లో implicit coerce** – `"id: " + sym` → TypeError; `String(sym)` explicit.
- **`Object.keys` symbols చూపిస్తుందనుకోవడం** – చూపించదు; `Reflect.ownKeys` / `getOwnPropertySymbols`.

Key Points

- **Symbol** = 7వ primitive; **ప్రతిదీ unique** (`Symbol("x") !== Symbol("x")`).
- Collision-free object keys; normal iteration (`keys` / `for...in` /JSON) నుండి hidden (semi-private).
- **`Symbol.for(key)`** = global registry (shared, equal).
- **Well-known Symbols** built-in behaviors ని customize: `Symbol.iterator` (for...of/spread), `toPrimitive`, `hasInstance`, `toStringTag`.

- `Reflect.ownKeys` / `getOwnPropertySymbols` తో symbols పొందవచ్చు (నిజ privacy కాదు).

Interview దృష్టి

Q: Symbol అంటే? ఎందుకు? A: ES6 యొక్క unique primitive type. ప్రతి Symbol పూర్తిగా unique (అదే description ఇచ్చినా `Symbol("x") !== Symbol("x")`). ప్రధాన use: collision-free object keys – వేర్వేరు libraries ఒకే object లో properties పెట్టినా conflict అవ్వకుండా. Symbol keys normal iteration/JSON నుండి hidden (semi-private). Well-known symbols తో built-in behaviors customize.

Q: `Symbol.iterator` ఏం చేస్తుంది? A: Object ని iterable చేసే well-known symbol. ఒక object లో `[Symbol.iterator]()` method (iterator protocol – `next()` returning `{value, done}`) implement చేస్తే, ఆ object మీద `for...of`, spread `...`, destructuring, `Array.from` పని చేస్తాయి. Arrays/strings/Maps/Sets కి built-in; plain objects కి లేదు (అందుకే వాటిపై `for...of` fail).

Q: Symbols నిజమైన private keys అవుతాయా? A: కాదు – semi-private. `Object.keys`, `for...in`, `JSON.stringify` లో కనిపించవు (accidental access నుండి రక్షణ), కానీ `Object.getOwnPropertySymbols(obj)` లేదా `Reflect.ownKeys(obj)` తో పొందవచ్చు. నిజమైన privacy కావాలంటే class `#private` fields (Topic 23).

---

26. JSON

వివరణ

JSON (JavaScript Object Notation) = data ని exchange చేయడానికి universal text format. పేరు JS నుండి వచ్చినా, ఇది భాష-స్వతంత్రం (language-independent) – ప్రతి భాష దీన్ని అర్థం చేసుకుంటుంది. APIs, config files, localStorage, network – అన్నిచోటా JSON. JS లో `JSON.stringify` (object → string) మరియు `JSON.parse` (string → object) రెండు methods. వీటి subtleties, deep clone use, gotchas ముఖ్యం.

Real-life Scenario

> **JSON = అంతర్జాతీయ shipping కోసం వస్తువుని flat-pack చేయడం.** నీ 3D furniture (JS object – methods, references, types తో) ని పంపాలంటే, దాన్ని విడగొట్టి, flat cardboard (text string) లో pack చేస్తావు (`stringify`). అవతలి వ్యక్తి దాన్ని తిరిగి assemble చేస్తాడు (`parse`). కానీ flat-pack లో కొన్ని పెట్టలేవు – batteries (functions), కదిలే భాగాలు (Date methods) – అవి పోతాయి. అందుకే assemble చేశాక కొంత మిస్ అవుతుంది.

stringify & parse – basics

```
const user = { name: "Surya", age: 30, active: true, roles: ["admin"] };

// —— object → JSON string ——
const json = JSON.stringify(user);
```

```
// '{"name":"Surya","age":30,"active":true,"roles":["admin"]}'

// ── JSON string → object ──
const obj = JSON.parse(json);
obj.name;  // "Surya"

// ── pretty print (3వ argument = indentation) ──
JSON.stringify(user, null, 2);   // 2-space indented (readable)
JSON.stringify(user, null, "\t"); // tab indented
```

లోపల ఏం జరుగుతుంది – stringify ఏం పోగొడుతుంది

`JSON.stringify` కొన్ని JS values ని handle చేయలేదు – అవి **skip** అవుతాయి లేదా `null` అవుతాయి. ఇది deep clone లో data loss కి కారణం.

```
JSON.stringify({
  fn: () => {},          // ❌ functions — skip (key మాయం)
  und: undefined,        // ❌ undefined — skip (key మాయం)
  sym: Symbol("x"),      // ❌ symbols — skip
  num: NaN,              // → null
  inf: Infinity,         // → null
  date: new Date(),      // → "2026-07-16T..." (string! Date object పోతుంది)
  big: 10n,              // ❌ TypeError! (BigInt serialize కాదు)
});
// arrays లో undefined/function → null (position hold చేయాలి):
JSON.stringify([1, undefined, () => {}, 4]); // "[1,null,null,4]"

// circular reference → Error:
const a = {}; a.self = a;
// JSON.stringify(a); // ❌ TypeError: Converting circular structure
```

Value	stringify ఫలితం
function, undefined, symbol (object లో)	**skip** (key మాయం)
function, undefined (array లో)	`null`
`NaN`, `Infinity`	`null`
`Date`	ISO string (Date object పోతుంది)
`BigInt`	**TypeError**
`Map`, `Set`	`{}` (empty!)
circular reference	**TypeError**

Replacer & Reviver – customization

Replacer (stringify 2వ arg) – ఏ keys include చేయాలో లేదా values ఎలా transform చేయాలో control:

```
// array replacer — allowlist (ఈ keys మాత్రమే):
JSON.stringify(user, ["name", "age"]);   // {"name":"Surya","age":30}
```

```
// function replacer — ప్రతి key-value transform:
JSON.stringify(user, (key, value) => {
  if (key === "age") return undefined;   // age ని దాచు (skip)
  if (typeof value === "string") return value.toUpperCase();
  return value;
});
// sensitive fields (password) దాచడానికి, values mask చేయడానికి ఉపయోగం
```

Reviver (parse 2వ arg) – parse చేసేటప్పుడు values ని transform (ఉదా. date strings → Date objects):

```
const json = '{"name":"Surya","joined":"2026-01-01T00:00:00.000Z"}';
const obj = JSON.parse(json, (key, value) => {
  // ISO date string ని Date object గా revive:
  if (typeof value === "string" && /^\d{4}-\d{2}-\d{2}T/.test(value)) {
    return new Date(value);
  }
  return value;
});
obj.joined instanceof Date;  // true ✅ (revived!)
```

toJSON – custom serialization

Object లో `toJSON()` method ఉంటే, stringify దాన్ని వాడుతుంది (Date లోపల ఇలానే చేస్తుంది):

```
class User {
  constructor(name, pwd) { this.name = name; this.pwd = pwd; }
  toJSON() { return { name: this.name }; }  // pwd ని ఎప్పుడూ serialize చేయవద్దు
}
JSON.stringify(new User("Surya", "secret")); // {"name":"Surya"} (pwd లేదు)
```

Deep Clone via JSON (limited – Topic 19)

```
const original = { a: 1, nested: { b: 2 } };
const clone = JSON.parse(JSON.stringify(original)); // deep, కానీ limited

// ✅ works: nested plain objects/arrays/primitives
// ❌ loses: functions, undefined, Date→string, Map/Set, symbols, circular
// → modern: structuredClone(original) (Topic 19)
```

Gotchas (సాధారణ తప్పులు)

- `JSON.parse` **invalid JSON → throws** – ఎప్పుడూ `try/catch` లో wrap చేయి (Topic 33).
- **Deep clone data loss** – functions/undefined/Date/Map పోతాయి; `structuredClone` వాడు.
- **BigInt stringify → TypeError** – custom replacer లో `toString()` చేయాలి.
- **Circular reference → TypeError** – `stringify` fail; custom handling లేదా libraries.
- **JSON keys ఎప్పుడూ strings, double-quotes** – `{name: 1}` invalid JSON; `{"name": 1}`. Single quotes, trailing commas, comments invalid.
- `undefined` **vs missing** – `JSON.stringify({a: undefined})` = `"{}"` (a మాయం).

Key Points

- `JSON.stringify` (object → string), `JSON.parse` (string → object). Universal, language-independent.
- **Data loss:** functions, `undefined`, symbols skip; `Date` →string; `NaN` / `Infinity` →null; `Map` / `Set` → `{}`; BigInt/circular → **TypeError**.
- **Replacer** (stringify arg 2) = filter/transform on write; **Reviver** (parse arg 2) = transform on read (date revival).
- `toJSON()` method = custom serialization (sensitive fields దాచడం).
- Deep clone via JSON = limited; `structuredClone` better. `parse` ని `try/catch` లో.

Interview దృష్టి

Q: `JSON.stringify` ఏ values ని handle చేయలేదు? A: functions, `undefined`, symbols – object లో skip అవుతాయి (key మాయం), array లో `null` అవుతాయి. `NaN` / `Infinity` → `null`. `Date` → ISO string (Date object పోతుంది). `Map` / `Set` → empty `{}`. `BigInt` మరియు circular references → **TypeError**. అందుకే JSON-based deep clone lossy – `structuredClone` preferred.

Q: Replacer మరియు reviver అంటే? A: **Replacer** – `JSON.stringify(obj, replacer)` 2వ argument; array (allowlist keys) లేదా function (ప్రతి key-value transform/filter – sensitive fields దాచడం, values mask). **Reviver** – `JSON.parse(str, reviver)` 2వ argument; parse చేసేటప్పుడు ప్రతి value ని transform (సాధారణంగా ISO date strings ని `Date` objects గా revive చేయడానికి, ఎందుకంటే JSON కి native date type లేదు).

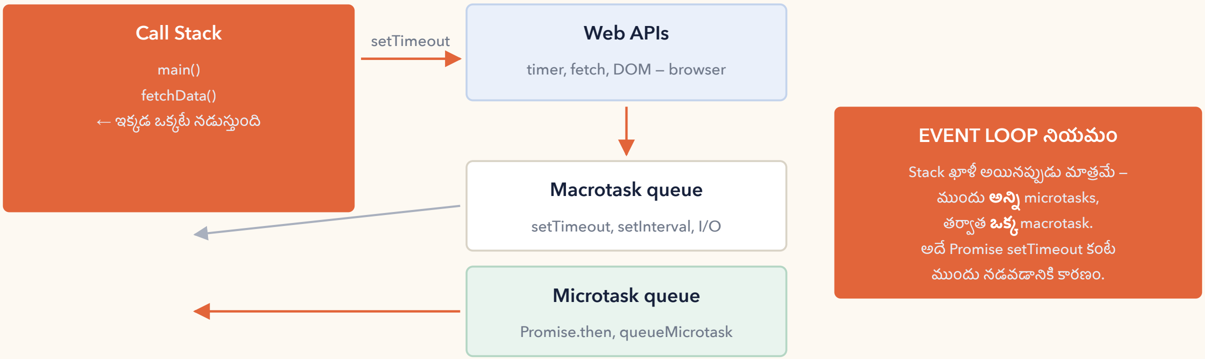
Q: `JSON.parse(JSON.stringify(obj))` deep clone గా వాడటంలో సమస్యలు? A: (1) functions, `undefined`, symbols పోతాయి. (2) `Date` string అవుతుంది (Date object కాదు). (3) `Map` / `Set` empty `{}` అవుతాయి. (4) BigInt/circular references TypeError throws. (5) `NaN` / `Infinity` → null. (6) performance నెమ్మది. Modern replacement: `structuredClone(obj)` – ఇవన్నీ (functions తప్ప) సరిగ్గా handle చేస్తుంది.

---

27. Event Loop deep (call stack, Web APIs, task vs microtask queue, rendering, starvation)

EVENT LOOP · STACK, QUEUES, మరియు క్రమం

EVENT LOOP – JS single-threaded అయినా async ఎలా పని చేస్తుంది



దీని పరిణామం – starvation

Microtask లోపల మళ్ళీ microtask queue చేస్తే – macrotasks ఎప్పటికీ నడవవు.
Browser render కూడా ఒక macrotask లాంటిదే → page స్తంభిస్తుంది.
అందుకే భారీ recursive promise chains ప్రమాదకరం.

Interview lo క్లాసిక్ ప్రశ్న: `setTimeout(f, 0)` మరియు `Promise.resolve().then(g)` – ఏది ముందు? జవాబు **g**, ఎందుకంటే microtask queue ఎప్పుడూ ముందు పూర్తిగా ఖాళీ అవుతుంది.

వివరణ

JavaScript అనేది **single-threaded** language – అంటే ఒకే ఒక్క call stack ఉంటుంది, ఒక సమయంలో ఒకే ఒక్క పని చేస్తుంది. మరి అలాంటప్పుడు network calls, timers, DOM events అన్నీ ఒకేసారి ఎలా handle అవుతున్నాయి? దీనికి సమాధానం **Event Loop**.

Event loop అంటే call stack ఖాళీ అయినప్పుడు queue నుండి pending callbacks ని తీసుకొచ్చి run చేసే mechanism. ఇది JS engine లో భాగం కాదు – ఇది **runtime** (browser లేదా Node.js) అందించే feature.

ముఖ్యమైన components:

Component	పని
Call Stack	ప్రస్తుతం execute అవుతున్న functions ని LIFO order లో ఉంచుతుంది
Web APIs	`setTimeout`, `fetch`, DOM events లాంటివి background లో handle చేస్తాయి (browser అందిస్తుంది)
Macrotask Queue (Task Queue)	`setTimeout`, `setInterval`, I/O, UI events callbacks ఇక్కడ చేరతాయి
Microtask Queue	Promises (`.then`), `queueMicrotask`, `MutationObserver` callbacks – **higher priority**
Event Loop	Stack ఖాళీ అయ్యాక queues నుండి tasks ని stack కి push చేస్తుంది

Real-life Scenario

> ఒక **restaurant** ఊహించుకోండి. ఒకే ఒక్క **waiter** (call stack) ఉన్నాడు. Customer order ఇస్తే waiter kitchen (Web API) కి చెప్పి, వెంటనే వేరే customers ని attend అవుతాడు. Kitchen వంట complete చేసాక bell కొడుతుంది (callback ready). Waiter తన current పని ముగించాకే (stack empty) ఆ ready అయిన dish ని serve చేస్తాడు. అంతేకాదు – VIP customers (microtasks) కి normal customers (macrotasks) కంటే ముందు serve చేస్తాడు. ఒక్కో batch VIP లందరినీ ముందు clear చేసాకే next normal customer.

Internals – ఖచ్చితమైన algorithm

Event loop ఒక్కో "tick" లో ఇలా జరుగుతుంది:

```
1. Call stack లో ఏదైనా ఉంటే, అది పూర్తిగా empty అయ్యేదాకా run చేయి.
2. Stack empty అయ్యాక → MICROTASK queue మొత్తం drain చేయి
   (ఒక microtask మరో microtask add చేస్తే, అదీ ఇదే tick లో run అవుతుంది!)
3. Microtasks అన్నీ అయిపోయాక → (అవసరమైతే) RENDER చేయి (repaint, ~60fps).
4. తర్వాత MACROTASK queue నుండి ఒక్కటే task తీసుకో, stack కి push చేయి.
5. మళ్ళీ step 1 కి వెళ్ళు.
```

కీలక నియమం: ప్రతి **ఒక్క** macrotask తర్వాత, **అన్ని** microtasks run అవుతాయి. Macrotasks ఒక్కొక్కటిగా, microtasks batch గా.

ASCII Diagram

```
          ┌──────────────────────────────────────────┐
          │                CALL STACK                │
          │  (LIFO — top frame మాత్రమే execute అవుతుంది)  │
          │                                          │
          │       ┌────────────┐                     │
          │       │  foo()     │  ← currently running│
          │       ├────────────┤                     │
          │       │  main()    │                     │
          │       └────────────┘                     │
          └──────────────────────────────────────────┘
                ▲                          │
                │ push (stack empty అయ్యాక)   │ setTimeout / fetch
                │                          ▼ register అవుతాయి
          ┌──────────┐              ┌───────────────────┐
          │EVENT LOOP│◄─────────────│      WEB APIs     │
          │(monitor) │              │ timers, fetch, DOM│
          └──────────┘              │ events, geolocation│
                │                   └───────────────────┘
                │ dequeue                     │ callback ready అయ్యాక
                │                             │ queue లోకి push
       ┌────────┴─────────────────────────────┴─────────┐
       │                                                │
       ▼  (HIGH priority — మొత్తం drain)      ▼ (ఒక్కటే / tick)
   ┌─────────────────────┐       ┌─────────────────────┐
   │  MICROTASK QUEUE    │       │   MACROTASK QUEUE   │
   │ .then / catch / finally│     │ setTimeout / setInterval │
   │ queueMicrotask      │       │ setImmediate / I/O  │
```

```
│  MutationObserver         │        │  UI events / message      │
│  await continuation       │        │                           │
└───────────────────────────┘        └──────────────────────────┘
```

Code – microtask vs macrotask ordering

```
console.log("1: script start");

setTimeout(() => console.log("2: setTimeout (macrotask)"), 0);

Promise.resolve().then(() => console.log("3: promise 1 (microtask)"))
                 .then(() => console.log("4: promise 2 (microtask)"));

queueMicrotask(() => console.log("5: queueMicrotask"));

console.log("6: script end");

// ==== Predicted Output ====
// 1: script start        ← synchronous, వెంటనే
// 6: script end          ← synchronous, వెంటనే
// 3: promise 1 (microtask)  ← stack empty → microtasks మొదట
// 5: queueMicrotask         ← ఇదీ microtask
// 4: promise 2 (microtask)  ← మొదటి .then పూర్తయ్యాక queue అయింది
// 2: setTimeout (macrotask) ← అన్ని microtasks అయ్యాకే macrotask
```

గమనించండి: `setTimeout(..., 0)` అయినా, అన్ని microtasks (`3, 4, 5`) దాని కంటే ముందే run అవుతాయి. ఇదే **microtask starvation** కి దారితీయగలదు.

Code – nested ordering (interview favorite)

```
console.log("A");
setTimeout(() => {
  console.log("B");
  Promise.resolve().then(() => console.log("C"));
}, 0);
Promise.resolve().then(() => {
  console.log("D");
  setTimeout(() => console.log("E"), 0);
});
console.log("F");

// Output: A  F  D  B  C  E
// A, F → sync
// D → microtask (E ను macrotask గా schedule చేస్తుంది)
// B → మొదటి macrotask (C ను microtask గా schedule చేస్తుంది)
// C → B tick తర్వాత microtask drain
// E → చివరి macrotask
```

Starvation (సాధారణ తప్పులు / Gotchas)

- **Microtask starvation:** ఒక microtask నిరంతరం మరో microtask ని add చేస్తూ ఉంటే, macrotasks **ఎప్పటికీ** run అవ్వవు, rendering block అవుతుంది, page freeze అవుతుంది.

```
// ⚠️ ఇది page ని freeze చేస్తుంది — render ఎప్పటికీ జరగదు
function loop() { Promise.resolve().then(loop); }
loop();
```

- **Rendering blocking:** Microtask queue drain అయ్యేదాకా browser paint చేయదు. పెద్ద synchronous loops UI ని freeze చేస్తాయి.
- `setTimeout(fn, 0)` నిజంగా 0ms కాదు – HTML spec ప్రకారం minimum ~4ms clamp (nested timers కి), అలాగే stack empty అయ్యేదాకా wait.
- `await` తర్వాత వచ్చే code ఎప్పుడూ microtask గానే schedule అవుతుంది (synchronous కాదు).

Key Points

- JS single-threaded; concurrency ని event loop + runtime APIs అందిస్తాయి.
- **Microtasks > Macrotasks** priority. ప్రతి macrotask తర్వాత అన్ని microtasks.
- Promises, `await`, `queueMicrotask` → microtask. `setTimeout`, I/O, events → macrotask.
- Rendering microtasks తర్వాత, macrotasks మధ్యలో జరుగుతుంది.

Interview దృష్టి

- "setTimeout 0 vs Promise.then – ఏది ముందు run అవుతుంది?" → **Promise** (microtask).
- "Event loop ఎందుకు అవసరం?" → single thread లో non-blocking async కోసం.
- Node లో అదనంగా `process.nextTick` (microtasks కంటే ముందు!) మరియు phases (timers, poll, check) ఉంటాయి.

---

28. Callbacks & Callback Hell

వివరణ

Callback అంటే ఒక function ని argument గా మరో function కి pass చేయడం – తర్వాత అది "call back" (తిరిగి పిలవబడుతుంది) అవుతుంది. JavaScript లో functions **first-class citizens** కాబట్టి ఇది సాధ్యం (functions ని variables లా pass చేయవచ్చు, return చేయవచ్చు).

రెండు రకాలు:

- **Synchronous callback:** వెంటనే run అవుతుంది. ఉదా: `[1,2,3].map(x => x*2)` లో `x => x*2`.
- **Asynchronous callback:** తర్వాత ఎప్పుడో run అవుతుంది. ఉదా: `setTimeout`, event handlers, file reads.

Async programming కి callbacks మొదటి pattern. కానీ nested async operations ఎక్కువైతే – **Callback Hell** (aka "Pyramid of Doom") వస్తుంది.

Real-life Scenario

> మీరు **courier delivery** చేస్తున్నారు అనుకోండి. "Package deliver అయ్యాక నాకు call చెయ్యి" అని చెప్పడం = callback. కానీ "package1 deliver అయ్యాక, package2 pickup చెయ్యి, అది deliver అయ్యాక package3 pickup చెయ్యి, అది..." అని ఒక్కో step మరో దానిలో nest చేస్తూ పోతే – ఒక పెద్ద గజిబిజి instruction sheet అవుతుంది. ఏ step లో ఏ error వచ్చిందో track చేయడం nightmare. అదే callback hell.

Code – basic callback

```
function fetchUser(id, callback) {
  setTimeout(() => {
    console.log("User తెచ్చాం");
    callback(null, { id, name: "Ravi" }); // Node style: (error, data)
  }, 1000);
}

fetchUser(1, (err, user) => {
  if (err) return console.error("Error:", err);
  console.log("Got:", user); // Got: { id: 1, name: 'Ravi' }
});
```

Code – Callback Hell (Pyramid of Doom)

```
// ⚠️ ఇది చదవడం, maintain చేయడం చాలా కష్టం
getUser(1, (err, user) => {
  if (err) return handle(err);
  getOrders(user.id, (err, orders) => {
    if (err) return handle(err);
    getOrderDetails(orders[0].id, (err, details) => {
      if (err) return handle(err);
      getShipping(details.shipId, (err, shipping) => {
        if (err) return handle(err);
        console.log(shipping); // 4 levels deep! 😱
      });
    });
  });
});
```

సమస్యలు (Callback Hell problems)

సమస్య	వివరణ
Readability	కుడివైపుకి పెరిగే pyramid, code follow చేయడం కష్టం
Error handling	ప్రతి level లో `if (err)` repeat, ఒక్కటి మర్చిపోతే silent failure
Inversion of control	callback ఎప్పుడు, ఎన్నిసార్లు call అవుతుందో మనం control చేయలేం (3rd party code)
No return values	result ను compose చేయడం కష్టం

పరిష్కారాలు

1. Named functions (flatten చేయడం):

```
function onShipping(err, shipping) { console.log(shipping); }
function onDetails(err, details) { getShipping(details.shipId, onShipping); }
function onOrders(err, orders) { getOrderDetails(orders[0].id, onDetails); }
getUser(1, (err, user) => getOrders(user.id, onOrders));
```

2. Promises (అసలైన fix): chaining తో flat structure – Topic 29 చూడండి.

3. async/await (best): synchronous లా కనిపించే async code – Topic 30.

```
// Promises తో అదే logic – flat, readable
getUser(1)
  .then(user => getOrders(user.id))
  .then(orders => getOrderDetails(orders[0].id))
  .then(details => getShipping(details.shipId))
  .then(shipping => console.log(shipping))
  .catch(handle); // ఒకే చోట error handling! 🎉
```

Gotchas (సాధారణ తప్పులు)

- **Error-first convention** (Node.js): callback మొదటి argument ఎప్పుడూ `error`. దీన్ని పాటించకపోతే confusion.
- **Callback రెండుసార్లు call అవడం:** guard లేకపోతే bug. `return callback(...)` వాడండి.
- **Try/catch async callbacks ని పట్టుకోదు:** `setTimeout` లోపల throw అయితే బయటి try/catch పనిచేయదు.

```
try {
  setTimeout(() => { throw new Error("boom"); }, 0);
} catch (e) {
  // ఇది ఎప్పటికీ execute అవ్వదు! callback వేరే tick లో run అవుతుంది
}
```

Key Points

- Callback = function ని argument గా pass చేయడం; async కి foundation.
- Callback hell = deep nesting → readability + error handling సమస్యలు.
- Modern JS: Promises → async/await తో పరిష్కరించండి.
- Node.js "error-first" callback convention గుర్తుంచుకోండి.

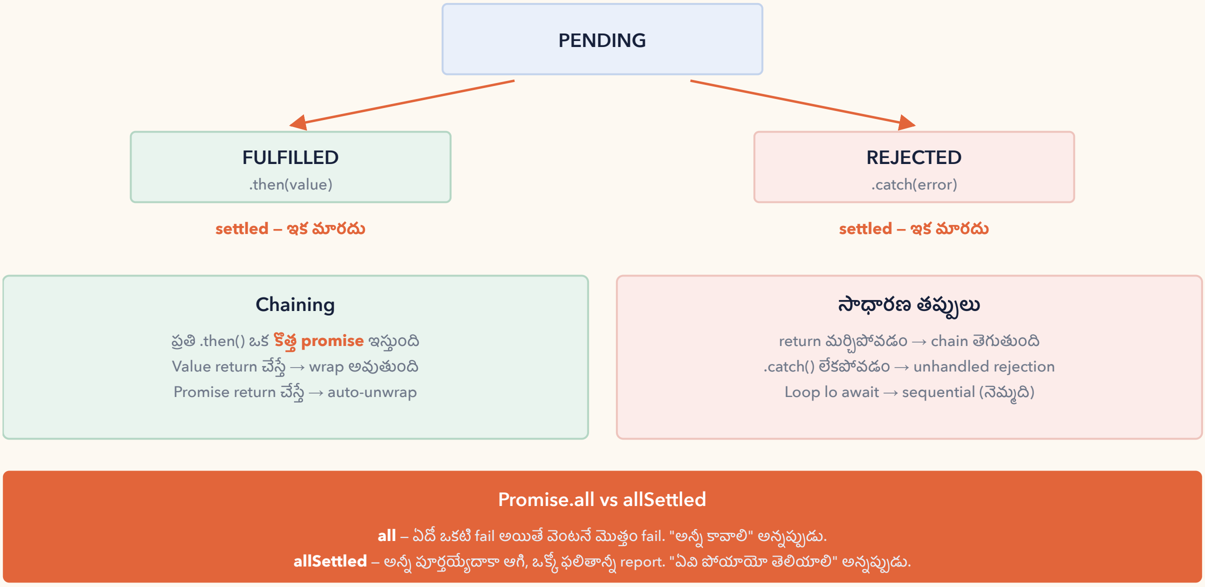
Interview దృష్టి

- "Callback hell ఎలా avoid చేస్తారు?" → Promises/async-await, modularization, named functions.
- "Inversion of control problem అంటే?" → callback control 3rd-party code చేతిలో ఉండటం; Promises దీన్ని పరిష్కరిస్తాయి (control తిరిగి caller కి).

29. Promises (states, then/catch/finally, chaining, Promise.all/allSettled/race/any)

PROMISES · స్థితులు మరియు CHAINING

PROMISE – మూడు స్థితులు, ఒక్కసారే మారతాయి



వివరణ

Promise అంటే భవిష్యత్తులో వచ్చే విలువకి (async operation result) ఒక **placeholder object**. ఇది "నేను ఇప్పుడు value ఇవ్వలేను, కానీ తర్వాత తప్పకుండా result లేదా error ఇస్తాను" అనే వాగ్దానం.

Promise కి **3 states** ఉంటాయి:

State	అర్థం
pending	ప్రారంభ state, ఇంకా పూర్తవలేదు
fulfilled (resolved)	operation విజయవంతం, value దొరికింది
rejected	operation విఫలం, error వచ్చింది

ముఖ్యం: Promise ఒకసారి settle (fulfilled/rejected) అయ్యాక **మారదు** – immutable. ఇదే "settled" state.

Real-life Scenario

> మీరు **pizza order** చేసారు, shop వాళ్ళు **token/receipt** (Promise) ఇచ్చారు. ప్రస్తుతం pizza లేదు (pending). రెండు అవకాశాలు: pizza వస్తుంది (fulfilled) లేదా "ingredients అయిపోయాయి" అని refund (rejected). Token తీసుకున్నప్పుడే మీరు "pizza వచ్చాక తినేస్తా" (`.then`), "cancel అయితే వేరే order చేస్తా" (`.catch`), "ఏదైనా receipt shop కి తిరిగి ఇస్తా" (`.finally`) అని ప్లాన్ చెప్పేస్తారు.

Code – Promise సృష్టించడం, వాడడం

```
const orderPizza = new Promise((resolve, reject) => {
  const success = true;
  setTimeout(() => {
    if (success) resolve("🍕 Pizza ready!");
    else reject(new Error("Ingredients అయిపోయాయి"));
  }, 1000);
});

orderPizza
  .then(pizza => console.log("Got:", pizza))   // fulfilled → value
  .catch(err => console.error("Failed:", err.message)) // rejected → error
  .finally(() => console.log("Order closed")); // ఏదైనా జరుగుతుంది

// Output (1s తర్వాత):
// Got: 🍕 Pizza ready!
// Order closed
```

then / catch / finally

- `.then(onFulfilled, onRejected)` – రెండో argument కూడా rejection ని పట్టుకుంటుంది, కానీ `.catch` preferred (chain లో పైన వచ్చిన errors అన్నీ పట్టుకుంటుంది).
- `.catch(fn)` = `.then(null, fn)` కి syntactic sugar.
- `.finally(fn)` – value/error ని మార్చదు, cleanup కి (loader hide చేయడం లాంటివి).

Chaining – flat structure

ప్రతి `.then` ఒక **కొత్త Promise** return చేస్తుంది. `.then` లో return చేసిన value next `.then` కి forward అవుతుంది. Promise return చేస్తే, అది settle అయ్యేదాకా wait చేస్తుంది (auto-flatten).

```
Promise.resolve(2)
  .then(n => n * 3)          // 6 (plain value → wrap అవుతుంది)
  .then(n => Promise.resolve(n + 1)) // 7 (Promise → unwrap అవుతుంది)
  .then(n => { throw new Error("stop at " + n); }) // reject
  .then(n => console.log("skipped")) // ← skip అవుతుంది
  .catch(e => console.log("Caught:", e.message)); // Caught: stop at 7
```

Promise combinators – comparison table

Method	ఎప్పుడు settle అవుతుంది	Result	ఎప్పుడు వాడాలి
`Promise.all`	అన్నీ fulfill అయితే / **ఏదైనా ఒకటి reject అయితే వెంటనే**	values array / మొదటి error	అన్నీ కావాలి, ఒకటి fail అయినా fail
`Promise.allSettled`	**అన్నీ** settle అయ్యాక (reject అయినా)	`{status, value/reason}` array	అన్ని results కావాలి, fail అయినా OK
`Promise.race`	**మొదటిది** settle అయితే (fulfill/reject)	ఆ మొదటి result/error	timeout patterns, fastest wins

Method	ఎప్పుడు settle అవుతుంది	Result	ఎప్పుడు వాడాలి
`Promise.any`	**మొదటి fulfill** అయితే / అన్నీ reject అయితే	మొదటి success / `AggregateError`	ఏదో ఒకటి success చాలు

Code – combinators

```
const p1 = Promise.resolve("A");
const p2 = new Promise(r => setTimeout(() => r("B"), 100));
const p3 = Promise.reject(new Error("C failed"));

// all — ఒకటి fail → మొత్తం fail
Promise.all([p1, p2]).then(v => console.log(v)); // ["A", "B"]
Promise.all([p1, p3]).catch(e => console.log(e.message)); // "C failed"

// allSettled — అన్నీ report
Promise.allSettled([p1, p3]).then(r => console.log(r));
// [ {status:"fulfilled", value:"A"},
//   {status:"rejected", reason: Error("C failed")} ]

// race — మొదటిది గెలుస్తుంది
Promise.race([p2, p1]).then(v => console.log(v)); // "A" (p1 వెంటనే)

// any — మొదటి success
Promise.any([p3, p2]).then(v => console.log(v)); // "B" (p3 fail, p2 success)
```

Real-world: timeout pattern (race)

```
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error("Timeout!")), ms)
  );
  return Promise.race([promise, timeout]);
}
withTimeout(fetch("/api/data"), 5000)
  .then(res => console.log("Got in time"))
  .catch(err => console.log(err.message)); // 5s లో రాకపోతే "Timeout!"
```

Internals – Promises microtasks

`.then/.catch/.finally` callbacks ఎప్పుడూ **microtask queue** లోకి వెళ్తాయి (Topic 27). అందుకే `setTimeout` కంటే ముందు run అవుతాయి. Already-settled promise పైన `.then` పెట్టినా, callback వెంటనే కాకుండా current sync code అయ్యాక microtask గా run అవుతుంది.

Gotchas (సాధారణ తప్పులు)

- **Missing return in chain:** `.then(user => { getOrders(user.id); })` – return లేదు కాబట్టి next `.then` కి `undefined` వెళ్తుంది.
- **Unhandled rejection:** `.catch` లేకపోతే → "UnhandledPromiseRejection" warning/crash.
- **Promise constructor executor synchronous గా run అవుతుంది** – `new Promise(fn)` లో `fn` వెంటనే execute.

- `.catch` **తర్వాత chain కొనసాగుతుంది** (recover అయినట్లు) – తర్వాతి `.then` run అవుతుంది.
- `Promise.all` fail-fast – ఒకటి reject అయితే మిగతావి cancel కావు, కానీ result ignore అవుతుంది.

Key Points

- 3 states: pending → fulfilled/rejected (settle అయ్యాక immutable).
- Chaining flat structure ఇస్తుంది; ప్రతి `.then` కొత్త promise.
- `all` (అన్నీ/fail-fast), `allSettled` (అన్నీ report), `race` (మొదటిది), `any` (మొదటి success).
- Callbacks microtasks; `.catch` chain-wide error handling.

Interview దృష్టి

- "all vs allSettled?" → all fail-fast, allSettled ఎప్పుడూ అన్నీ report.
- "race vs any?" → race మొదటి settle (reject అయినా), any మొదటి fulfill.
- "Promise resolve అయ్యాక మళ్ళీ resolve చేస్తే?" → మొదటిదే stays; తర్వాతివి ignore.

---

30. async/await (try/catch, sequential vs parallel, top-level await, mistakes)

ASYNC/AWAIT · SEQUENTIAL VS PARALLEL

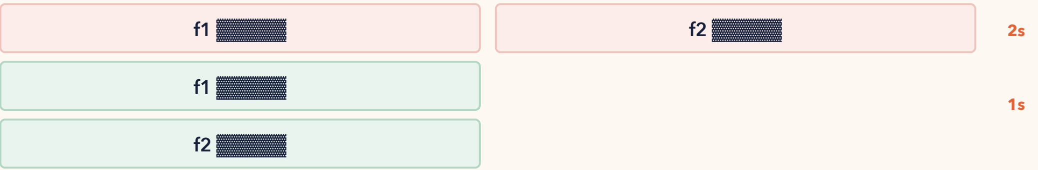
SEQUENTIAL vs PARALLEL await – ఇది చాలా ముఖ్యం

Sequential (నెమ్మది)

```
const a = await f1(); // 1s
const b = await f2(); // 1s
మొత్తం 2 సెకన్లు
```

Parallel (వేగం)

```
const [a,b] = await Promise.all(
[f1(), f2()]);
మొత్తం 1 సెకను
```



ఎప్పుడు sequential అవసరం

రెండోది మొదటి ఫలితం మీద **ఆధారపడితే** మాత్రమే.
ఆధారపడకపోతే – for loop lo await రాయడం ఒక performance bug.
Loop కావాలంటే:

```
await Promise.all(items.map(async x => …))
```

వివరణ

`async/await` అనేది Promises పైన ఉన్న **syntactic sugar** – async code ని synchronous code లా చదవగలిగేలా చేస్తుంది. ఇది కొత్త mechanism కాదు, promises పైనే పనిచేస్తుంది.

- `async` function ఎప్పుడూ **Promise return చేస్తుంది** (మీరు plain value return చేసినా అది promise లో wrap అవుతుంది).

- `await` అనేది promise settle అయ్యేదాకా function execution ను **pause** చేస్తుంది (కానీ ఇతర code block అవ్వదు – event loop స్వేచ్ఛగా ఉంటుంది).

Real-life Scenario

> **Callbacks/Promises** = మీరు friend కి "పని అయ్యాక నాకు message చెయ్యి, అప్పుడు నేను next పని చేస్తా" అని instructions ఇవ్వడం. **async/await** = మీరు ఆ పని పూర్తయ్యేదాకా అక్కడే ఆగి (`await`), కానీ ఈలోపు మీ brain freeze అవ్వకుండా (thread block కాదు) వేరే వాళ్ళకి కూడా help చేస్తూ, పని పూర్తయ్యాక మళ్ళీ ఇక్కడి నుంచి కొనసాగడం. Code పైనుంచి కిందకి సాఫీగా చదవొచ్చు.

Code – basic

```
function delay(ms, val) {
  return new Promise(res => setTimeout(() => res(val), ms));
}

async function getData() {
  console.log("Start");
  const a = await delay(1000, "A"); // 1s wait
  console.log("Got", a);            // Got A
  const b = await delay(1000, "B"); // మరో 1s wait
  console.log("Got", b);            // Got B
  return "done";
}

getData().then(r => console.log(r)); // Start → (2s) → Got A → Got B → done
```

Error handling – try/catch

async/await తో errors ని synchronous code లాగే `try/catch` తో పట్టుకోవచ్చు – ఇది async/await యొక్క పెద్ద ప్లస్.

```
async function loadUser(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const user = await res.json();
    return user;
  } catch (err) {
    console.error("Load failed:", err.message);
    return null; // fallback
  } finally {
    console.log("Load attempt complete"); // ఎప్పుడూ run
  }
}
```

Sequential vs Parallel – చాలా ముఖ్యం!

ఇది interviews లో, real bugs లో అత్యంత సాధారణ topic. Independent async operations ని **sequential** గా await చేస్తే time వృథా.

```
// ❌ SEQUENTIAL – 3 seconds (ఒకదాని తర్వాత ఒకటి)
async function slow() {
  const a = await delay(1000, "A"); // 1s wait
  const b = await delay(1000, "B"); // అది అయ్యాకే start → మరో 1s
  const c = await delay(1000, "C"); // మరో 1s
  return [a, b, c]; // మొత్తం 3s
}

// ✅ PARALLEL – 1 second (అన్నీ ఒకేసారి start)
async function fast() {
  const pa = delay(1000, "A"); // వెంటనే start (await లేదు)
  const pb = delay(1000, "B"); // వెంటనే start
  const pc = delay(1000, "C"); // వెంటనే start
  return [await pa, await pb, await pc]; // అన్నీ ఒకేసారి → 1s
}

// ✅ ఇంకా మంచిది – Promise.all
async function best() {
  return Promise.all([delay(1000,"A"), delay(1000,"B"), delay(1000,"C")]);
} // 1s, cleaner
```

నియమం: operations ఒకదానిపై ఒకటి ఆధారపడితే (dependent) → sequential. Independent అయితే → parallel (`Promise.all`).

Top-level await

ES2022 నుండి, **ES modules** లో function బయట (top level) `await` వాడొచ్చు. CommonJS లో కుదరదు.

```
// module.mjs – top-level await
const config = await fetch("/config.json").then(r => r.json());
export { config };
// ఈ module ని import చేసే modules, config load అయ్యేదాకా wait చేస్తాయి
```

గమనిక: top-level await module loading ని block చేస్తుంది – dependent modules delay అవుతాయి, జాగ్రత్తగా వాడాలి.

async/await vs Promises – comparison

అంశం	Promises (.then)	async/await
Readability	chaining, కొంచెం nested	synchronous లా, flat
Error handling	`.catch()`	`try/catch` (familiar)
Debugging	stack traces అస్పష్టం	clean stack traces, breakpoints పనిచేస్తాయి

అంశం	Promises (.then)	async/await
Conditional/loops లో async	కష్టం	సులభం (if/for లోపల await)
Parallel	సహజం (`Promise.all`)	జాగ్రత్త అవసరం (sequential trap)

Gotchas / మిస్టేక్‌లు

- **`await` in a `forEach`** పనిచేయదు – `forEach` async callback కి wait చేయదు. `for...of` వాడండి.

```
// ❌ wait చేయదు — అన్నీ ఒకేసారి fire, order guarantee లేదు
[1,2,3].forEach(async id => { await save(id); });

// ✅ sequential
for (const id of [1,2,3]) { await save(id); }

// ✅ parallel
await Promise.all([1,2,3].map(id => save(id)));
```

- **Sequential trap** (పైన చూసినట్లు) – independent awaits time వృథా.
- **Forgotten await:** `const x = asyncFn();` → x ఒక Promise, actual value కాదు.
- **`return await` in try/catch:** try block లో `return await` వాడాలి, లేకపోతే rejection catch కాదు (bare `return promise` అయితే catch miss అవుతుంది).
- **Unhandled rejection:** await చేసిన promise reject అయి try/catch లేకపోతే crash.

Key Points

- `async` fn ఎప్పుడూ Promise return; `await` execution ను pause (thread block కాదు).
- Errors → `try/catch/finally`.
- Independent operations → parallel (`Promise.all`), dependent → sequential.
- `forEach` తో await పనిచేయదు; `for...of` లేదా `Promise.all`.
- Top-level await ES modules లో మాత్రమే.

Interview దృష్టి

- "async/await internally ఎలా పనిచేస్తుంది?" → Promises + generators; `await` continuation microtask గా schedule.
- "3 independent API calls fastest ఎలా?" → `Promise.all`.
- "await తర్వాత code ఎప్పుడు run?" → promise settle అయ్యాక, microtask గా.

31. Generators & Iterators (function*, yield, custom iterables, async generators, for-await-of)

వివరణ

Iterator అనేది ఒక object తో `next()` method ఉంటుంది, అది `{ value, done }` return చేస్తుంది. **Iterable** అంటే `Symbol.iterator` method ఉన్న object (arrays, strings, Maps, Sets). `for...of`, spread `[...x]`, destructuring – ఇవన్నీ iterables పైన పనిచేస్తాయి.

Generator అనేది ఒక ప్రత్యేక function (`function*`) – ఇది execution ని **pause/resume** చేయగలదు. `yield` దగ్గర ఆగుతుంది, `next()` పిలిస్తే అక్కడి నుండి కొనసాగుతుంది. Generator దానంతట అదే iterator + iterable.

Real-life Scenario

> **Netflix "Next Episode" (lazy) vs మొత్తం download.** ఒక series మొత్తం ఒకేసారి download చేయడం = array (అంతా memory లో). Generator = మీరు "next episode" నొక్కినప్పుడే (`next()`) ఆ episode ని stream చేయడం. అవసరమైనప్పుడు మాత్రమే produce చేయడం – **lazy evaluation**. Infinite series కూడా possible (మొత్తం memory లో పట్టనవసరం లేదు).

Code – basic generator

```
function* counter() {
  console.log("start");
  yield 1;      // ఇక్కడ pause
  console.log("resumed");
  yield 2;
  return 3;     // done: true తో
}

const it = counter();
console.log(it.next()); // "start" → { value: 1, done: false }
console.log(it.next()); // "resumed" → { value: 2, done: false }
console.log(it.next()); // { value: 3, done: true }
console.log(it.next()); // { value: undefined, done: true }
```

గమనించండి: `counter()` పిలిస్తే వెంటనే body run **అవ్వదు** – iterator object మాత్రమే వస్తుంది. మొదటి `next()` కే "start" print అవుతుంది.

Code – infinite lazy sequence

```
function* naturals() {
  let n = 1;
  while (true) yield n++; // infinite – కానీ lazy కాబట్టి OK
}

const nums = naturals();
```

```
console.log(nums.next().value); // 1
console.log(nums.next().value); // 2
console.log(nums.next().value); // 3
// Array తో ఇది impossible — memory అయిపోతుంది. Generator తో సురక్షితం.
```

Custom iterable – Symbol.iterator

ఏ object ని అయినా `for...of` తో వాడేలా చేయవచ్చు – `Symbol.iterator` implement చేస్తే.

```
const range = {
  from: 1, to: 5,
  [Symbol.iterator]() {          // generator తో సులభం
    let current = this.from, last = this.to;
    return {
      next() {
        return current <= last
          ? { value: current++, done: false }
          : { value: undefined, done: true };
      }
    };
  }
};
console.log([...range]);        // [1, 2, 3, 4, 5]
for (const n of range) process.stdout.write(n + " "); // 1 2 3 4 5

// Generator వాడితే ఇంకా clean:
const range2 = {
  from: 1, to: 5,
  *[Symbol.iterator]() { for (let i = this.from; i <= this.to; i++) yield i; }
};
console.log([...range2]); // [1, 2, 3, 4, 5]
```

Two-way communication – next(value), yield expression

`yield` ఒక value ని బయటకు పంపడమే కాదు – `next(x)` ద్వారా బయటనుండి value ని లోపలికి తీసుకోగలదు.

```
function* conversation() {
  const name = yield "మీ పేరు?";      // next() తో వచ్చే value ఇక్కడ
  const age = yield `Hi ${name}, వయసు?`;
  return `${name}, ${age} years`;
}
const c = conversation();
console.log(c.next().value);      // "మీ పేరు?"
console.log(c.next("Ravi").value);// "Hi Ravi, వయసు?"
console.log(c.next(25).value);    // "Ravi, 25 years"
```

Async generators & for-await-of

Async generator (`async function*`) – `yield` చేసే values promises కావొచ్చు. `for await...of` తో iterate చేస్తాం. Streaming data (paginated APIs, file chunks) కి perfect.

```
async function* fetchPages(url) {
  let page = 1;
  while (page <= 3) {
    // నిజంగా అయితే: const data = await fetch(`${url}?page=${page}`);
    const data = await Promise.resolve([`item-${page}-a`, `item-${page}-b`]);
    yield data;   // ప్రతి page ని lazily yield
    page++;
  }
}

(async () => {
  for await (const page of fetchPages("/api")) {
    console.log(page); // ["item-1-a","item-1-b"] → page 2 → page 3
  }
})();
```

Iterator vs Generator – comparison

అంశం	Iterator (manual)	Generator (`function*`)
Syntax	`next()` manually implement	`yield` – engine handle చేస్తుంది
State	మనం track చేయాలి	automatic (pause/resume)
Code	verbose	concise
Return	iterator object	generator (iterator + iterable)

Internals – ఎలా pause అవుతుంది?

Generator function యొక్క execution context ని engine **suspend** చేసి పక్కన పెడుతుంది (call stack నుండి తీసేసి). `next()` పిలిచినప్పుడు ఆ saved context ని తిరిగి తెచ్చి, local variables, position అన్నీ restore చేసి, తర్వాతి `yield` దాకా run చేస్తుంది. ఇదే async/await కి foundation (async/await = generators + promises + auto-runner).

Gotchas (సాధారణ తప్పులు)

- Generator call వెంటనే run అవ్వదు – iterator return అవుతుంది; `next()` కావాలి.
- Arrow functions generators కావు – `function*` మాత్రమే.
- ఒకసారి `done: true` అయ్యాక reset లేదు – కొత్త generator సృష్టించాలి.
- `yield` ను generator function లోపలే వాడాలి (nested normal function లో కాదు; `yield*` delegation కి).

Key Points

- Iterable = `Symbol.iterator` ; Iterator = `next()` → `{value, done}` .
- Generator = pausable function (`function*` , `yield`), lazy evaluation, infinite sequences.
- Two-way: `yield` బయటకు, `next(v)` లోపలికి.
- Async generators + `for await...of` = streaming async data.

Interview దృష్టి

- "async/await internally?" → generators + promises.
- "Infinite sequence ఎలా?" → generator (lazy).
- "for...of ఎలా పనిచేస్తుంది?" → `Symbol.iterator` invoke చేసి `next()` loop.

---

32. ES Modules (import/export, dynamic import, CommonJS vs ESM, tree-shaking)

ES MODULES VS COMMONJS

వివరణ

Module అంటే తనదైన scope ఉన్న ఒక file – దానిలోని variables/functions default గా private. మీరు explicit గా `export` చేసినవి మాత్రమే బయటకు కనిపిస్తాయి, `import` తో వాడతారు. దీనివల్ల code organized, reusable, namespace pollution లేకుండా ఉంటుంది.

JavaScript లో రెండు major module systems:

- **ESM (ES Modules)** – modern standard, `import` / `export`, browsers + Node రెండింటిలో.
- **CommonJS (CJS)** – Node.js యొక్క పాత system, `require()` / `module.exports`.

Real-life Scenario

> ఒక **factory** లో వేర్వేరు departments (modules). ప్రతి department తన internal పరికరాలను (private variables) బయటకు చూపించదు. కానీ "finished products" (exports) మాత్రం loading dock ద్వారా బయటకి పంపుతుంది, వేరే departments వాటిని తీసుకుంటాయి (import). ప్రతి department స్వతంత్రంగా పనిచేస్తుంది – ఒకదాని internal మార్పు మరొకదాన్ని affect చేయదు.

Code – named & default exports

```
// math.js
export const PI = 3.14159;            // named export
```

```
export function add(a, b) { return a + b; } // named export
export default function multiply(a, b) { return a * b; } // default (ఒక్కటే)

// అలాగే export చేయవచ్చు:
const sub = (a, b) => a - b;
export { sub };                          // grouped named export
```

```
// app.js
import multiply, { PI, add, sub } from "./math.js";
// multiply = default, {} లోనివి named
import * as math from "./math.js";       // namespace import
import { add as sum } from "./math.js";  // rename

console.log(add(2, 3));      // 5
console.log(multiply(2, 3)); // 6
console.log(math.PI);        // 3.14159
```

Dynamic import – on-demand loading

`import()` function form ఒక **Promise** return చేస్తుంది. Code splitting, lazy loading, conditional loading కి వాడతారు.

```
// అవసరమైనప్పుడు మాత్రమే load — bundle size తగ్గుతుంది
button.addEventListener("click", async () => {
  const { default: Chart } = await import("./heavy-chart.js");
  new Chart().render(); // click అయ్యాకే download అవుతుంది
});

// conditional loading
if (user.isAdmin) {
  const admin = await import("./admin-panel.js");
  admin.init();
}
```

CommonJS vs ESM – comparison (ముఖ్యమైన table)

అంశం	CommonJS (CJS)	ES Modules (ESM)
Syntax	`require()`, `module.exports`	`import`, `export`
Loading	**synchronous** (runtime లో)	**asynchronous**, static analysis
Timing	dynamic (runtime లో resolve)	static (parse time లో resolve)
`this` at top	`module.exports`	`undefined`
Tree-shaking	❌ కష్టం (dynamic)	✅ సాధ్యం (static)
Top-level await	❌	✅
File extension	`.cjs` / `.js`	`.mjs` / `.js` (`"type":"module"`)
Binding	value **copy**	live **reference** (read-only)

అంశం	CommonJS (CJS)	ES Modules (ESM)
Environment	Node.js (traditional)	browsers + modern Node

Live bindings – కీలక తేడా

ESM imports **live read-only references** – exporting module లో value మారితే, import చేసిన చోట కూడా మారుతుంది. CJS లో అది copy.

```
// counter.js
export let count = 0;
export function inc() { count++; }

// app.js
import { count, inc } from "./counter.js";
console.log(count); // 0
inc();
console.log(count); // 1 ← live binding! (copy అయితే 0 వచ్చేది)
// count = 5;  // ❌ Error – imports read-only
```

Tree-shaking

Tree-shaking అంటే bundler (Webpack, Rollup, esbuild) unused exports ని dead code గా గుర్తించి final bundle నుండి తీసేయడం. ESM యొక్క **static structure** (imports/exports compile time లో తెలుస్తాయి) దీన్ని సాధ్యం చేస్తుంది.

```
// utils.js – 3 functions export
export function used() {}
export function unused1() {}
export function unused2() {}

// app.js – ఒక్కటే import
import { used } from "./utils.js";
used();
// → bundler unused1, unused2 ని తీసేస్తుంది (tree-shaken)
```

Tree-shaking సరిగ్గా పనిచేయాలంటే: ESM syntax వాడాలి, side-effect-free code (`"sideEffects": false` in package.json), named imports (namespace import కాదు).

Internals – module loading phases

ESM loading 3 phases లో జరుగుతుంది:

1. **Construction/Parse:** files download + parse, dependency graph build (imports resolve).
2. **Instantiation:** memory allocate, exports/imports link (live bindings wire చేయడం), కానీ code run అవ్వదు.
3. **Evaluation:** module code top-to-bottom run, values fill అవుతాయి.

Modules **singleton** – ఒక module ఎన్నిసార్లు import చేసినా, ఒకేసారి evaluate అవుతుంది, cache అవుతుంది.

Gotchas (సాధారణ తప్పులు)

- **Circular dependencies:** A imports B, B imports A. ESM handle చేస్తుంది కానీ evaluation order వల్ల `undefined` రావొచ్చు – architecture సరిచేయండి.
- **`.js` extension** ESM లో browsers/Node లో explicit గా అవసరం (`./math.js`, `./math` కాదు).
- **ESM ↔ CJS mixing:** CJS లో `import` పనిచేయదు; ESM లో `require` పనిచేయదు (interop జాగ్రత్త).
- Default export ను ఏ పేరుతోనైనా import చేయవచ్చు – typos silent bugs.
- `import` statements **hoisted** – file పైకి move అవుతాయి, ఏ order లో రాసినా.

Key Points

- Module = own scope; `export` చేసినవి మాత్రమే బయటకు.
- ESM static (tree-shaking, top-level await, live bindings), CJS dynamic synchronous (copies).
- Dynamic `import()` → Promise, lazy/conditional loading.
- Tree-shaking ESM static structure వల్ల unused code తీసేస్తుంది.

Interview దృష్టి

- "CJS vs ESM ముఖ్య తేడా?" → static vs dynamic, async vs sync, tree-shakeable, live bindings vs copies.
- "Tree-shaking ఎందుకు ESM లోనే?" → static imports compile-time analysis.
- "Dynamic import ఎప్పుడు?" → code splitting, lazy load, conditional.

---

33. Error Handling (try/catch/finally, throw, Error types, custom errors, async errors)

వివరణ

Error handling అంటే program crash అవ్వకుండా, unexpected situations (network fail, invalid input, bugs) ని gracefully handle చేయడం. JavaScript లో errors **thrown** అవుతాయి, `try/catch` తో **caught** అవుతాయి.

- `try { }` – risky code ఇక్కడ.
- `catch (err) { }` – error వస్తే ఇది run.
- `finally { }` – error వచ్చినా రాకపోయినా **ఎప్పుడూ** run (cleanup కి).
- `throw` – స్వయంగా error raise చేయడం.

Real-life Scenario

> **Circus లో trapeze artist** ఊహించుకోండి. `try` = artist risky jump చేస్తున్నాడు. కింద **safety net** (`catch`) – jump fail అయితే artist ని పట్టుకుంటుంది, show ఆగదు. `finally` = jump success అయినా, net పట్టినా – ప్రతిసారీ లైట్లు ఆర్పి, ground clean చేయడం. Net లేకపోతే (try/catch లేకపోతే) ఒక్క fall తో whole show (program) crash.

Code – try/catch/finally

```
function divide(a, b) {
  try {
    if (b === 0) throw new Error("Zero తో divide చేయకూడదు");
    return a / b;
  } catch (err) {
    console.error("Caught:", err.message);
    return NaN;
  } finally {
    console.log("Division attempt పూర్తయింది"); // ఎప్పుడూ run
  }
}
console.log(divide(10, 2)); // "పూర్తయింది" → 5
console.log(divide(10, 0)); // "Caught: Zero..." → "పూర్తయింది" → NaN
```

Built-in Error types

Type	ఎప్పుడు వస్తుంది
`Error`	generic base error
`TypeError`	wrong type పైన operation (`null.foo` , `undefined()` call)
`ReferenceError`	undefined variable access (`console.log(x)` where x లేదు)
`SyntaxError`	invalid code (parse time; `JSON.parse("{")`)
`RangeError`	valid range బయట value (`new Array(-1)` , deep recursion)
`URIError`	`decodeURIComponent("%")` లాంటివి
`AggregateError`	multiple errors (`Promise.any` reject)

```
try { null.foo; } catch (e) { console.log(e.constructor.name); } // TypeError
try { undefinedVar; } catch (e) { console.log(e.constructor.name); } //
ReferenceError
```

Error object properties

```
const err = new Error("something broke");
console.log(err.message); // "something broke"
```

```
console.log(err.name);    // "Error"
console.log(err.stack);   // call stack trace (debugging కి)
```

Custom errors – Error extend చేయడం

Domain-specific errors కోసం `Error` ని extend చేయండి. దీనివల్ల `instanceof` తో specific handling చేయవచ్చు.

```
class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = "ValidationError"; // stack లో సరైన పేరు
    this.field = field;            // extra context
  }
}

class NotFoundError extends Error {
  constructor(resource) {
    super(`${resource} దొరకలేదు`);
    this.name = "NotFoundError";
    this.statusCode = 404;
  }
}

function validate(user) {
  if (!user.email) throw new ValidationError("Email అవసరం", "email");
}

try {
  validate({});
} catch (err) {
  if (err instanceof ValidationError) {
    console.log(`Field '${err.field}': ${err.message}`); // Field 'email': Email
అవసరం
  } else {
    throw err; // తెలియని errors ని re-throw
  }
}
```

Async error handling

Promises: `.catch()` వాడండి.

```
fetch("/api").then(r => r.json()).catch(err => console.error(err));
```

async/await: `try/catch` వాడండి.

```
async function load() {
  try {
    const res = await fetch("/api");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
```

```
    console.error("Load failed:", err.message);
  }
}
```

కీలక gotcha: synchronous `try/catch` async callback errors ని పట్టుకోదు (Topic 27, 28).

```
try {
  setTimeout(() => { throw new Error("boom"); }, 0);
} catch (e) { /* ❌ ఇది పనిచేయదు — callback వేరే tick */ }
```

Global error handlers (last resort)

```
// Browser — uncaught synchronous errors
window.addEventListener("error", e => console.log("Global:", e.message));
// Browser — unhandled promise rejections
window.addEventListener("unhandledrejection", e => console.log("Unhandled:",
e.reason));

// Node.js
process.on("uncaughtException", err => { console.error(err); process.exit(1); });
process.on("unhandledRejection", reason => console.error(reason));
```

Gotchas (సాధారణ తప్పులు)

- `finally` **లో return** try/catch return ని **overwrite** చేస్తుంది — జాగ్రత్త.

```
function f() { try { return 1; } finally { return 2; } } // returns 2! 😱
```

- **Silent catch:** `catch (e) {}` — errors మింగేయడం worst practice; కనీసం log చేయండి.
- **Non-Error throw:** `throw "string"` valid కానీ చెడ్డది — stack trace ఉండదు; ఎప్పుడూ `Error` object throw చేయండి.
- `this.name` **set చేయకపోతే** custom error `"Error"` గా చూపిస్తుంది.
- **Catch and rethrow:** తెలియని errors ని swallow చేయకుండా re-throw చేయండి.

Key Points

- `try/catch/finally`; `finally` ఎప్పుడూ run (cleanup).
- Built-in types: TypeError, ReferenceError, SyntaxError, RangeError.
- Custom errors → `extend Error`, `this.name` set చేయండి, `instanceof` తో handle.
- Async: promises → `.catch`, async/await → `try/catch`; sync try/catch async callbacks ని పట్టుకోదు.
- ఎప్పుడూ Error objects throw చేయండి (strings కాదు).

Interview దృష్టి

- "try/catch async errors ఎందుకు పట్టుకోదు?" → callback వేరే event loop tick లో, stack వేరు.
- "finally return overwrite చేస్తుందా?" → అవును, avoid చేయండి.

- "Custom error ఎలా?" → Error extend, name set.

---

34. Regular Expressions (flags, groups, lookahead/behind, methods)

వివరణ

Regular Expression (regex) అంటే text లో patterns ని match/search/replace చేసే ఒక mini-language. JavaScript లో regex ఒక object (`RegExp`). రెండు రకాలుగా create చేయవచ్చు:

```
const re1 = /pattern/flags;              // literal (compile time, faster)
const re2 = new RegExp("pattern", "flags"); // constructor (dynamic patterns)
```

Real-life Scenario

> Regex అంటే ఒక **అనుభవజ్ఞుడైన సెక్యూరిటీ గార్డ్** – ఒక template ("ID card ఇలా ఉండాలి: 2 అక్షరాలు + 4 అంకెలు") పట్టుకుని, వచ్చే ప్రతి వ్యక్తి ID ని ఆ pattern తో సరిపోతుందా అని check చేస్తాడు. Pattern చాలా strict గానో, flexible గానో రాయవచ్చు. కానీ pattern చాలా complex అయితే గార్డ్ కూడా confuse అవుతాడు (unreadable regex).

Flags (modifiers)

Flag	అర్థం
`g`	global – అన్ని matches (మొదటిది మాత్రమే కాదు)
`i`	case-insensitive
`m`	multiline – `^` / `$` ప్రతి line కి
`s`	dotAll – `.` newline ని కూడా match
`u`	unicode – proper unicode handling
`y`	sticky – `lastIndex` నుండే match
`d`	indices – match positions ఇస్తుంది

Common patterns / metacharacters

```
.      ఏదైనా ఒక char (newline తప్ప)
\d \D  digit / non-digit
\w \W  word char [A-Za-z0-9_] / non-word
\s \S  whitespace / non-whitespace
^  $   line start / end
\b     word boundary
```

```
*      0+   +   1+   ?   0 లేదా 1
{n}    సరిగ్గా n   {n,}  n+   {n,m}  n నుండి m
[abc]  a లేదా b లేదా c   [^abc] కాకుండా   [a-z] range
|      OR   ( )  group   (?: )  non-capturing group
```

Methods – comparison table

Method	Owner	Returns	వాడకం
`test()`	RegExp	`true/false`	match ఉందా లేదా
`exec()`	RegExp	match array / `null`	detailed match (groups తో), `g` తో loop
`match()`	String	array / null	matches (g లేకుంటే groups)
`matchAll()`	String	iterator	అన్ని matches + groups (g అవసరం)
`replace()`	String	new string	replace (function కూడా)
`replaceAll()`	String	new string	అన్నీ replace (g అవసరం లేదు)
`split()`	String	array	regex తో split
`search()`	String	index / -1	మొదటి match position

Code – basics

```
const text = "Contact: ravi@mail.com, priya@work.org";

// test — ఉందా?
console.log(/\d/.test("abc123")); // true

// match with g — అన్ని emails
const emails = text.match(/\w+@\w+\.\w+/g);
console.log(emails); // ["ravi@mail.com", "priya@work.org"]

// replace with function
const masked = text.replace(/(\w+)@(\w+)/g, (m, user, domain) =>
  `${user[0]}***@${domain}`);
console.log(masked); // "Contact: r***.com, p***.org"

// split
console.log("a1b2c3".split(/\d/)); // ["a", "b", "c", ""]
```

Groups – capturing, named, backreferences

```
// Capturing groups — ( )
const date = "2026-07-16";
const m = date.match(/(\d{4})-(\d{2})-(\d{2})/);
console.log(m[1], m[2], m[3]); // "2026" "07" "16"

// Named groups — (?<name>...)
const m2 = date.match(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/);
```

```
console.log(m2.groups.year, m2.groups.month); // "2026" "07"

// Backreference — \1 (అదే group మళ్ళీ)
console.log(/(\w)\1/.test("hello")); // true (ll — double letter)
console.log(/(\w)\1/.test("world")); // false

// replace with named groups
console.log(date.replace(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/, "$<d>/$<m>/$<y>"));
// "16/07/2026"
```

Lookahead & Lookbehind (assertions)

Assertions match చేస్తాయి కానీ **consume చేయవు** (result లో include అవ్వవు).

Syntax	పేరు	అర్థం
`X(?=Y)`	positive lookahead	X, తర్వాత Y ఉంటేనే
`X(?!Y)`	negative lookahead	X, తర్వాత Y లేకపోతేనే
`(?<=Y)X`	positive lookbehind	ముందు Y ఉంటేనే X
`(?<!Y)X`	negative lookbehind	ముందు Y లేకపోతేనే X

```
// price కి ముందున్న $ ని consume చేయకుండా number తీయడం
console.log("$100".match(/(?<=\$)\d+/)[0]); // "100"

// "100" తర్వాత "px" ఉంటేనే
console.log("100px 50em".match(/\d+(?=px)/)[0]); // "100"

// password validation: 8+ chars, 1 digit, 1 uppercase
const strong = /^(?=.*\d)(?=.*[A-Z]).{8,}$/;
console.log(strong.test("Passw0rd")); // true
console.log(strong.test("password")); // false (uppercase/digit లేదు)
```

Greedy vs Lazy quantifiers

Default గా quantifiers **greedy** (ఎక్కువ match). `?` జోడిస్తే **lazy** (తక్కువ match).

```
const html = "<b>bold</b> and <i>italic</i>";
console.log(html.match(/<.+>/)[0]);  // greedy: "<b>bold</b> and <i>italic</i>"
console.log(html.match(/<.+?>/)[0]); // lazy:   "<b>"
```

Internals – ఎలా పనిచేస్తుంది

Regex engine **backtracking** వాడుతుంది – pattern match కాకపోతే వెనక్కి వెళ్ళి వేరే combination try చేస్తుంది. Complex nested quantifiers (`(a+)+`) తో ఇది **exponential** అవ్వొచ్చు – **catastrophic backtracking** → app freeze (ReDoS attack, Topic 46).

Gotchas (సాధారణ తప్పులు)

- `g` **flag +** `test/exec` **= stateful** `lastIndex`**:** అదే regex object మీద పిలిస్తే position గుర్తుంచుకుంటుంది → false results loops లో.

```
const re = /a/g;
console.log(re.test("aaa"), re.test("aaa"), re.test("aaa"), re.test("aaa"));
// true true true false (lastIndex 0→1→2→3→reset)
```

- `.` **newline ని match చేయదు** – `s` flag అవసరం.
- **Special chars escape:** `.`, `*`, `+`, `?`, `(`, `)` literal గా కావాలంటే `\` (`\.`).
- **ReDoS:** nested quantifiers user input పైన dangerous.
- Constructor లో `\\d` (double backslash) string escaping వల్ల.

Key Points

- Flags: `g` (all), `i` (case), `m` (multiline), `s` (dotAll), `u`, `y`.
- Groups: capturing `()`, named `(?<n>)`, non-capturing `(?:)`, backreference `\1`.
- Lookahead/behind consume చేయవు; validation కి powerful.
- Greedy default, lazy `?`; backtracking జాగ్రత్త (ReDoS).
- `g` flag తో `lastIndex` stateful.

Interview దృష్టి

- "Greedy vs lazy?" → greedy max, lazy min match.
- "Lookahead అంటే?" → position check, consume చేయకుండా.
- "ReDoS?" → catastrophic backtracking, untrusted input పైన regex జాగ్రత్త.

---

35. Proxy & Reflect (meta-programming, traps, reactivity/validation)

వివరణ

Proxy అనేది ఒక object చుట్టూ ఒక wrapper – original object పైన జరిగే operations (property read, write, delete, function call) ని **intercept** చేసి custom behavior జోడించడానికి. ఇదే **meta-programming** – code యొక్క behavior ని program చేయడం.

```
const proxy = new Proxy(target, handler);
```

- `target` – wrap చేసే original object.
- `handler` – **traps** (interceptor functions) ఉన్న object.

Reflect అనేది ఆ operations యొక్క default behavior ని perform చేసే built-in object – proxy traps లోపల "అసలు పని" చేయడానికి వాడతాం.

Real-life Scenario

> **Proxy = ఒక personal secretary.** మీరు boss (target object) ని కలవాలంటే ముందు secretary (proxy) ద్వారా వెళ్ళాలి. Secretary ప్రతి request ని intercept చేస్తుంది – "ఈ visitor కి appointment ఉందా?" (validation), "ఈ meeting log చెయ్యి" (logging), "boss lunch లో ఉంటే వేరే answer" (default value). Boss కి ఏమీ తెలియదు – secretary అంతా handle చేస్తుంది. `Reflect` = secretary చివరికి నిజంగా boss కి message pass చేసే standard విధానం.

Common traps table

Trap	ఎప్పుడు trigger అవుతుంది
`get(target, prop, receiver)`	property read (`obj.x`)
`set(target, prop, value, receiver)`	property write (`obj.x = 5`)
`has(target, prop)`	`in` operator (`"x" in obj`)
`deleteProperty(target, prop)`	`delete obj.x`
`apply(target, thisArg, args)`	function call (`fn()`)
`construct(target, args, newTarget)`	`new Fn()`
`ownKeys(target)`	`Object.keys` , `for...in`
`defineProperty`	`Object.defineProperty`

Code – logging & default values

```
const user = { name: "Ravi", age: 25 };

const logged = new Proxy(user, {
  get(target, prop, receiver) {
    console.log(`Reading: ${prop}`);
    // లేని property కి default value
    if (!(prop in target)) return `<${String(prop)} లేదు>`;
    return Reflect.get(target, prop, receiver); // default behavior
  },
  set(target, prop, value, receiver) {
    console.log(`Setting: ${prop} = ${value}`);
    return Reflect.set(target, prop, value, receiver);
  }
});

console.log(logged.name);  // "Reading: name" → "Ravi"
console.log(logged.email); // "Reading: email" → "<email లేదు>"
logged.age = 26;           // "Setting: age = 26"
```

Code – validation

```
function createValidatedUser() {
  return new Proxy({}, {
    set(target, prop, value) {
      if (prop === "age") {
        if (typeof value !== "number" || value < 0 || value > 150) {
          throw new TypeError("age 0–150 number అయి ఉండాలి");
        }
      }
      if (prop === "email" && !value.includes("@")) {
        throw new TypeError("Invalid email");
      }
      return Reflect.set(target, prop, value);
    }
  });
}

const u = createValidatedUser();
u.age = 30;           // OK
u.email = "a@b.com";  // OK
try { u.age = -5; } catch (e) { console.log(e.message); } // "age 0–150..."
```

Code – reactivity (Vue 3 core idea)

```
function reactive(obj, onChange) {
  return new Proxy(obj, {
    set(target, prop, value, receiver) {
      const result = Reflect.set(target, prop, value, receiver);
      onChange(prop, value); // మార్పు జరిగింది → re-render trigger
      return result;
    }
  });
}

const state = reactive({ count: 0 }, (prop, val) =>
  console.log(`${prop} మారింది → ${val}, UI update!`));

state.count = 1; // "count మారింది → 1, UI update!"
state.count = 2; // "count మారింది → 2, UI update!"
```

Reflect – ఎందుకు అవసరం?

`Reflect` methods proxy operations కి perfect match (అదే signature). ప్రయోజనాలు:

- **సరైన `this` (receiver):** `Reflect.get(target, prop, receiver)` – inherited getters లో `this` సరిగ్గా bind అవుతుంది. `target[prop]` అయితే receiver miss అవుతుంది.
- **Consistent return values:** `Reflect.set` `true/false` return; direct assignment అలా ఇవ్వదు.
- **Cleaner meta-programming:** `Reflect.has`, `Reflect.deleteProperty`, `Reflect.ownKeys` – operators బదులు functions.

```
// receiver ఎందుకు ముఖ్యం
const parent = { get name() { return this._n; } };
const child = { _n: "child" };
Object.setPrototypeOf(child, parent);
const p = new Proxy(parent, {
  get(t, prop, receiver) { return Reflect.get(t, prop, receiver); }
});
```

Internals

Proxy operations engine level లో intercept అవుతాయి – ఏ property access అయినా ముందు trap ను చెక్ చేస్తుంది. అందుకే proxies కొంచెం slow (direct access కంటే). Trap లేకపోతే operation నేరుగా target కి pass అవుతుంది (transparent).

వాస్తవ వాడకాలు

- **Reactivity systems** (Vue 3, MobX) – state మార్పులు track చేయడం.
- **Validation** – schema enforcement.
- **API wrappers** – `api.users.get()` లాంటి dynamic property chains.
- **Negative array indices, default values, immutability** (read-only objects).
- **Logging/debugging, access control.**

Gotchas (సాధారణ తప్పులు)

- **Performance:** ప్రతి operation trap గుండా → hot paths లో slow. అవసరమైన చోట వాడండి.
- `this` **binding:** proxy లోని methods `this` proxy ను reference చేయాలి; internal methods bypass అవ్వొచ్చు.
- **Trap invariants:** కొన్ని traps rules పాటించాలి (non-configurable property ని lie చేయలేరు) – లేకపోతే `TypeError`.
- `Reflect` **వాడకపోతే** receiver/return value bugs.
- Proxy ను revoke చేయవచ్చు (`Proxy.revocable`) – తర్వాత access చేస్తే error.

Key Points

- Proxy = object operations ని intercept చేసే wrapper (meta-programming).
- Traps: `get` , `set` , `has` , `deleteProperty` , `apply` , `construct` , ...
- Reflect = default behavior perform చేసే helper; సరైన receiver/return కి proxy లో వాడండి.
- Real uses: reactivity (Vue 3), validation, logging, API wrappers.

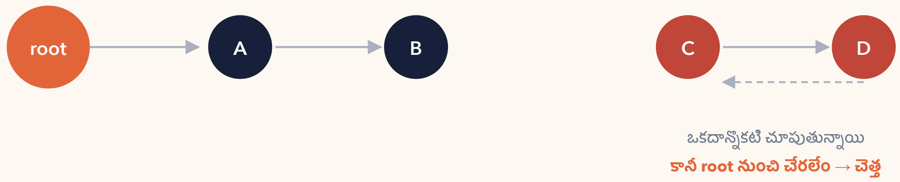
Interview దృష్టి

- "Vue 3 reactivity ఎలా?" → Proxy (get track, set trigger); Vue 2 `Object.defineProperty` .
- "Reflect ఎందుకు?" → సరైన receiver/this, consistent returns, functional meta-ops.
- "Proxy trade-off?" → flexibility vs performance overhead.

36. Memory Management & Garbage Collection (reachability, mark-sweep, leaks, WeakRef/FinalizationRegistry)

GARBAGE COLLECTION · REACHABILITY, REFERENCE COUNT కాదు

GARBAGE COLLECTION — reachability



Mark and sweep

root నుంచి చేరగలిగే అన్నిటినీ mark చేయడం
mark కానివి అన్నీ తీసేయడం
Reference counting కాదు – cycles ని handle చేస్తుంది

Memory leak ఎక్కడ వస్తుంది

Global variables · తీసేయని event listeners
Closures lo పెద్ద objects · setInterval
Detached DOM nodes (JS ఇంకా పట్టుకుంది)

వివరణ

JavaScript **automatic memory management** – మీరు manually memory allocate/free చేయరు (C లో `malloc` / `free` లేదు). **Garbage Collector (GC)** automatically ఇక అవసరం లేని memory ని విడుదల చేస్తుంది.

Memory lifecycle:

1. **Allocate** – object సృష్టించినప్పుడు memory కేటాయించబడుతుంది.
2. **Use** – read/write.
3. **Release** – reachable కాకపోతే GC విడుదల చేస్తుంది.

Stack – primitives, function frames (fixed size, fast). **Heap** – objects, arrays, functions (dynamic size, GC ఇక్కడ పనిచేస్తుంది).

Real-life Scenario

> **Library లో పుస్తకాలు.** ఒక పుస్తకాన్ని ఎవరూ చదవడం లేదు, ఎవరూ reserve చేయలేదు, ఏ catalog లోనూ reference లేదు (unreachable) → librarian (GC) దాన్ని తీసేసి shelf ఖాళీ చేస్తాడు (memory free). కానీ ఒక్క వ్యక్తి అయినా ఆ పుస్తకాన్ని hold చేసి ఉంటే (reference), అది shelf లోనే ఉంటుంది. సమస్య: మీరు పుస్తకం చదవడం మానేసినా bookmark (reference) తీయకపోతే – అది ఎప్పటికీ shelf లో ఉండిపోతుంది → **memory leak**.

Reachability – GC యొక్క ప్రధాన concept

JS GC **reachability** ఆధారంగా పనిచేస్తుంది. **Roots** (global object, current call stack, closures) నుండి reference chain ద్వారా చేరుకోగలిగే objects "reachable" – అవి ఉంటాయి. చేరుకోలేని (unreachable) objects garbage.

```
┌────────── ROOTS ──────────┐
│ globalThis, call stack │
└───────────────────────────┘
           │ reference
           ▼
    ┌────────┐       ┌────────┐
    │ objA   │──────▶│ objB   │    ← reachable (ఉంటాయి)
    └────────┘       └────────┘

    ┌────────┐       ┌────────┐
    │ objC   │──────▶│ objD   │    ← unreachable (garbage!)
    └────────┘       └────────┘
    (ఏ root నుండీ చేరుకోలేం — GC తీసేస్తుంది)
```

Mark-and-Sweep algorithm

Modern JS engines (V8) **mark-and-sweep** వాడతాయి:

1. **Mark:** roots నుండి మొదలుపెట్టి, reachable objects అన్నింటినీ "mark" చేస్తుంది (graph traversal).
2. **Sweep:** mark కాని (unreachable) objects memory ని free చేస్తుంది.

పాత **reference counting** (references count = 0 అయితే free) సమస్య – **circular references** ని handle చేయలేదు. Mark-sweep దీన్ని పరిష్కరిస్తుంది (cycle unreachable అయితే మొత్తం free).

```
// circular reference — reference counting fail అవుతుంది, mark-sweep OK
function cycle() {
  const a = {}, b = {};
  a.ref = b; b.ref = a; // ఒకదానికొకటి reference
} // function అయ్యాక a, b unreachable → mark-sweep free చేస్తుంది
```

V8 generational GC (internals)

V8 objects ని రెండు generations గా విభజిస్తుంది (చాలా objects త్వరగా చనిపోతాయి అనే "generational hypothesis" ఆధారంగా):

- **Young generation (nursery):** కొత్త objects; తరచుగా, వేగంగా GC (Scavenge – copying).
- **Old generation:** GC survive అయిన objects; అరుదుగా, ఖరీదైన GC (mark-sweep-compact).

GC "**stop-the-world**" – GC జరిగేటప్పుడు JS execution కొద్దిసేపు ఆగుతుంది. అందుకే GC ని efficient గా ఉంచడం ముఖ్యం (V8 incremental/concurrent GC వాడుతుంది pauses తగ్గించడానికి).

Memory Leaks – సాధారణ కారణాలు

Leak రకం	ఉదాహరణ
Global variables	accidental globals (`x = 5` without let), GC చేయలేదు
Forgotten timers	`setInterval` clear చేయకపోతే callback + closure alive
Detached DOM nodes	JS లో reference ఉంది కానీ DOM నుండి remove చేసారు

Leak రకం	ఉదాహరణ
Closures	పెద్ద data ని closure hold చేస్తే release అవ్వదు
Event listeners	remove చేయని listeners
Growing caches/Maps	పరిమితి లేని caches

```
// ❌ Leak: forgotten interval
function start() {
  const bigData = new Array(1000000).fill("data");
  setInterval(() => console.log(bigData.length), 1000);
  // interval ఎప్పటికీ clear కాదు → bigData ఎప్పటికీ free కాదు
}

// ❌ Leak: detached DOM
const cache = {};
function addNode() {
  const div = document.createElement("div");
  cache.node = div;          // JS reference
  document.body.appendChild(div);
}
function removeNode() {
  document.body.removeChild(cache.node); // DOM నుండి తీసేసారు
  // కానీ cache.node ఇంకా reference → GC చేయలేదు (leak)
}
```

WeakMap / WeakSet / WeakRef

Weak references – GC ని అడ్డుకోవు. Referenced object మిగతా చోట్ల unreachable అయితే, weak reference ఉన్నా GC దాన్ని collect చేస్తుంది.

```
// WeakMap — keys weakly held; metadata caching కి perfect
const metadata = new WeakMap();
let user = { name: "Ravi" };
metadata.set(user, { lastLogin: Date.now() });
user = null; // user unreachable → WeakMap entry కూడా auto-GC (leak లేదు)
```

WeakRef (ES2021) – object కి weak reference; `deref()` తో access (object ఇంకా ఉంటే వస్తుంది, లేకపోతే `undefined`).

```
let obj = { data: "big" };
const weak = new WeakRef(obj);
console.log(weak.deref()?.data); // "big"
obj = null; // GC తర్వాత
// weak.deref() → undefined కావచ్చు (GC అయితే)
```

FinalizationRegistry – object GC అయ్యాక cleanup callback (unreliable timing – depend అవ్వకండి).

```
const registry = new FinalizationRegistry(name => {
  console.log(`${name} garbage collected అయింది`);
});
let cache = { id: 1 };
registry.register(cache, "cache-object");
cache = null; // ఎప్పుడో GC తర్వాత callback (timing guarantee లేదు)
```

Gotchas (సాధారణ తప్పులు)

- **WeakRef/FinalizationRegistry timing unpredictable** – critical logic కి వాడకండి.
- Closures అనవసర data hold చేస్తాయి – పెద్ద variables closure లో ఉంచవద్దు.
- `console.log(obj)` DevTools లో reference hold చేయవచ్చు (debugging లో false leaks).
- Global caches పరిమితి లేకపోతే memory పెరుగుతూ పోతుంది → LRU eviction వాడండి.

Key Points

- Automatic GC; reachability ఆధారంగా (roots నుండి చేరుకోగలిగితే alive).
- Mark-and-sweep – circular references handle చేస్తుంది (reference counting చేయలేదు).
- V8 generational (young/old), stop-the-world (కొద్దిసేపు).
- Leaks: timers, listeners, detached DOM, closures, unbounded caches.
- WeakMap/WeakSet/WeakRef – GC ని అడ్డుకోవు; caching/metadata కి.

Interview దృష్టి

- "JS GC ఎలా పనిచేస్తుంది?" → mark-and-sweep, reachability from roots.
- "Reference counting ఎందుకు కాదు?" → circular references leak అవుతాయి.
- "Memory leak ఎలా debug?" → Chrome DevTools Memory tab, heap snapshots, detached nodes.
- "WeakMap ఎప్పుడు?" → object metadata, keys auto-cleanup కావాలంటే.

37. Execution Context & Call Stack deep (creation/execution phase, variable environment)

EXECUTION CONTEXT · CREATION మరియు EXECUTION

EXECUTION CONTEXT – ప్రతి function call కి ఒకటి

Creation phase

Variable environment తయారు
var → undefined · let/const → TDZ
Function declarations పూర్తిగా
this నిర్ణయం

Execution phase

Code line by line నడుస్తుంది
Assignments జరుగుతాయి
కొత్త calls → కొత్త context

CALL STACK

inner()
outer()
global()

పైనున్నది ముగిస్తే pop – కిందిది కొనసాగుతుంది

ఇది ఎందుకు తెలియాలి

Hoisting, TDZ, this binding, closures – ఇవన్నీ ఇక్కడి నుంచే పుడతాయి.
Stack trace చదవడం కూడా ఇదే – పైనున్నది తాజా call.

వివరణ

Execution Context (EC) అంటే JavaScript code run అయ్యే "environment" – ఒక code piece ఏ variables, functions, `this` value తో execute అవుతుందో నిర్వచించే box. ప్రతి function call కి కొత్త EC సృష్టించబడుతుంది.

3 రకాల ECs:

- **Global Execution Context (GEC):** default, program ప్రారంభంలో ఒక్కటే. `globalThis` / `window` create అవుతుంది.
- **Function Execution Context (FEC):** ప్రతి function call కి ఒకటి.
- **Eval Execution Context:** `eval()` (అరుదు, avoid చేయండి).

Real-life Scenario

> **Execution Context = ఒక వంటగదిలో ఒక recipe ప్రిపరేషన్ station.** ప్రతి డిష్ (function call) కి ఒక కొత్త station setup అవుతుంది – ఆ station కి కావలసిన పదార్థాలు (local variables), instructions (code), ఏ chef పని చేస్తున్నాడు (`this`) అన్నీ arrange అవుతాయి. మీరు ఒక recipe మధ్యలో మరో sub-recipe మొదలుపెడితే (nested call), కొత్త station పైన పెడతారు (stack). Sub-recipe అయ్యాక ఆ station తీసేసి, పాత station కి తిరిగి వస్తారు.

Two phases – Creation & Execution

ప్రతి EC రెండు phases లో ప్రాసెస్ అవుతుంది:

1. Creation Phase (memory allocation):

- **Variable Environment** setup అవుతుంది.
- `var` declarations → `undefined` తో hoist.
- `let` / `const` declarations → hoist అవుతాయి కానీ **TDZ** (Temporal Dead Zone) లో (uninitialized).
- Function declarations → పూర్తిగా hoist (body తో సహా).
- `this` binding నిర్ధారించబడుతుంది.
- Outer environment reference (scope chain) set అవుతుంది.

2. Execution Phase:

- Code line-by-line run అవుతుంది.
- Variables కి actual values assign అవుతాయి.
- Function calls కొత్త ECs create చేస్తాయి.

```
console.log(x);       // undefined (var hoisted, creation phase)
// console.log(y);    // ❌ ReferenceError (let TDZ లో)
console.log(greet()); // "hi" (function fully hoisted)

var x = 10;
let y = 20;
function greet() { return "hi"; }

// Creation phase: x=undefined, y=<TDZ>, greet=<function>
// Execution phase: x=10, y=20
```

Execution Context – components

ప్రతి EC లో:

Component	వివరణ
Variable Environment	`var`, function declarations (hoisted)
Lexical Environment	`let` / `const` + outer scope reference (scope chain)
`this` **binding**	ఎలా call చేసారో దాన్నిబట్టి (Topic 48)

(ES2015+ లో VariableEnvironment మరియు LexicalEnvironment కొంచెం విడిగా – var vs let/const scoping కోసం.)

Call Stack – ECs ని manage చేయడం

Call Stack అనేది active execution contexts ని LIFO order లో track చేసే stack. Function call → EC push; function return → EC pop.

```
function first() {
  console.log("first start");
  second();
  console.log("first end");
```

```
}
function second() {
  console.log("second start");
  third();
  console.log("second end");
}
function third() { console.log("third"); }

first();

// Output:
// first start, second start, third, second end, first end
```

Call stack evolution:

```
Step 1: [ global ]
Step 2: [ global, first ]            ← first() call
Step 3: [ global, first, second ]    ← second() call
Step 4: [ global, first, second, third ] ← third() call
Step 5: [ global, first, second ]    ← third returns (pop)
Step 6: [ global, first ]            ← second returns (pop)
Step 7: [ global ]                   ← first returns (pop)
```

Scope chain – variable resolution

ఒక variable కావాలంటే, current EC లో వెతుకుతుంది; లేకపోతే **outer (lexical) environment** లో; అలా global దాకా – ఇదే **scope chain**. Lexical scope = code ఎక్కడ **రాసారో** దాన్నిబట్టి (ఎక్కడ call చేసారో కాదు).

```
const global = "G";
function outer() {
  const o = "O";
  function inner() {
    const i = "I";
    console.log(i, o, global); // I O G – scope chain పైకి వెతుకుతుంది
  }
  inner();
}
outer();
```

Stack Overflow

Call stack కి పరిమిత size ఉంది. చాలా deep/infinite recursion → **"Maximum call stack size exceeded"** (RangeError).

```
function recurse() { return recurse(); } // base case లేదు
// recurse(); // ❌ RangeError: Maximum call stack size exceeded
```

Internals – engine implementation

- **Global EC** program start లో create; `globalThis`, `this` (global) setup.

- ప్రతి function call కి engine కొత్త stack frame allocate చేస్తుంది (local variables, return address, `this`).
- Function return అయ్యాక frame pop అవుతుంది, local variables (closure తో reference కాకపోతే) GC కి eligible.
- Async callbacks కొత్త stack frames గా (stack empty అయ్యాక) event loop ద్వారా push అవుతాయి – అందుకే async stack traces అస్పష్టం.

Gotchas (సాధారణ తప్పులు)

- **Hoisting confusion:** `var` `undefined`, `let/const` TDZ – creation phase వల్ల.
- `this` **in creation phase** determine అవుతుంది కాబట్టి, call చేసిన విధానం మీద ఆధారపడుతుంది.
- **Stack overflow** deep recursion; tail-call optimization JS లో reliable కాదు.
- Each call new EC → recursion లో ప్రతి level కి separate variable copies.

Key Points

- EC = code run అయ్యే environment (variables, `this`, scope chain).
- 2 phases: Creation (hoisting, memory) → Execution (values, run).
- Call stack – LIFO, ECs push/pop; overflow = deep recursion.
- Scope chain lexical (code position); global దాకా వెతుకుతుంది.

Interview దృష్టి

- "Hoisting ఎందుకు జరుగుతుంది?" → creation phase లో declarations memory కి allocate.
- "let vs var hoisting?" → var undefined, let/const TDZ.
- "Call stack అంటే?" → active ECs LIFO tracking; overflow deep recursion.

---

38. The DOM (selection, traversal, manipulation, nodes, attributes/classes)

వివరణ

DOM (Document Object Model) అనేది browser ఒక HTML document ని memory లో ఒక **tree structure** గా represent చేసే model. ప్రతి HTML element, text, comment ఒక **node**. JavaScript ఈ tree ని read/modify చేయగలదు – dynamic web pages కి ఇదే foundation.

ముఖ్యం: DOM అనేది JavaScript లో భాగం **కాదు** – ఇది browser అందించే **Web API** (`document` object ద్వారా).

Real-life Scenario

> **DOM = ఒక family tree diagram.** HTML document = మొత్తం కుటుంబం. `<html>` = మూలపురుషుడు (root), `<body>` దాని బిడ్డ, ప్రతి `<div>`, `<p>`, text దాని descendants. మీరు ఏ ఒక్క person (element) ని అయినా వెతకవచ్చు (selection), వారి parents/siblings/children కి వెళ్ళవచ్చు (traversal), కొత్త members జోడించవచ్చు లేదా తీసేయవచ్చు (manipulation). Diagram లో మార్పు = నిజ కుటుంబంలో మార్పు (screen లో reflect).

Node types

Node type	ఉదాహరణ
Element node	`<div>`, `<p>`, `<a>`
Text node	element లోని text
Comment node	`<!-- ... -->`
Document node	`document` (root)

కీలక తేడా: `children` (elements మాత్రమే) vs `childNodes` (text/comments తో సహా అన్ని nodes).

Selection — elements ని పట్టుకోవడం

```
// ID తో (ఒక్కటే)
const el = document.getElementById("app");

// CSS selector — మొదటి match
const btn = document.querySelector(".submit-btn");
const first = document.querySelector("ul li:first-child");

// CSS selector — అన్నీ (static NodeList)
const items = document.querySelectorAll("li.active");

// tag / class / name (live HTMLCollection)
const divs = document.getElementsByTagName("div");
const reds = document.getElementsByClassName("red");
```

Method	Returns	Live?
`getElementById`	ఒక Element / null	–
`querySelector`	మొదటి Element / null	–
`querySelectorAll`	NodeList	**static** (snapshot)
`getElementsByTagName/ClassName`	HTMLCollection	**live** (auto-update)

Traversal – tree లో కదలడం

```
const el = document.querySelector(".item");

// Parents
el.parentElement;        // parent element
el.closest(".container"); // ancestor matching selector (self కూడా)

// Children
el.children;             // child elements (HTMLCollection)
el.firstElementChild;    // మొదటి child element
el.lastElementChild;

// Siblings
el.nextElementSibling;   // తర్వాతి sibling element
el.previousElementSibling;

// (Node versions: parentNode, childNodes, firstChild — text nodes కూడా)
```

Manipulation – content, attributes, structure

```
const el = document.querySelector("#box");

// Content
el.textContent = "Plain text";     // text (safe, XSS-proof)
el.innerHTML = "<b>Bold</b>";      // HTML parse (⚠️ XSS risk with user data)
el.innerText = "Visible text";     // rendered text (styles-aware, slow)

// Attributes
el.setAttribute("data-id", "42");
el.getAttribute("data-id");        // "42"
el.hasAttribute("disabled");
el.removeAttribute("data-id");
el.dataset.id;                     // "42" (data-* కి shortcut)

// Properties vs attributes
el.id = "newId";                   // property (direct)
el.value = "text";                 // form value (property, attribute కాదు)
```

Creating & inserting nodes

```
// కొత్త element
const li = document.createElement("li");
li.textContent = "కొత్త item";
li.className = "list-item";

const ul = document.querySelector("ul");

// Modern insertion methods (recommended)
ul.append(li);          // చివర్లో (multiple + text కూడా)
ul.prepend(li);         // మొదట్లో
el.before(newEl);       // element ముందు
el.after(newEl);        // element తర్వాత
el.replaceWith(newEl);  // element ని replace
li.remove();            // తీసేయడం
```

```
// Legacy methods
ul.appendChild(li);
ul.insertBefore(li, ul.firstChild);
ul.removeChild(li);

// insertAdjacentHTML — position తో HTML
el.insertAdjacentHTML("beforeend", "<span>hi</span>");
// positions: beforebegin, afterbegin, beforeend, afterend
```

Classes & styles

```
const el = document.querySelector(".card");

// classList (recommended)
el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("open");        // ఉంటే తీయి, లేకపోతే జోడించు
el.classList.contains("active");    // true/false
el.classList.replace("old", "new");

// inline styles
el.style.color = "red";
el.style.backgroundColor = "blue";  // camelCase (CSS: background-color)
el.style.cssText = "color:red; font-size:16px";
```

Performance – DocumentFragment & batching

DOM operations **ఖరీదైనవి** (reflow/repaint trigger చేస్తాయి, Topic 45). చాలా elements జోడించాలంటే **DocumentFragment** వాడండి – memory లో batch చేసి, ఒకేసారి DOM కి push.

```
// ❌ Slow — ప్రతి append reflow trigger చేస్తుంది
for (let i = 0; i < 1000; i++) {
  ul.appendChild(createItem(i)); // 1000 reflows!
}

// ✅ Fast — ఒక్కటే reflow
const fragment = document.createDocumentFragment();
for (let i = 0; i < 1000; i++) {
  fragment.appendChild(createItem(i)); // memory లో, DOM కి కాదు
}
ul.appendChild(fragment); // ఒకేసారి insert → 1 reflow
```

Internals – DOM vs JS engine

DOM operations JavaScript engine (V8) బయట, browser rendering engine (Blink) లో జరుగుతాయి. ప్రతి JS↔DOM interaction ఒక "bridge" crossing – అందుకే DOM access slow, minimize చేయాలి. Frameworks (React) **Virtual DOM** వాడతాయి – memory లో diff చేసి, minimal real DOM updates.

Gotchas (సాధారణ తప్పులు)

- `innerHTML` **+ user data = XSS** (Topic 46). Untrusted data కి `textContent`.
- **Live vs static collections:** `getElementsBy*` live – loop లో modify చేస్తే infinite/skip bugs.

- `querySelectorAll` **static** – DOM మారినా NodeList update అవ్వదు.
- **null checks:** `querySelector` దొరకకపోతే `null` → `.textContent` పైన TypeError.
- **Reflow thrashing:** loop లో read+write alternate చేస్తే multiple reflows (batch చేయండి).

Key Points

- DOM = HTML యొక్క tree representation (Web API, JS కాదు).
- Selection: `getElementById`, `querySelector(All)`; live (`getElementsBy*`) vs static.
- Traversal: `parentElement`, `children`, `nextElementSibling`, `closest`.
- Manipulation: `textContent` (safe) vs `innerHTML` (XSS), `classList`, `createElement`.
- Performance: DocumentFragment, batching (DOM ఖరీదైనది).

Interview దృష్టి

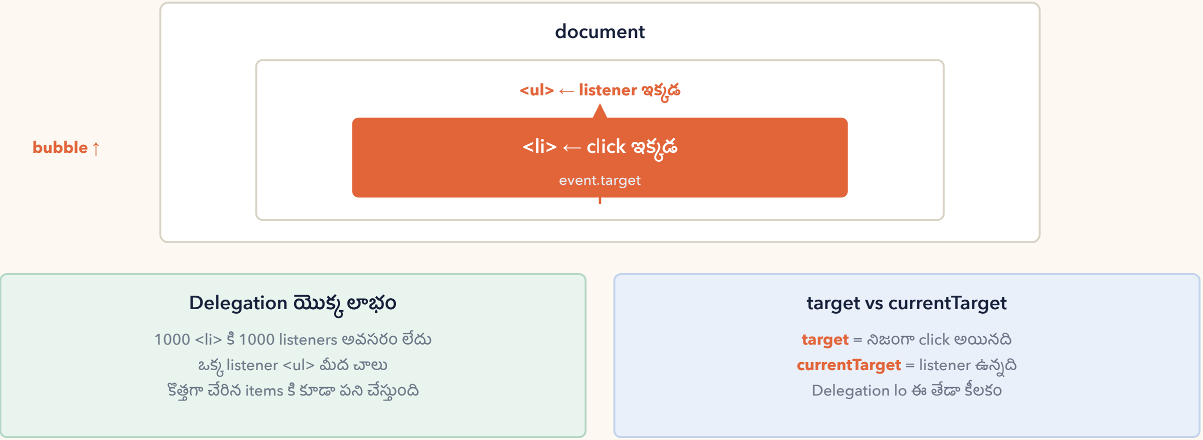
- "textContent vs innerHTML vs innerText?" → text safe / HTML parse (XSS) / rendered (slow).
- "Live vs static NodeList?" → getElementsBy* live, querySelectorAll static.
- "DOM ఎందుకు slow?" → JS↔rendering engine bridge, reflow/repaint.

---

39. Events (capturing/bubbling, delegation, addEventListener, custom events, preventDefault)

EVENT BUBBLING & DELEGATION

EVENT BUBBLING & DELEGATION



వివరణ

Event అనేది page లో జరిగే ఏదైనా చర్య – click, keypress, mouse move, form submit, page load, network response. JavaScript ఈ events కి **listeners** (handler functions) జోడించి react చేస్తుంది. ఇదే interactive web pages యొక్క గుండె.

Real-life Scenario

> **నీటిలో రాయి వేయడం.** మీరు ఒక button (ఒక specific point) పైన click చేసారు. ఆ చర్య అక్కడితో ఆగదు – **ripples** (event) చుట్టూ వ్యాపిస్తాయి. Bubbling = ripple లోపలి element నుండి బయటి parents వైపు (button → div → body → document). ప్రతి parent "ఇక్కడ click జరిగింది" అని తెలుసుకోగలదు. మీరు ఒకే listener ని pond అంచున (parent) పెడితే, ఏ రాయి ఎక్కడ పడినా తెలుస్తుంది (event delegation).

addEventListener – basics

```
const btn = document.querySelector("#submit");

function handleClick(event) {
  console.log("Clicked!", event.target); // event object automatic
}

btn.addEventListener("click", handleClick);
btn.removeEventListener("click", handleClick); // అదే function reference కావాలి!

// options
btn.addEventListener("click", handleClick, {
  once: true,      // ఒక్కసారే run అయి auto-remove
  capture: true,   // capturing phase లో
  passive: true    // preventDefault పిలవను (scroll performance)
});
```

`onclick` vs `addEventListener`: `onclick` ఒక్క handler మాత్రమే (overwrite అవుతుంది); `addEventListener` multiple handlers allow చేస్తుంది. Modern code లో `addEventListener` preferred.

Event propagation – 3 phases

ఒక event 3 phases లో ప్రయాణిస్తుంది:

```
        document
           │  1. CAPTURING phase (పైనుండి కిందకి) ↓
        <body>
           │
        <div>
           │
      ┌────▼────┐
      │ <button>│   2. TARGET phase (అసలు element)
      └────┬────┘
           │  3. BUBBLING phase (కిందనుండి పైకి) ↑
        <div>
           │
        <body>
           │
        document
```

- **Capturing (phase 1):** document → target (పైనుండి కిందకి). `addEventListener(..., {capture: true})` తో మాత్రమే.
- **Target (phase 2):** అసలు element పైన.
- **Bubbling (phase 3):** target → document (కిందనుండి పైకి). **Default** behavior.

```
// మూడు nested elements — ఏ order లో fire అవుతాయో చూద్దాం
outer.addEventListener("click", () => console.log("outer bubble"));
inner.addEventListener("click", () => console.log("inner bubble"));
outer.addEventListener("click", () => console.log("outer capture"), true);
inner.addEventListener("click", () => console.log("inner capture"), true);

// inner పైన click చేస్తే Output:
// outer capture  ← capturing (పైనుండి)
// inner capture
// inner bubble   ← bubbling (కిందనుండి)
// outer bubble
```

Event object – ముఖ్య properties/methods

Property/Method	వివరణ
`event.target`	అసలు event trigger చేసిన element
`event.currentTarget`	listener attach చేసిన element (`this`)
`event.type`	"click", "keydown"
`event.preventDefault()`	default browser action ఆపడం
`event.stopPropagation()`	propagation ఆపడం (bubbling/capturing)
`event.stopImmediatePropagation()`	అదే element మీద మిగతా listeners కూడా ఆపడం

target vs currentTarget – event delegation కి కీలకం:

```
ul.addEventListener("click", function(e) {
  console.log(e.target);        // click అయిన అసలు <li>
  console.log(e.currentTarget); // <ul> (listener ఉన్నది)
});
```

Event Delegation – powerful pattern

ప్రతి child కి separate listener బదులు, **ఒక్క listener ని parent కి** జోడించి, bubbling ద్వారా events ని catch చేయడం. `event.target` తో ఏ child అనేది తెలుస్తుంది.

```
// ❌ చెడ్డది — ప్రతి item కి listener (1000 items = 1000 listeners)
document.querySelectorAll("li").forEach(li =>
  li.addEventListener("click", handler));

// ✅ మంచిది — parent కి ఒక్క listener (delegation)
document.querySelector("ul").addEventListener("click", (e) => {
```

```
  const li = e.target.closest("li");
  if (li) console.log("Clicked:", li.textContent);
});
```

ప్రయోజనాలు:

- Memory efficient (ఒక్క listener).
- **Dynamic elements** work – తర్వాత జోడించిన items కూడా automatic handle (re-attach అవసరం లేదు).
- Cleaner code.

preventDefault vs stopPropagation

రెండూ వేరు – గందరగోళం పడకండి:

- `preventDefault()` – browser యొక్క **default action** ఆపుతుంది (form submit, link navigation, checkbox toggle). Propagation ఆపదు.
- `stopPropagation()` – event **bubbling/capturing** ఆపుతుంది. Default action ఆపదు.

```
form.addEventListener("submit", (e) => {
  e.preventDefault(); // page reload ఆపి, AJAX తో submit చేయడానికి
  submitViaAjax();
});

link.addEventListener("click", (e) => {
  e.preventDefault(); // navigation ఆపడం (SPA routing)
});
```

Custom Events

మీ స్వంత events create + dispatch చేయవచ్చు – components మధ్య communication కి.

```
// custom event create (data తో)
const event = new CustomEvent("userLoggedIn", {
  detail: { userId: 42, name: "Ravi" },
  bubbles: true
});

// listen
document.addEventListener("userLoggedIn", (e) => {
  console.log("User:", e.detail.name); // "Ravi"
});

// dispatch (trigger)
document.dispatchEvent(event);
```

Internals

Browser event handling event loop లో macrotasks (Topic 27). User interaction → event queue → stack empty అయ్యాక handler run. Handlers synchronous – ఒక handler ఎక్కువసేపు run అయితే UI freeze. అందుకే heavy work debounce/throttle (Topic 45) చేయాలి.

Gotchas (సాధారణ తప్పులు)

- `removeEventListener` **కి same function reference కావాలి** – anonymous functions remove చేయలేరు.

```
el.addEventListener("click", () => {}); // ❌ remove చేయలేరు
```

- `stopPropagation` **అతిగా వాడటం** – వేరే delegation listeners break అవుతాయి.
- `this` **in arrow vs regular handler** – arrow లో `this` lexical (element కాదు); regular లో `this` = currentTarget.
- **Passive listeners లో** `preventDefault` పనిచేయదు (warning).
- **Memory leaks** – SPA లో listeners remove చేయకపోతే.

Key Points

- `addEventListener` (multiple, options); `onclick` (single).
- Propagation: capturing (↓) → target → bubbling (↑, default).
- `target` (అసలు element) vs `currentTarget` (listener element).
- **Delegation** – parent కి ఒక్క listener, dynamic elements, memory efficient.
- `preventDefault` (default action) ≠ `stopPropagation` (bubbling).

Interview దృష్టి

- "Event delegation అంటే + ప్రయోజనం?" → parent listener + bubbling; memory, dynamic elements.
- "target vs currentTarget?" → అసలు element vs listener element.
- "Bubbling vs capturing?" → కిందనుండి పైకి (default) vs పైనుండి కిందకి.
- "preventDefault vs stopPropagation?" → default action vs propagation.

---

40. Browser Web APIs (fetch, Storage/cookies, timers, observers, Web Workers)

వివరణ

Web APIs అనేవి browser అందించే built-in functionalities – JavaScript language లో భాగం కాదు, కానీ `window` object ద్వారా JS కి అందుబాటులో ఉంటాయి. Network requests, storage, timers, DOM observation, background threads – ఇవన్నీ Web APIs.

Real-life Scenario

> **JavaScript = ఒక కొత్త ఉద్యోగి, Browser = office building.** Employee (JS) కి తనవైన skills ఉన్నాయి (language features). కానీ phone (fetch), filing cabinet (localStorage), alarm clock (timers), security camera (observers), assistant (Web Workers) – ఇవన్నీ office (browser) అందించే tools (Web APIs). Employee వాటిని వాడుకుంటాడు కానీ అవి అతని s skills కావు – office సదుపాయాలు.

fetch – modern networking

`fetch` promise-based HTTP requests. పాత `XMLHttpRequest` కి replacement.

```
// GET
async function getUser(id) {
  const res = await fetch(`/api/users/${id}`);
  if (!res.ok) throw new Error(`HTTP ${res.status}`); // fetch 4xx/5xx కి reject
అవ్వదు!
  return res.json();
}

// POST
async function createUser(data) {
  const res = await fetch("/api/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data)
  });
  return res.json();
}

// AbortController – request cancel చేయడం
const controller = new AbortController();
fetch("/api/slow", { signal: controller.signal })
  .catch(e => { if (e.name === "AbortError") console.log("Cancelled"); });
setTimeout(() => controller.abort(), 5000); // 5s తర్వాత cancel
```

కీలక gotcha: `fetch` network failure కే reject అవుతుంది. HTTP 404/500 కి **reject అవ్వదు** – `res.ok` / `res.status` manually check చేయాలి.

Storage – comparison table

Feature	localStorage	sessionStorage	Cookies	IndexedDB
Capacity	~5-10MB	~5-10MB	~4KB	~GBs
Expiry	ఎప్పటికీ (manual clear)	tab close అయితే	set చేయవచ్చు	manual
Server కి పంపబడతాయా?	❌	❌	✅ (ప్రతి request)	❌
Access	sync	sync	sync	async

Feature	localStorage	sessionStorage	Cookies	IndexedDB
Type	string మాత్రమే	string మాత్రమే	string	objects

```
// localStorage — persistent (strings మాత్రమే)
localStorage.setItem("user", JSON.stringify({ id: 1, name: "Ravi" }));
const user = JSON.parse(localStorage.getItem("user"));
localStorage.removeItem("user");
localStorage.clear();

// sessionStorage — tab close అయ్యేదాకా (అదే API)
sessionStorage.setItem("temp", "value");

// Cookies — server కి పంపబడతాయి
document.cookie = "token=abc123; max-age=3600; path=/; Secure; SameSite=Strict";
```

Timers

```
// setTimeout — ఒకసారి, delay తర్వాత
const id = setTimeout(() => console.log("2s తర్వాత"), 2000);
clearTimeout(id); // cancel

// setInterval — పదేపదే
const iid = setInterval(() => console.log("ప్రతి 1s"), 1000);
clearInterval(iid); // cancel — లేకపోతే memory leak!

// requestAnimationFrame — smooth animations (60fps, browser sync)
function animate() {
  moveBox();
  requestAnimationFrame(animate); // repaint కి ముందు
}
requestAnimationFrame(animate);
```

`setTimeout` / `setInterval` macrotasks (Topic 27). Animations కి `requestAnimationFrame` — browser refresh rate తో sync, tab background లో pause (battery friendly).

Observers

Modern, efficient way to watch changes (polling కంటే మెరుగు):

```
// IntersectionObserver — element viewport లోకి వచ్చిందా? (lazy loading, infinite scroll)
const io = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.src = entry.target.dataset.src; // image lazy load
      io.unobserve(entry.target);
    }
  });
});
document.querySelectorAll("img[data-src]").forEach(img => io.observe(img));

// MutationObserver — DOM మార్పులు watch
const mo = new MutationObserver(mutations =>
```

```
  mutations.forEach(m => console.log("DOM మారింది:", m.type)));
mo.observe(document.body, { childList: true, subtree: true });

// ResizeObserver — element size మార్పులు
const ro = new ResizeObserver(entries =>
  entries.forEach(e => console.log("Size:", e.contentRect.width)));
ro.observe(document.querySelector(".box"));
```

గమనిక: `MutationObserver` callbacks microtasks (Topic 27).

Web Workers – background threads

JavaScript single-threaded, కానీ **Web Workers** heavy computations ని separate thread లో run చేయగలవు – main thread (UI) block అవ్వకుండా. Workers DOM access చేయలేవు; `postMessage` ద్వారా communicate.

```
// main.js
const worker = new Worker("worker.js");
worker.postMessage({ numbers: [1, 2, 3, 1000000] });
worker.onmessage = (e) => console.log("Result:", e.data); // UI freeze అవ్వదు
worker.terminate(); // ఆపడం

// worker.js
self.onmessage = (e) => {
  const sum = e.data.numbers.reduce((a, b) => a + b, 0); // heavy work
  self.postMessage(sum); // result తిరిగి పంపడం
};
```

Web Workers ఎప్పుడు: heavy computation (image processing, large data parsing, encryption), UI responsive గా ఉంచడానికి.

ఇతర ముఖ్య APIs

- **`navigator`** – browser info (`navigator.onLine` , `navigator.geolocation` , `navigator.clipboard`).
- **`history`** – SPA routing (`history.pushState` , `popstate` event).
- **`URL` / `URLSearchParams`** – URL parsing.
- **WebSocket** – real-time bidirectional communication.
- **Notification, Geolocation, Clipboard** – permission-based.

Gotchas (సాధారణ తప్పులు)

- **`fetch` HTTP errors కి reject అవ్వదు** – `res.ok` check మర్చిపోవద్దు.
- **`setInterval` clear చేయకపోతే** memory leak + duplicate runs (Topic 36).
- **localStorage synchronous** – పెద్ద data blocking; JSON.parse errors handle చేయండి.
- **localStorage strings మాత్రమే** – objects కి `JSON.stringify/parse` .
- **Web Workers DOM access చేయలేవు** – computation కి మాత్రమే.
- **Cookies ప్రతి request తో పంపబడతాయి** – పెద్ద cookies performance hit.

Key Points

- Web APIs browser అందిస్తుంది (JS language కాదు); `window` ద్వారా.
- `fetch` promise-based; HTTP errors కి reject అవ్వదు (`res.ok` check).
- Storage: localStorage (persistent), sessionStorage (tab), cookies (server, 4KB), IndexedDB (large, async).
- Observers (Intersection/Mutation/Resize) polling కంటే efficient.
- Web Workers – background threads, DOM access లేదు, postMessage.

Interview దృష్టి

- "localStorage vs cookies vs sessionStorage?" → capacity, expiry, server transmission.
- "fetch 404 కి catch trigger అవుతుందా?" → కాదు, `res.ok` check అవసరం.
- "Heavy computation UI freeze అవ్వకుండా?" → Web Worker.
- "Lazy loading ఎలా?" → IntersectionObserver.

---

41. Node.js essentials (runtime, modules, process, EventEmitter, streams, fs, non-blocking I/O)

వివరణ

Node.js అనేది Chrome యొక్క **V8 engine** ని browser బయట server-side లో run చేసే **runtime**. దీనివల్ల JavaScript తో servers, CLI tools, backend systems build చేయవచ్చు. Node non-blocking, event-driven architecture వాడుతుంది – అందుకే I/O-heavy applications కి efficient.

Real-life Scenario

> **Node.js = ఒక అత్యంత సమర్థుడైన waiter ఉన్న restaurant.** ఒకే waiter (single thread) కానీ, order తీసుకున్నాక kitchen (I/O – file/DB/network) దగ్గర నిలబడడు – వెంటనే వేరే tables కి వెళ్తాడు. Kitchen ready అయ్యాక notify చేస్తుంది (callback/event), అప్పుడు serve చేస్తాడు (non-blocking I/O). PHP/Java లాంటి traditional servers ప్రతి customer కి ఒక waiter (thread-per-request) – Node ఒక్క smart waiter తో వందల customers ని handle చేస్తాడు.

Runtime – browser vs Node తేడాలు

అంశం	Browser	Node.js
Global object	`window`	`global` / `globalThis`
DOM	✅	❌ (server, UI లేదు)
Modules	ESM	CommonJS (default) + ESM
APIs	fetch, localStorage, DOM	fs, http, os, crypto, path

అంశం	Browser	Node.js
`this` (top)	`window`	`module.exports` (`{}`)
File access	❌ (sandboxed)	✅ (full fs)

Modules – CommonJS

Node traditionally **CommonJS** వాడుతుంది (`require` / `module.exports`). ESM కూడా support (`.mjs` లేదా `"type": "module"`).

```
// math.js
function add(a, b) { return a + b; }
module.exports = { add };            // export
// లేదా: exports.add = add;

// app.js
const { add } = require("./math");   // import
const fs = require("fs");            // built-in module
console.log(add(2, 3));              // 5
```

process – runtime info & control

`process` global object – running program గురించి info + control.

```
process.argv;           // CLI arguments array
process.env.NODE_ENV;   // environment variables
process.cwd();          // current working directory
process.pid;            // process ID
process.platform;       // "darwin", "win32", "linux"
process.exit(0);        // program ఆపడం (0 = success)

// events
process.on("exit", () => console.log("Cleanup"));
process.on("uncaughtException", err => console.error(err));

// process.nextTick – microtask కంటే ముందు! (Node-specific)
process.nextTick(() => console.log("nextTick – highest priority"));
```

Node event loop: browser కంటే కొంచెం complex – phases (timers → pending → poll → check → close). `process.nextTick` అన్నిటికంటే ముందు, తర్వాత promise microtasks, తర్వాత phases.

EventEmitter – event-driven pattern

Node యొక్క core pattern – objects events emit చేసి, listeners react చేయడం. చాలా Node APIs (streams, http) దీని పైనే build.

```
const EventEmitter = require("events");

class OrderSystem extends EventEmitter {
  placeOrder(item) {
    console.log(`Order: ${item}`);
```

```
    this.emit("orderPlaced", item); // event fire
  }
}

const orders = new OrderSystem();
orders.on("orderPlaced", item => console.log(`Email పంపాం: ${item}`));
orders.on("orderPlaced", item => console.log(`Inventory update: ${item}`));
orders.once("orderPlaced", () => console.log("మొదటి order మాత్రమే"));

orders.placeOrder("Pizza");
// Order: Pizza → Email పంపాం: Pizza → Inventory update: Pizza → మొదటి order మాత్రమే
```

Streams – పెద్ద data ని chunks లో handle

Streams పెద్ద data ని ఒకేసారి memory లోకి కాకుండా, **chunks** గా process చేస్తాయి. Memory efficient – GB files కూడా handle.

Stream type	వివరణ
Readable	data చదవడం (file read, HTTP request)
Writable	data రాయడం (file write, HTTP response)
Duplex	రెండూ (sockets)
Transform	read + modify + write (gzip)

```
const fs = require("fs");

// ❌ మొత్తం file memory లోకి (పెద్ద files కి crash)
fs.readFile("huge.txt", (err, data) => { /* అంతా RAM లో */ });

// ✅ Stream – chunks గా (memory efficient)
const readStream = fs.createReadStream("huge.txt", "utf8");
readStream.on("data", chunk => console.log("Chunk:", chunk.length));
readStream.on("end", () => console.log("పూర్తయింది"));

// pipe – read నుండి write కి (backpressure automatic)
fs.createReadStream("input.txt")
  .pipe(fs.createWriteStream("output.txt")); // copy, memory efficient
```

fs – file system

```
const fs = require("fs");
const fsp = require("fs/promises"); // promise-based

// Async (non-blocking, recommended)
const data = await fsp.readFile("file.txt", "utf8");
await fsp.writeFile("out.txt", "content");
await fsp.appendFile("log.txt", "line\n");

// Callback style
fs.readFile("file.txt", "utf8", (err, data) => { });
```

```
// Sync (blocking — startup/scripts లో మాత్రమే, servers లో వద్దు!)
const content = fs.readFileSync("config.json", "utf8");
```

Non-blocking I/O — Node యొక్క గుండె

Node **libuv** library వాడుతుంది — I/O operations (file, network, DB) ని OS/thread pool కి delegate చేసి, JS thread ని free గా ఉంచుతుంది. I/O పూర్తయ్యాక callback event loop ద్వారా run అవుతుంది.

```
console.log("1");
fs.readFile("file.txt", () => console.log("3: file చదివాం"));
console.log("2");
// Output: 1, 2, 3 — file read block చేయలేదు, ఈలోపు "2" print
```

Blocking (`readFileSync`) ఒక్క request ని process చేస్తుండగా మిగతా అన్ని requests wait — servers లో వద్దు.

Internals — Node architecture

```
┌──────────────────────────────────┐
│        Your JS Code              │
├──────────────────────────────────┤
│  Node APIs (fs, http, crypto...) │
├───────────────┬──────────────────┤
│  V8 (JS engine) │  libuv (event loop, │
│               │  thread pool, async │
│               │  I/O — C++)         │
└───────────────┴──────────────────┘
```

libuv thread pool (default 4 threads) file I/O, DNS, crypto లాంటి కొన్ని ops కి — కానీ మీ JS single-threaded.

Gotchas (సాధారణ తప్పులు)

- `readFileSync` **servers లో** — ఒక్క blocking call అన్ని requests ఆపుతుంది.
- `require` **cache అవుతుంది** — module ఒక్కసారే load, singleton.
- **EventEmitter memory leak** — 10+ listeners అయితే warning; remove చేయండి.
- **Streams error handling** — `.on("error")` మర్చిపోతే crash.
- `process.exit()` **async ops ని కట్ చేస్తుంది** — pending writes lost.

Key Points

- Node = V8 based server-side runtime; non-blocking, event-driven.
- CommonJS (`require` / `module.exports`) default; ESM కూడా.
- `process` — env, argv, nextTick (highest priority).
- EventEmitter — event-driven pattern (streams, http దీని పైనే).
- Streams — chunks లో పెద్ద data, memory efficient, `pipe` .
- Non-blocking I/O — libuv, JS thread free.

Interview దృష్టి

- "Node single-threaded అయినా thousands requests ఎలా?" → non-blocking I/O, event loop, libuv.
- "Streams ఎందుకు?" → memory efficient, పెద్ద data chunks లో.
- "process.nextTick vs setImmediate vs Promise?" → nextTick > promises (microtask) > setImmediate (check phase).
- "readFile vs readFileSync?" → async non-blocking vs sync blocking (servers లో async).

---

42. ES version-by-version features (ES6 → ES2024 highlights)

వివరణ

ECMAScript (ES) అనేది JavaScript యొక్క official specification. **TC39** committee ప్రతి సంవత్సరం కొత్త features add చేస్తుంది (2015 నుండి yearly releases). ES6 (ES2015) పెద్ద మలుపు – modern JS ఇక్కడే మొదలైంది. ఒక senior engineer గా ఏ feature ఏ version లో వచ్చిందో, ఎందుకు వచ్చిందో తెలియాలి.

Real-life Scenario

> **JavaScript = ఒక city, ES versions = yearly infrastructure upgrades.** ES6 (2015) = మెట్రో rail వచ్చిన సంవత్సరం (arrow functions, classes, promises – everything changed). తర్వాత ప్రతి సంవత్సరం చిన్న చిన్న upgrades – flyovers (async/await), smart signals (optional chaining), కొత్త parks (array methods). పాత features ఎప్పుడూ పనిచేస్తాయి (backward compatible) – city పాత రోడ్లను తీసేయదు, కొత్తవి జోడిస్తుంది.

ES6 / ES2015 – పెద్ద విప్లవం

అత్యధిక features ఇక్కడే. Modern JS = ES6+.

```
// let / const (block scope)
let x = 1; const PI = 3.14;

// Arrow functions
const add = (a, b) => a + b;

// Template literals
const msg = `Hello ${name}, ${1 + 1}`;

// Destructuring
const { name, age } = user;
const [first, ...rest] = arr;

// Default + rest + spread
function fn(a = 10, ...args) {}
const merged = { ...obj1, ...obj2 };
```

```
// Classes
class Dog { constructor(n) { this.name = n; } bark() {} }

// Promises, Modules (import/export), Generators, Symbol
// Map, Set, WeakMap, WeakSet
// for...of, computed property names
```

Version-by-version highlights table

Version	ముఖ్య features
ES2015 (ES6)	let/const, arrow fns, classes, template literals, destructuring, spread/rest, promises, modules, generators, Map/Set, Symbol, default params
ES2016 (ES7)	`Array.includes()`, exponentiation `**`
ES2017 (ES8)	**async/await**, `Object.entries/values`, string padding, `Object.getOwnPropertyDescriptors`
ES2018 (ES9)	rest/spread for objects `{...o}`, async iterators (`for await`), Promise.finally, regex named groups/lookbehind
ES2019 (ES10)	`Array.flat/flatMap`, `Object.fromEntries`, `String.trimStart/End`, optional catch binding `catch {}`
ES2020 (ES11)	**optional chaining** `?.`, **nullish coalescing** `??`, `BigInt`, `Promise.allSettled`, `globalThis`, dynamic import, `matchAll`
ES2021 (ES12)	`String.replaceAll`, `Promise.any`, logical assignment `??= &&= ||=`, numeric separators `1_000_000`, WeakRef
ES2022 (ES13)	**top-level await**, class fields (`#private`), static blocks, `Array.at()`, `Object.hasOwn()`, error `cause`
ES2023 (ES14)	`Array.findLast/findLastIndex`, immutable array methods (`toSorted`, `toReversed`, `with`), hashbang
ES2024 (ES15)	`Object.groupBy`, `Promise.withResolvers`, `Array.fromAsync`, `Atomics.waitAsync`

అత్యంత ముఖ్యమైన modern features – code

Optional chaining `?.` (ES2020): nested property safely access.

```
const city = user?.address?.city;          // undefined if any null/undefined
const first = arr?.[0];                     // array
const result = obj.method?.();              // method call if exists
// పాత విధానం: user && user.address && user.address.city
```

Nullish coalescing `??` (ES2020): `null` / `undefined` కి మాత్రమే fallback (`||` కంటే safer).

```
const port = config.port ?? 3000;   // 0 valid! (|| అయితే 0 → 3000 wrong)
const name = input ?? "Guest";      // "" valid (|| అయితే "" → "Guest" wrong)
// || 0, "", false, NaN అన్నింటికీ fallback; ?? null/undefined కి మాత్రమే
```

Logical assignment (ES2021):

```
a ??= 5;   // a = a ?? 5  (a null/undefined అయితేనే)
b ||= 10;  // b = b || 10 (b falsy అయితే)
c &&= 20;  // c = c && 20 (c truthy అయితే)
```

Array grouping & immutable methods (ES2023/2024):

```
// toSorted — original ని మార్చకుండా (sort mutates!)
const sorted = [3, 1, 2].toSorted();   // [1,2,3], original అలాగే
const reversed = arr.toReversed();
const updated = arr.with(0, 99);       // index 0 ని 99 తో, కొత్త array

// Object.groupBy (ES2024)
const grouped = Object.groupBy([1,2,3,4], n => n % 2 ? "odd" : "even");
// { odd: [1,3], even: [2,4] }

// findLast (ES2023)
[1,2,3,4].findLast(n => n < 3); // 2
```

Class fields & private (ES2022):

```
class Counter {
  count = 0;                // public field
  #secret = 42;             // private field (# — నిజమైన privacy)
  static instances = 0;     // static field
  static { console.log("class initialized"); } // static block
  #increment() { this.count++; } // private method
}
```

Internals – TC39 process

కొత్త feature 4 stages గుండా వెళుతుంది:

- **Stage 0** (Strawperson) → **Stage 1** (Proposal) → **Stage 2** (Draft) → **Stage 3** (Candidate) → **Stage 4** (Finished – spec లో చేరుతుంది).

Stage 3+ features Babel/browsers experiment గా implement చేస్తాయి. అందుకే production లో Stage 3 features వాడేముందు browser support/polyfills చెక్ చేయండి.

Gotchas (సాధారణ తప్పులు)

- `??` **vs** `||`**:** `0`, `""`, `false` valid values అయితే `??` వాడండి.
- **Browser support:** పాత browsers కి Babel transpilation అవసరం (Topic 43).
- **Immutable methods (toSorted etc.) కొత్తవి** – పాత environments లో లేవు, polyfill/check.

- `#private` **నిజమైన privacy** – `_private` convention కాదు (bypass చేయవచ్చు); `#` engine-enforced.
- **Optional chaining short-circuits** – `a?.b.c` లో `a` null అయితే `.c` కూడా skip.

Key Points

- ES6 (2015) = modern JS foundation (arrow, class, promise, modules, destructuring).
- ES2017 = async/await; ES2020 = `?.`, `??`, BigInt, allSettled.
- ES2022 = top-level await, `#private` fields; ES2023 = immutable array methods.
- Yearly releases; TC39 4-stage process; backward compatible.

Interview దృష్టి

- "?? vs ||?" → ?? null/undefined కి మాత్రమే, || అన్ని falsy కి.
- "ES6 ముఖ్య features?" → let/const, arrow, class, promise, destructuring, modules.
- "#private vs _private?" → # engine-enforced నిజమైన privacy.
- "async/await ఏ version?" → ES2017.

---

43. Tooling ecosystem (npm/package.json, bundlers, Babel, TypeScript intro)

వివరణ

Modern JavaScript development కి కేవలం language కాదు – ఒక పెద్ద **tooling ecosystem** అవసరం: package manager (npm), bundlers (Vite/Webpack), transpiler (Babel), type checker (TypeScript), linters (ESLint), formatters (Prettier). ఇవి productivity, code quality, browser compatibility ని manage చేస్తాయి.

Real-life Scenario

> **ఒక ఇల్లు కట్టడం.** Language = raw materials (ఇటుకలు, cement). Tooling = construction equipment: npm = supplier (materials తెప్పించడం), bundler = అన్నీ కలిపి ఒక building గా assemble చేసే crane, Babel = పాత-style plans ని కొత్త workers కి అర్థమయ్యేలా translate చేసేవాడు, TypeScript = blueprint checker (కట్టేముందే errors పట్టుకునేవాడు). Tools లేకుండా కూడా కట్టవచ్చు కానీ చాలా slow, error-prone.

npm & package.json

npm (Node Package Manager) – world's largest software registry. Dependencies install/manage చేస్తుంది.

```
{
  "name": "my-app",
  "version": "1.0.0",
```

```
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "vitest"
  },
  "dependencies": {          // production లో అవసరం
    "react": "^18.2.0"
  },
  "devDependencies": {       // development లో మాత్రమే
    "vite": "^5.0.0",
    "eslint": "^8.0.0"
  }
}
```

Semantic Versioning (semver): `MAJOR.MINOR.PATCH` (e.g., `2.4.1`).

Symbol	అర్థం	ఉదాహరణ (`1.2.3`)
`^` (caret)	minor + patch updates	`1.x.x` (2.0.0 కాదు)
`~` (tilde)	patch updates మాత్రమే	`1.2.x`
exact	సరిగ్గా అదే	`1.2.3`

- **MAJOR** – breaking changes.
- **MINOR** – కొత్త features (backward compatible).
- **PATCH** – bug fixes.

```
npm install react          # dependency జోడించడం
npm install -D vitest      # devDependency
npm install                # package.json నుండి అన్నీ install
npm run build              # script run
npm ci                     # package-lock.json నుండి exact install (CI)
```

`package-lock.json` – exact versions lock చేస్తుంది (reproducible builds). commit చేయాలి.
`node_modules` – actual code (gitignore).

Bundlers

Bundler అనేక JS files, dependencies, assets (CSS, images) ని కొన్ని optimized files గా combine చేస్తుంది – browser కి efficient గా load అయ్యేలా.

Bundler	ప్రత్యేకత
Vite	అత్యంత fast (ESM + esbuild), modern default
Webpack	most configurable, mature, పెద్ద ecosystem
Rollup	libraries కి ideal (tree-shaking)

Bundler	ప్రత్యేకత
esbuild	Go-based, cực fast
Parcel	zero-config

Bundler పనులు:

- **Bundling** – files combine (fewer HTTP requests).
- **Tree-shaking** – unused code తీసేయడం (Topic 32).
- **Code splitting** – lazy loading కి chunks.
- **Minification** – whitespace/names తగ్గించడం.
- **Transpilation** – Babel integration.
- **Asset handling** – CSS, images, fonts.

Babel – transpiler

Babel modern JS (ES2020+) ని పాత browsers అర్థం చేసుకునే older JS (ES5) గా **transpile** చేస్తుంది. కొత్త syntax వాడుతూ, పాత browsers ని support చేయడానికి.

```
// మీరు రాసేది (ES2020+)
const greet = (name) => `Hi ${name}`;
const city = user?.address?.city ?? "Unknown";

// Babel transpiles to (ES5 — పాత browsers కి)
var greet = function(name) { return "Hi " + name; };
var city = user != null && user.address != null
  ? user.address.city : "Unknown";
```

Babel **plugins/presets** (`@babel/preset-env`) – target browsers బట్టి ఏ features transpile చేయాలో నిర్ధారిస్తుంది. **Polyfills** (core-js) కొత్త APIs (Promise, Array.includes) పాత browsers కి జోడిస్తాయి.

Transpiling vs Polyfilling: transpiling = syntax మార్చడం (arrow → function); polyfilling = missing APIs జోడించడం (Promise implementation).

TypeScript – typed JavaScript

TypeScript (TS) అనేది JavaScript కి **static types** జోడించే superset. Compile time లో type errors పట్టుకుంటుంది (runtime కి ముందు). Browser లో run అవ్వదు – JS కి compile అవుతుంది.

```
// types జోడించడం
interface User {
  id: number;
  name: string;
  email?: string;      // optional
}

function greet(user: User): string {
```

```
  return `Hi ${user.name}`;
}

greet({ id: 1, name: "Ravi" });        // OK
greet({ id: "1", name: "Ravi" });      // ❌ compile error: id number కావాలి
greet({ name: "Ravi" });               // ❌ id missing

// generics
function first<T>(arr: T[]): T { return arr[0]; }
const n: number = first([1, 2, 3]);    // T = number inferred
```

ప్రయోజనాలు: early error detection, better IDE support (autocomplete, refactoring), self-documenting code, పెద్ద teams/codebases కి safer. **Trade-off:** learning curve, build step, కొంచెం verbose.

ఇతర ముఖ్య tools

- **ESLint** – code quality/bug patterns catch (linter).
- **Prettier** – code formatting (consistent style).
- **Vitest/Jest** – testing (Topic 44).
- **pnpm/yarn** – alternative package managers (pnpm disk-efficient).
- **tsx/ts-node** – TS ని నేరుగా run చేయడం.

Gotchas (సాధారణ తప్పులు)

- `node_modules` **commit చేయవద్దు** – gitignore; `package-lock.json` commit చేయండి.
- `^` **unexpected updates** – minor version bugs; lock file వాడండి, `npm ci`.
- **dependencies vs devDependencies గందరగోళం** – build tools devDependencies లో.
- **TS types runtime లో లేవు** – compile అయ్యాక erased; runtime validation కావాలంటే Zod లాంటివి.
- **Bundle size bloat** – పెద్ద dependencies; tree-shaking, bundle analyzer చెక్ చేయండి.

Key Points

- npm – packages; package.json (scripts, deps); semver (`^` minor, `~` patch).
- package-lock.json commit (reproducible); node_modules gitignore.
- Bundlers (Vite/Webpack) – bundle, tree-shake, split, minify.
- Babel – transpile (modern → old syntax); polyfills (missing APIs).
- TypeScript – static types, compile-time errors, JS కి compile.

Interview దృష్టి

- "^1.2.3 అంటే?" → 1.x.x (minor+patch, major కాదు).
- "Babel vs polyfill?" → syntax transpile vs missing API జోడించడం.
- "dependencies vs devDependencies?" → production vs dev-only.
- "TypeScript ప్రయోజనం?" → compile-time type safety, IDE support.
- "Bundler ఎందుకు?" → combine, tree-shake, minify, optimize.

44. Testing (unit testing, Jest/Vitest, mocking, TDD)

వివరణ

Testing అంటే code సరిగ్గా పనిచేస్తుందని automated గా verify చేయడం. Manual testing (browser లో refresh) slow, error-prone. Automated tests – confidence తో code మార్చడానికి, refactor చేయడానికి, bugs ముందే పట్టుకోడానికి.

Testing levels (**Testing Pyramid**):

- **Unit tests** – ఒక్క function/module isolated గా (ఎక్కువ, fast, cheap).
- **Integration tests** – multiple units కలిసి పనిచేస్తున్నాయా (మధ్యస్థం).
- **E2E tests** – మొత్తం app real browser లో (తక్కువ, slow, expensive).

Real-life Scenario

> **కారు తయారీ.** ప్రతి భాగాన్ని (engine, brake, tyre) విడిగా test చేయడం = unit tests (fast, specific). భాగాలు కలిపి assemble చేసి test = integration. మొత్తం కారుని road పైన నడిపి test = E2E (real conditions, కానీ slow, ఖరీదు). Manufacturer ఎక్కువ unit tests, కొన్ని road tests చేస్తాడు – ఇదే testing pyramid. TDD = ముందు "ఈ brake ఇలా పనిచేయాలి" అని test రాసి, తర్వాత brake తయారు చేయడం.

Anatomy of a test – AAA pattern

```
test("description", () => {
  // Arrange – setup
  const calc = new Calculator();
  // Act – పని చేయడం
  const result = calc.add(2, 3);
  // Assert – verify
  expect(result).toBe(5);
});
```

Jest / Vitest – basics

Jest (Facebook) మరియు **Vitest** (Vite-native, faster) – most popular test frameworks. APIs దాదాపు ఒకటే.

```
import { describe, test, expect, beforeEach } from "vitest";

function add(a, b) { return a + b; }

describe("Calculator", () => {          // test group
  test("2 + 3 = 5", () => {
    expect(add(2, 3)).toBe(5);
  });
```

```
  test("negative numbers", () => {
    expect(add(-1, -1)).toBe(-2);
  });

  it("works as alias for test", () => { // it = test alias
    expect(add(0, 0)).toBe(0);
  });
});
```

Matchers – common assertions

Matcher	వాడకం
`toBe(x)`	primitive equality (`===`)
`toEqual(x)`	deep equality (objects/arrays)
`toBeTruthy()` / `toBeFalsy()`	truthiness
`toBeNull()` / `toBeUndefined()`	null / undefined
`toContain(x)`	array/string contains
`toThrow()`	function error throw చేస్తుందా
`toHaveBeenCalled()`	mock function called అయిందా
`toHaveBeenCalledWith(args)`	specific args తో called
`resolves` / `rejects`	promise assertions

```
expect({ a: 1 }).toBe({ a: 1 });      // ❌ fail (వేరే references)
expect({ a: 1 }).toEqual({ a: 1 });   // ✅ pass (deep equal)
expect(() => JSON.parse("{")).toThrow(); // ✅
expect([1, 2, 3]).toContain(2);       // ✅
```

Async testing

```
// Promise — return / async-await
test("fetches user", async () => {
  const user = await fetchUser(1);
  expect(user.name).toBe("Ravi");
});

// resolves / rejects
test("resolves", async () => {
  await expect(fetchUser(1)).resolves.toHaveProperty("name");
  await expect(fetchUser(-1)).rejects.toThrow("Not found");
});
```

Mocking – dependencies ని fake చేయడం

Mocking అంటే real dependencies (API calls, DB, timers) ని fake versions తో replace చేయడం – tests fast, deterministic, isolated గా ఉంచడానికి.

```
import { vi, test, expect } from "vitest";

// mock function (spy)
const mockFn = vi.fn();
mockFn("hello");
expect(mockFn).toHaveBeenCalledWith("hello");
expect(mockFn).toHaveBeenCalledTimes(1);

// return value నిర్ధారించడం
const mockApi = vi.fn().mockResolvedValue({ id: 1, name: "Ravi" });

// module mocking — నిజమైన fetch ని fake చేయడం
vi.mock("./api", () => ({
  fetchUser: vi.fn().mockResolvedValue({ name: "Test User" })
}));

// timer mocking — setTimeout wait అవసరం లేదు
vi.useFakeTimers();
const cb = vi.fn();
setTimeout(cb, 5000);
vi.advanceTimersByTime(5000); // 5s "fast-forward"
expect(cb).toHaveBeenCalled();
```

ఎందుకు mock: real API tests slow + unreliable (network); DB tests state pollute చేస్తాయి; timers tests slow చేస్తాయి. Mock వాటిని deterministic + fast చేస్తుంది.

Setup / Teardown hooks

```
beforeEach(() => { /* ప్రతి test కి ముందు — fresh setup */ });
afterEach(() => { /* ప్రతి test తర్వాత — cleanup */ });
beforeAll(() => { /* అన్నిటికీ ఒక్కసారి ముందు (DB connect) */ });
afterAll(() => { /* అన్నిటికీ ఒక్కసారి తర్వాత (DB close) */ });
```

TDD – Test-Driven Development

TDD – code కి ముందు test రాయడం. **Red-Green-Refactor** cycle:

```
1. RED    — failing test రాయి (ఇంకా code లేదు)
2. GREEN  — test pass అయ్యేంత minimal code రాయి
3. REFACTOR — code clean చేయి (tests pass గా ఉంచుతూ)
   → repeat
```

```
// 1. RED — test మొదట
test("isPalindrome", () => {
  expect(isPalindrome("racecar")).toBe(true);
  expect(isPalindrome("hello")).toBe(false);
});
```

```
// (isPalindrome ఇంకా లేదు → fail)

// 2. GREEN — pass అయ్యేలా
function isPalindrome(s) {
  return s === s.split("").reverse().join("");
}

// 3. REFACTOR — improve (tests ఇంకా pass)
```

TDD ప్రయోజనాలు: requirements ముందే స్పష్టం, over-engineering తగ్గుతుంది, high coverage, confident refactoring, better design (testable code).

Code coverage

Tests ఎంత % code cover చేస్తున్నాయో measure. కానీ **100% coverage = bug-free కాదు** – coverage quality కాదు, quantity. Meaningful assertions ముఖ్యం.

```
vitest --coverage  # coverage report
```

Gotchas (సాధారణ తప్పులు)

- **`toBe` vs `toEqual`:** objects/arrays కి `toEqual` (deep); primitives కి `toBe`.
- **Async test await మర్చిపోవడం** – test promise కంటే ముందు pass అయినట్లు (false positive).
- **Mocks reset చేయకపోవడం** – tests ఒకదానిపై ఒకటి ఆధారపడతాయి (`vi.clearAllMocks` in beforeEach).
- **Testing implementation, not behavior** – internal details test చేస్తే refactor అయినప్పుడు break.
- **100% coverage obsession** – meaningless tests కంటే meaningful assertions.
- **Flaky tests** – timing/order dependent; deterministic గా ఉంచండి.

Key Points

- Testing pyramid: unit (ఎక్కువ) → integration → E2E (తక్కువ).
- AAA pattern: Arrange, Act, Assert.
- `toBe` (primitives), `toEqual` (deep); async → await + resolves/rejects.
- Mocking – dependencies fake (API, DB, timers) → fast, deterministic.
- TDD – Red-Green-Refactor (test మొదట).

Interview దృష్టి

- "Unit vs integration vs E2E?" → isolation level, speed, cost.
- "Mocking ఎందుకు?" → external dependencies isolate, fast/deterministic tests.
- "TDD cycle?" → Red (fail) → Green (pass) → Refactor.
- "toBe vs toEqual?" → reference vs deep equality.
- "100% coverage = bug-free?" → కాదు, coverage quantity, quality కాదు.

45. Performance (debounce/throttle, memoization, lazy loading, reflows, Big-O)

వివరణ

Performance optimization అంటే code ని faster, memory-efficient, responsive గా చేయడం. కానీ **premature optimization** చెడ్డది – ముందు measure చేయండి (profile), bottleneck కనుక్కోండి, తర్వాత optimize. "Make it work, make it right, make it fast" – ఈ order లో.

Real-life Scenario

> **Highway traffic management.** Debounce = "అందరూ office నుండి బయటకి వచ్చేదాకా ఆగి, ఆఖరి వ్యక్తి వచ్చాక ఒకేసారి gate close చేయడం" (last event). Throttle = "ప్రతి 5 నిమిషాలకి ఒక batch cars మాత్రమే వదలడం" (rate limit). Memoization = "ఒకసారి calculate చేసిన toll amount ని board పైన రాసి, మళ్ళీ అడిగితే recalculate చేయకుండా చూపడం" (cache). Lazy loading = "అవసరమైన exit వచ్చినప్పుడే ఆ road ని open చేయడం."

Debounce vs Throttle – చాలా ముఖ్యం

రెండూ rapid-fire events (scroll, resize, keypress) ని control చేస్తాయి కానీ వేరుగా:

అంశం	Debounce	Throttle
ఎప్పుడు run	events **ఆగిపోయాక** (last event + delay)	క్రమం తప్పకుండా (ప్రతి N ms కి ఒకసారి)
ఉదాహరణ	search box (typing ఆగాక API call)	scroll position tracking, infinite scroll
analogy	lift door (అందరూ ఎక్కేదాకా ఆగి close)	metro (ప్రతి 5 min కి ఒకటి)

```
// DEBOUNCE — చివరి call తర్వాత delay
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);                        // ప్రతి call timer reset
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
const search = debounce((q) => console.log("API:", q), 500);
// user "hello" type చేస్తే → typing ఆగిన 500ms తర్వాత ఒక్కసారే API call

// THROTTLE — ప్రతి interval కి ఒకసారి
function throttle(fn, limit) {
  let inThrottle = false;
  return function (...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => (inThrottle = false), limit);
    }
  };
```

```
}
const onScroll = throttle(() => console.log("scroll pos"), 200);
// scroll ఎంత fast అయినా → ప్రతి 200ms కి ఒకసారే
```

Memoization – caching results

Memoization అంటే ఒకే inputs కి function result ని cache చేసి, మళ్ళీ అదే inputs వస్తే recompute చేయకుండా cached result ఇవ్వడం. Expensive/pure functions కి perfect.

```
function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);  // cache hit
    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}

const slowSquare = (n) => { /* heavy */ return n * n; };
const fastSquare = memoize(slowSquare);
fastSquare(5); // compute → cache
fastSquare(5); // cache నుండి (recompute కాదు)

// fibonacci — memoization తో O(2^n) → O(n)
const fib = memoize(n => n < 2 ? n : fib(n - 1) + fib(n - 2));
```

React లో `useMemo`, `useCallback`, `React.memo` ఇదే idea.

Big-O notation – algorithmic complexity

Code ఎంత "scale" అవుతుందో measure – input పెరిగితే time/space ఎలా పెరుగుతుంది.

Big-O	పేరు	ఉదాహరణ
O(1)	constant	array index, Map/Set lookup
O(log n)	logarithmic	binary search
O(n)	linear	loop, `array.find`
O(n log n)	linearithmic	efficient sort (`Array.sort`)
$O(n^2)$	quadratic	nested loops (bubble sort)
$O(2^n)$	exponential	naive recursive fibonacci

```
// O(n²) — ❌ nested loop (duplicates కనుక్కోవడం)
function hasDupSlow(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++)
      if (arr[i] === arr[j]) return true;
```

```
    return false;
  }

  // O(n) — ✅ Set వాడి (space O(n) trade-off)
  function hasDupFast(arr) {
    const seen = new Set();
    for (const x of arr) {
      if (seen.has(x)) return true;   // O(1) lookup
      seen.add(x);
    }
    return false;
  }
```

Data structure choice ముఖ్యం: `array.includes` O(n), కానీ `Set.has` / `Map.has` O(1). పెద్ద data lookups కి Set/Map వాడండి.

Reflow & Repaint — rendering performance

Browser rendering:

- **Reflow (layout):** element size/position మారితే → browser layout recalculate (ఖరీదైనది).
- **Repaint:** visual (color, visibility) మారితే → redraw (తక్కువ ఖరీదు).

Reflow trigger: DOM add/remove, size/position మార్పు, `offsetHeight` లాంటివి read చేయడం.

```
// ❌ Layout thrashing — read-write alternate → multiple reflows
for (let i = 0; i < items.length; i++) {
  items[i].style.width = items[i].offsetWidth + 10 + "px"; // read + write loop
}

// ✅ Batch reads, then writes
const widths = items.map(el => el.offsetWidth);  // అన్ని reads మొదట
items.forEach((el, i) => el.style.width = widths[i] + 10 + "px"); // writes

// ✅ DocumentFragment (Topic 38) — ఒకేసారి DOM update
```

Lazy loading

అవసరమైనప్పుడే resources load చేయడం — initial load fast.

```
// images — native lazy loading
// <img src="pic.jpg" loading="lazy">

// code splitting — dynamic import (Topic 32)
const module = await import("./heavy-feature.js"); // click అయ్యాక

// IntersectionObserver — viewport లోకి వచ్చినప్పుడు (Topic 40)
```

ఇతర techniques

- **Virtualization** — పెద్ద lists లో visible items మాత్రమే render (react-window).
- **Web Workers** — heavy computation off main thread (Topic 40).

- `requestAnimationFrame` – smooth animations.
- **Minimize DOM access** – cache references, batch updates.
- **Efficient loops** – `for` fastest; unnecessary work avoid.

Gotchas (సాధారణ తప్పులు)

- **Premature optimization** – measure మొదట (Chrome DevTools Performance/Profiler).
- **Debounce vs throttle గందరగోళం** – search=debounce, scroll=throttle.
- **Memoization memory leak** – unbounded cache; పెద్ద data కి LRU/WeakMap.
- **Layout thrashing** – read/write interleaving; batch చేయండి.
- **JSON.stringify memoization key** – order-dependent, functions handle చేయదు.
- **Over-memoizing** – cheap functions కి overhead ఎక్కువ.

Key Points

- Debounce (last event తర్వాత) vs throttle (rate limit) – search vs scroll.
- Memoization – pure/expensive functions results cache.
- Big-O – O(1) Map/Set lookup > O(n) array > O(n^2) nested loops.
- Reflow (layout, ఖరీదు) vs repaint (visual); batch DOM ops.
- Lazy loading, code splitting, Web Workers, virtualization.
- Measure మొదట; premature optimization avoid.

Interview దృష్టి

- "Debounce vs throttle + ఉదాహరణ?" → search box vs scroll tracking.
- "array.includes vs Set.has complexity?" → O(n) vs O(1).
- "Reflow ని ఎలా minimize?" → batch reads/writes, DocumentFragment, class toggle.
- "Memoization ఎప్పుడు?" → pure, expensive, repeated inputs.

---

46. Security (XSS, CSRF, prototype pollution, CSP)

వివరణ

Web applications constant attacks కి గురవుతాయి. ఒక senior engineer గా common vulnerabilities, వాటి defenses తెలియాలి. Security "afterthought" కాదు – design లో భాగం. ప్రధాన నియమం: **"ఎప్పుడూ user input ని నమ్మవద్దు"** (never trust user input).

Real-life Scenario

> **మీ ఇల్లు security.** XSS = దొంగ మీ ఇంటికి పంపిన ఒక "gift package" (malicious input) లో దాచిన device – మీరు తెరిస్తే అది activate అవుతుంది (script runs). CSRF = ఎవరో మీ signature ని forge చేసి, మీ bank కి "ఈ transfer చెయ్యి" అని letter పంపడం (మీ authenticated session ని misuse). Prototype pollution = ఇంటి master key template ని modify చేసి, అన్ని తాళాలనూ ప్రభావితం చేయడం. CSP = "ఈ vendors నుండి మాత్రమే deliveries accept చేయి" అనే security policy.

XSS (Cross-Site Scripting)

XSS – attacker malicious JavaScript ని website లోకి inject చేసి, ఇతర users browsers లో run చేయడం. Cookies steal, session hijack, data theft కి.

మూడు రకాలు:

- **Stored XSS** – malicious script DB లో save (comment, profile) → ఇతరులు చూసినప్పుడు run.
- **Reflected XSS** – URL/input లో script → response లో reflect.
- **DOM-based XSS** – client-side JS unsafe గా DOM update.

```
// ❌ VULNERABLE — user input నేరుగా innerHTML కి
const comment = getUserInput(); // "<img src=x onerror='steal(document.cookie)'>"
div.innerHTML = comment;        // script runs! 😱

// ✅ SAFE — textContent (HTML parse అవ్వదు)
div.textContent = comment;      // literal text గా చూపిస్తుంది

// ✅ SAFE — sanitize (HTML అవసరమైతే)
import DOMPurify from "dompurify";
div.innerHTML = DOMPurify.sanitize(comment); // dangerous parts తీసేస్తుంది
```

XSS defenses:

- User data కి `textContent` (not `innerHTML`).
- HTML అవసరమైతే **sanitize** (DOMPurify).
- **Output encoding** (context-aware: HTML, JS, URL).
- **CSP** headers.
- `HttpOnly` cookies (JS access చేయలేదు).
- Frameworks (React auto-escapes `{userInput}`, కానీ `dangerouslySetInnerHTML` జాగ్రత్త).

CSRF (Cross-Site Request Forgery)

CSRF – attacker user యొక్క authenticated session ని misuse చేసి, వారి తరపున unwanted actions (money transfer, password change) చేయడం. User logged in ఉన్నప్పుడు malicious site నుండి request పంపడం.

```html
<!-- attacker site లో - user logged in అయితే auto-submit -->
<form action="https://bank.com/transfer" method="POST">
  <input name="to" value="attacker" />
  <input name="amount" value="10000" />
</form>
<script>document.forms[0].submit();</script>
<!-- browser bank.com cookies auto-attach → transfer జరుగుతుంది -->
```

CSRF defenses:

- **CSRF tokens** – server unique token ఇస్తుంది, ప్రతి request లో verify (attacker కి తెలియదు).
- **SameSite cookies** (`SameSite=Strict/Lax`) – cross-site requests కి cookies పంపబడవు.
- **Verify Origin/Referer** headers.
- State-changing actions కి POST (GET కాదు).

Prototype Pollution

Prototype pollution – attacker `Object.prototype` ని modify చేసి, అన్ని objects ని ప్రభావితం చేయడం. `__proto__` ద్వారా (unsafe merge/clone లో).

```javascript
// ❌ VULNERABLE – unsafe deep merge
function merge(target, source) {
  for (const key in source) {
    if (typeof source[key] === "object")
      merge(target[key] = target[key] || {}, source[key]);
    else target[key] = source[key];
  }
}
// attacker payload
const malicious = JSON.parse('{"__proto__": {"isAdmin": true}}');
merge({}, malicious);
console.log({}.isAdmin); // true! 😱 అన్ని objects కి isAdmin వచ్చింది

// ✅ SAFE – __proto__ skip, Object.create(null), Map వాడండి
function safeMerge(target, source) {
  for (const key in source) {
    if (key === "__proto__" || key === "constructor") continue; // block
    // ...
  }
}
```

Defenses: `__proto__` / `constructor` / `prototype` keys block, `Object.create(null)` (prototype లేని objects), `Map` వాడండి, `Object.freeze(Object.prototype)` , validated libraries (lodash safe versions).

CSP (Content Security Policy)

CSP – HTTP header ద్వారా browser కి "ఏ sources నుండి scripts/styles/images load చేయాలో" చెప్పడం. XSS కి powerful defense – inline scripts, unknown sources block.

```
Content-Security-Policy: default-src 'self';
  script-src 'self' https://trusted-cdn.com;
  style-src 'self' 'unsafe-inline';
  img-src 'self' data:;
```

దీనితో: inline `<script>` block, తెలియని domains నుండి scripts block. XSS inject అయినా, CSP execution ఆపుతుంది.

ఇతర security concerns

Threat	Defense
SQL Injection	parameterized queries (backend)
Sensitive data leak	HTTPS, secrets .env లో (client code లో కాదు)
Dependency vulnerabilities	`npm audit`, updated packages
Clickjacking	`X-Frame-Options: DENY`
`eval()` misuse	eval avoid (arbitrary code execution)
ReDoS	regex జాగ్రత్త, untrusted input (Topic 34)
Insecure secrets in JWT	proper signing, expiry

Gotchas (సాధారణ తప్పులు)

- **`innerHTML` with user data** – #1 XSS source.
- **API keys/secrets client code లో** – bundle లో visible; backend లో ఉంచండి.
- **`eval()`, `new Function()` user input** – arbitrary code execution.
- **CORS misconfiguration** – `Access-Control-Allow-Origin: *` sensitive APIs కి dangerous.
- **localStorage లో tokens** – XSS కి vulnerable; `HttpOnly` cookies safer.
- **Trusting client-side validation మాత్రమే** – server-side validation తప్పనిసరి.

Key Points

- ఎప్పుడూ user input నమ్మవద్దు; validate + sanitize.
- **XSS** – script injection; `textContent`, sanitize (DOMPurify), CSP, HttpOnly.
- **CSRF** – session misuse; CSRF tokens, SameSite cookies.
- **Prototype pollution** – `__proto__` block, `Object.create(null)`, Map.
- **CSP** – trusted sources మాత్రమే; inline scripts block.
- Secrets backend లో; server-side validation తప్పనిసరి.

Interview దృష్టి

- "XSS అంటే + prevention?" → script injection; textContent, sanitize, CSP.
- "CSRF vs XSS?" → CSRF session misuse (external), XSS script injection.

- "innerHTML ఎందుకు dangerous?" → HTML parse, script execution (XSS).
- "Prototype pollution ఎలా?" → **proto** ద్వారా Object.prototype modify.
- "Tokens ఎక్కడ store?" → HttpOnly cookies (localStorage XSS-vulnerable).

---

47. Design Patterns in JS (module, singleton, factory, observer/pub-sub, strategy, decorator)

వివరణ

Design patterns అనేవి తరచుగా వచ్చే programming problems కి reusable, proven solutions. వీటిని "recipes" అనుకోవచ్చు – ప్రతిసారీ కొత్తగా solve చేయకుండా, tested approach వాడటం. JavaScript యొక్క flexibility (functions, closures, prototypes) వల్ల patterns చాలా elegant గా implement అవుతాయి.

Real-life Scenario

> **Design patterns = వంట recipes.** ఒక అనుభవజ్ఞుడైన chef ప్రతి డిష్‌కీ కొత్తగా ఆలోచించడు – తనకి తెలిసిన proven recipes (patterns) వాడతాడు. "బిర్యానీ ఇలా చేయాలి" (Singleton – ఒక్కటే instance), "ఏ కూర అయినా ఇదే base gravy" (Factory), "ఆర్డర్ వచ్చాక అందరికీ notify" (Observer). Recipes తెలిస్తే కొత్త problems కూడా త్వరగా, reliable గా solve అవుతాయి.

1. Module Pattern – encapsulation

Private state ని closure తో దాచి, public API మాత్రమే expose చేయడం. Modern ES modules కి ముందు encapsulation కి వాడేవారు.

```
const counter = (function () {
  let count = 0;                    // private (closure)
  return {                          // public API
    increment() { count++; return count; },
    decrement() { count--; return count; },
    get value() { return count; }
  };
})();

counter.increment(); // 1
counter.increment(); // 2
console.log(counter.value); // 2
// console.log(counter.count); // undefined – private!
```

2. Singleton – ఒక్కటే instance

ఒక class కి ఒకే ఒక్క instance ఉండేలా, global access point ఇవ్వడం. Config, DB connection, cache, logger కి.

```
class Database {
  static #instance = null;
```

```
  constructor() {
    if (Database.#instance) return Database.#instance; // ఉంటే అదే
    this.connection = "connected";
    Database.#instance = this;
  }
  static getInstance() {
    if (!Database.#instance) Database.#instance = new Database();
    return Database.#instance;
  }
}

const db1 = new Database();
const db2 = new Database();
console.log(db1 === db2); // true — ఒక్కటే instance

// modern JS లో: module singleton (module ఒక్కసారే evaluate)
// export const db = new Database();
```

3. Factory — object creation abstraction

Object creation logic ని ఒక function లో centralize చేసి, type బట్టి తగిన object ఇవ్వడం. `new` calls scatter అవ్వకుండా.

```
class Dog { speak() { return "Woof"; } }
class Cat { speak() { return "Meow"; } }

function animalFactory(type) {
  switch (type) {
    case "dog": return new Dog();
    case "cat": return new Cat();
    default: throw new Error(`తెలియని type: ${type}`);
  }
}

const pet = animalFactory("dog");
console.log(pet.speak()); // "Woof"
// creation logic ఒక్క చోట — కొత్త type జోడించడం సులభం
```

4. Observer / Pub-Sub — event notification

ఒక object (subject) state మారితే, subscribed observers అందరికీ automatic notify. Event systems, reactive UIs కి foundation.

```
class EventEmitter {
  #listeners = {};
  on(event, callback) {
    (this.#listeners[event] ??= []).push(callback);
    return () => this.off(event, callback); // unsubscribe function
  }
  off(event, callback) {
    this.#listeners[event] = (this.#listeners[event] || [])
      .filter(cb => cb !== callback);
  }
  emit(event, data) {
    (this.#listeners[event] || []).forEach(cb => cb(data));
```

```
  }
}

const bus = new EventEmitter();
const unsub = bus.on("login", user => console.log(`${user} logged in`));
bus.emit("login", "Ravi"); // "Ravi logged in"
unsub(); // subscribe తీసేయడం
```

Observer vs Pub-Sub తేడా: Observer లో subject observers ని నేరుగా తెలుసు; Pub-Sub లో ఒక central broker (event bus) మధ్యలో – publishers/subscribers ఒకరినొకరు తెలియదు (loose coupling).

5. Strategy – interchangeable algorithms

ఒకే పని చేయడానికి multiple algorithms, runtime లో switch చేయగలిగేలా. `if/else` chains బదులు.

```
const paymentStrategies = {
  creditCard: (amount) => `Credit card: ₹${amount}`,
  upi: (amount) => `UPI: ₹${amount}`,
  wallet: (amount) => `Wallet: ₹${amount}`
};

function checkout(amount, method) {
  const strategy = paymentStrategies[method];
  if (!strategy) throw new Error("Invalid method");
  return strategy(amount);
}

console.log(checkout(500, "upi"));        // "UPI: ₹500"
console.log(checkout(1000, "creditCard")); // "Credit card: ₹1000"
// కొత్త payment method = కొత్త strategy జోడించడమే (existing code మార్చకుండా)
```

6. Decorator – behavior జోడించడం

Existing object/function ని modify చేయకుండా, కొత్త behavior wrap చేయడం. Logging, caching, timing జోడించడానికి.

```
// function decorator – logging జోడించడం
function withLogging(fn) {
  return function (...args) {
    console.log(`Calling ${fn.name}(${args})`);
    const result = fn.apply(this, args);
    console.log(`Result: ${result}`);
    return result;
  };
}

function add(a, b) { return a + b; }
const loggedAdd = withLogging(add);
loggedAdd(2, 3);
// "Calling add(2,3)" → "Result: 5" → returns 5

// decorators compose అవుతాయి (withCache(withLogging(fn)))
```

Patterns comparison table

Pattern	ఉద్దేశ్యం	ఉదాహరణ
Module	encapsulation (private state)	counter, utility libs
Singleton	ఒక్కటే instance	config, DB, logger
Factory	creation abstraction	UI components, parsers
Observer/Pub-Sub	event notification	event bus, reactivity
Strategy	interchangeable algorithms	payment, sorting, validation
Decorator	behavior wrapping	logging, caching, auth

Gotchas (సాధారణ తప్పులు)

- **Over-engineering** – simple problems కి patterns అనవసరం; patterns tool, goal కాదు.
- **Singleton = global state** – testing కష్టం, hidden dependencies; జాగ్రత్త.
- **Observer memory leaks** – unsubscribe మర్చిపోతే listeners pile up.
- **Modern JS often simpler:** ES modules (module pattern), closures (encapsulation) – classic patterns తక్కువ అవసరం.
- Pattern name కంటే problem-solving ముఖ్యం.

Key Points

- Design patterns = common problems కి proven solutions.
- Module (private state), Singleton (ఒక్కటే instance), Factory (creation).
- Observer/Pub-Sub (event notification), Strategy (algorithms), Decorator (wrapping).
- Over-engineering avoid; patterns tools, రూల్స్ కాదు.

Interview దృష్టి

- "Singleton use case + risk?" → config/DB; global state, testing కష్టం.
- "Observer vs Pub-Sub?" → direct coupling vs central broker (loose).
- "Strategy pattern ఎప్పుడు?" → multiple interchangeable algorithms, if/else తగ్గించడం.
- "JS లో module pattern ఎలా?" → IIFE + closure (private state).

---

48. Tricky Parts & Gotchas (hoisting, this, closures-in-loops, coercion, floating point, NaN, async order)

వివరణ

JavaScript లో కొన్ని "wat?" moments – unexpected behaviors, ఇవి interviews లో, debugging లో తరచుగా వస్తాయి. వీటిని లోతుగా అర్థం చేసుకుంటే, senior engineer గా bugs ముందే గుర్తించగలరు. ప్రతి

gotcha వెనుక ఒక logical reason ఉంది – అది తెలిస్తే మర్చిపోరు.

Real-life Scenario

> **JavaScript యొక్క gotchas = ఒక తెలివైన కానీ literal-minded స్నేహితుడు.** మీరు చెప్పింది సరిగ్గా, అక్షరాలా చేస్తాడు – కానీ మీ intention కి భిన్నంగా! "0.1 + 0.2 కలుపు" అంటే `0.30000000000000004` ఇస్తాడు (binary floating point – literal గా correct). "ఈ number ని string తో కలుపు" అంటే concatenate చేస్తాడు (`1 + "2" = "12"`). అతని logic తెలిస్తే, సరిగ్గా communicate చేయగలరు.

1. Hoisting

Declarations compile time లో పైకి "hoist" అవుతాయి (Topic 37 creation phase).

```
console.log(a); // undefined (var hoisted, initialized to undefined)
var a = 5;

console.log(b); // ❌ ReferenceError (let TDZ లో)
let b = 5;

foo();          // "hi" — function declaration fully hoisted
function foo() { console.log("hi"); }

bar();          // ❌ TypeError: bar is not a function
var bar = () => {}; // var hoisted (undefined), assignment కాదు
```

Declaration	Hoisted?	Initial value
`var`	✅	`undefined`
`let` / `const`	✅ (కానీ TDZ)	uninitialized (access = error)
function declaration	✅	పూర్తి function
function expression	var rules	undefined

2. `this` – 5 binding rules

`this` value **ఎలా call చేసారో** దాన్నిబట్టి (ఎక్కడ define చేసారో కాదు – arrow తప్ప).

```
// 1. Default — standalone call → global/undefined (strict)
function f() { console.log(this); } // window / undefined (strict)

// 2. Implicit — object method → ఆ object
const obj = { name: "Ravi", greet() { return this.name; } };
obj.greet(); // "Ravi" (this = obj)

// 3. Explicit — call/apply/bind
function greet() { return this.name; }
greet.call({ name: "Priya" });  // "Priya"
greet.apply({ name: "Priya" }); // "Priya"
```

```
const bound = greet.bind({ name: "Kiran" }); bound(); // "Kiran"

// 4. new — constructor → కొత్త object
function Person(n) { this.name = n; }
new Person("Ravi"); // this = కొత్త instance

// 5. Arrow — lexical this (enclosing scope, తనదైన this లేదు)
const o = {
  name: "Ravi",
  regular() { return (function () { return this?.name; })(); }, // undefined
  arrow() { return (() => this.name)(); }                      // "Ravi"
};
```

సాధారణ this bug:

```
const obj = {
  name: "Ravi",
  greetLater() {
    setTimeout(function () { console.log(this.name); }, 100); // undefined!
    setTimeout(() => console.log(this.name), 100);            // "Ravi" (arrow)
  }
};
```

3. Closures in loops — classic

```
// ❌ var — అన్నీ 3 print (var function-scoped, ఒకే i shared)
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 3, 3, 3
}

// ✅ let — 0, 1, 2 (let block-scoped, ప్రతి iteration కి కొత్త i)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 0, 1, 2
}

// ✅ var fix — IIFE తో closure capture
for (var i = 0; i < 3; i++) {
  ((j) => setTimeout(() => console.log(j), 100))(i); // 0, 1, 2
}
```

కారణం: `var` function-scoped కాబట్టి అన్ని callbacks ఒకే `i` ని reference చేస్తాయి; loop అయ్యేసరికి `i=3`. `let` ప్రతి iteration కి కొత్త binding create చేస్తుంది.

4. Type coercion

```
1 + "2"       // "12"  (number → string, concatenation)
"5" - 2       // 3     (string → number, subtraction)
"5" * "2"     // 10    (both → number)
true + 1      // 2     (true → 1)
[] + []       // ""    (both → "")
[] + {}       // "[object Object]"
[1,2] + [3]   // "1,23" (arrays → strings)
null + 1      // 1     (null → 0)
```

```
undefined + 1 // NaN  (undefined → NaN)

// == vs === (coercion)
0 == ""      // true  (both → 0/falsy)
0 == "0"     // true
"" == "0"    // false
null == undefined // true
null == 0    // false (special rule)
NaN == NaN   // false!
```

నియమం: ఎప్పుడూ `===` వాడండి (coercion లేదు). Coercion కావాలంటే explicit (`Number(x)`, `String(x)`).

5. Floating point

```
0.1 + 0.2          // 0.30000000000000004 (binary can't represent 0.1 exactly)
0.1 + 0.2 === 0.3  // false! 😱

// ✅ Fix — epsilon comparison
Math.abs((0.1 + 0.2) - 0.3) < Number.EPSILON; // true
// లేదా — money కి integers (cents/paise) వాడండి
(0.1 * 10 + 0.2 * 10) / 10; // 0.3
```

కారణం: IEEE 754 double-precision binary లో 0.1, 0.2 exactly represent కావు (1/3 decimal లా).

6. NaN quirks

```
typeof NaN          // "number" (!)
NaN === NaN         // false (NaN ఏదితోనూ equal కాదు, తనతో సహా)
Number.isNaN(NaN)   // true (సరైన check)
isNaN("hello")      // true (coerces — buggy)
Number.isNaN("hello") // false (no coercion — correct)
[NaN].includes(NaN) // true (SameValueZero — indexOf కంటే మెరుగు)
[NaN].indexOf(NaN)  // -1 (=== వాడుతుంది, కనుక్కోలేదు)
```

7. Async execution order

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1, 4, 3, 2
// 1,4 sync → 3 microtask → 2 macrotask (Topic 27)
```

ఇతర gotchas

```
typeof null         // "object" (historic bug)
typeof []           // "object" (Array.isArray వాడండి)
typeof function(){} // "function"
0.1.toFixed(0)      // "0"
```

```
parseInt("08")      // 8 (radix ఇవ్వండి: parseInt("08", 10))
[1,2,3].sort()      // OK
[10,1,2].sort()     // [1,10,2]! (default lexicographic — string sort)
[10,1,2].sort((a,b) => a-b) // [1,2,10] (సరైనది)
"b" + "a" + + "a" + "a" // "baNaNa" (+ "a" → NaN)
```

Key Points

- Hoisting: var undefined, let/const TDZ, function declarations full.
- `this` : call విధానం బట్టి (default/implicit/explicit/new/arrow lexical).
- Closures in loops: var shared bug, let per-iteration.
- Coercion: `+` concatenates, `===` వాడండి.
- Floating point: epsilon compare, money integers.
- NaN: `Number.isNaN` ; array sort default lexicographic.

Interview దృష్టి

- "for loop var vs let closures?" → var 3,3,3 (shared), let 0,1,2.
- "0.1 + 0.2 === 0.3?" → false, IEEE 754.
- "typeof null?" → "object" (bug); NaN === NaN → false.
- "arrow function this?" → lexical (enclosing scope).
- "== vs ===?" → coercion vs strict; ఎప్పుడూ ===.

---

49. Must-know Polyfills (bind, debounce, throttle, deepClone, Promise.all, curry, flatten, EventEmitter)

వివరణ

Polyfill అంటే ఒక feature/method ని scratch నుండి implement చేయడం — పాత browsers లో లేని దాన్ని జోడించడానికి, లేదా interviews లో internal understanding చూపడానికి. ఈ implementations ప్రతి senior JS engineer కి తప్పనిసరి — "మీరు `bind` ని ఎలా implement చేస్తారు?" అనేది classic interview question. కింద ఇవన్నీ **real, working** implementations.

Real-life Scenario

> **Polyfill = ఒక built-in tool ని మీరే తయారు చేయడం.** Market లో ready-made screwdriver (`Array.flat`) ఉంది కానీ, మీకు అది ఎలా పనిచేస్తుందో తెలియాలంటే మీరే ఒకటి తయారు చేసి చూడాలి. ఒకసారి తయారు చేస్తే — దాని internals శాశ్వతంగా అర్థమవుతాయి. Interview లో "ఈ tool ఎలా పనిచేస్తుంది?" అంటే మీరు దాన్ని కళ్ళ ముందు build చేయగలరు.

1. Function.prototype.bind

`this` ని fix చేసి, కొత్త function return చేస్తుంది (partial application తో).

```
Function.prototype.myBind = function (context, ...boundArgs) {
  const fn = this; // original function
  return function (...callArgs) {
    // new తో call అయితే original this, లేకపోతే context
    return fn.apply(this instanceof fn ? this : context,
                    [...boundArgs, ...callArgs]);
  };
};

function greet(greeting, name) { return `${greeting}, ${name}! I'm ${this.role}`; }
const bound = greet.myBind({ role: "dev" }, "Hi");
console.log(bound("Ravi")); // "Hi, Ravi! I'm dev"
```

2. Debounce

Events ఆగిపోయాక ఒక్కసారే run (Topic 45).

```
function debounce(fn, delay) {
  let timer;
  function debounced(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  }
  debounced.cancel = () => clearTimeout(timer); // bonus: cancel
  return debounced;
}

const log = debounce((x) => console.log("run:", x), 300);
log(1); log(2); log(3); // 300ms తర్వాత "run: 3" ఒక్కసారే
```

3. Throttle

ప్రతి interval కి ఒకసారి మాత్రమే (Topic 45).

```
function throttle(fn, limit) {
  let lastCall = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastCall >= limit) {
      lastCall = now;
      return fn.apply(this, args);
    }
  };
}

const onScroll = throttle(() => console.log("scroll", Date.now()), 200);
// ఎంత fast పిలిచినా ప్రతి 200ms కి ఒకసారే
```

4. Deep Clone

Nested objects/arrays ని fully copy (references share అవ్వకుండా).

```
function deepClone(obj, seen = new WeakMap()) {
  // primitives & null → అలాగే
  if (obj === null || typeof obj !== "object") return obj;
  // circular reference handle
  if (seen.has(obj)) return seen.get(obj);
  // Date, RegExp special cases
  if (obj instanceof Date) return new Date(obj);
  if (obj instanceof RegExp) return new RegExp(obj.source, obj.flags);

  const clone = Array.isArray(obj) ? [] : {};
  seen.set(obj, clone);
  for (const key of Reflect.ownKeys(obj)) {
    clone[key] = deepClone(obj[key], seen); // recursive
  }
  return clone;
}

const orig = { a: 1, b: { c: 2 }, d: [3, 4] };
const copy = deepClone(orig);
copy.b.c = 99;
console.log(orig.b.c); // 2 (unaffected — deep copy)
// Modern: structuredClone(obj) built-in (functions తప్ప)
```

5. Promise.all

అన్నీ fulfill అయితే values array; ఒకటి reject అయితే వెంటనే reject (Topic 29).

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = [];
    let completed = 0;
    if (promises.length === 0) return resolve([]);
    promises.forEach((p, i) => {
      Promise.resolve(p) // non-promise values కూడా handle
        .then(value => {
          results[i] = value;        // order preserve (index)
          if (++completed === promises.length) resolve(results);
        })
        .catch(reject);              // ఒకటి fail → వెంటనే reject
    });
  });
}

promiseAll([Promise.resolve(1), Promise.resolve(2), 3])
  .then(r => console.log(r)); // [1, 2, 3]
```

6. Curry

`f(a, b, c)` ని `f(a)(b)(c)` గా — args పూర్తయ్యేదాకా collect చేస్తుంది.

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {     // అన్ని args వచ్చాయా?
      return fn.apply(this, args);
    }
```

```
    return (...next) => curried.apply(this, [...args, ...next]); // మిగతా collect
  };
}

const sum = (a, b, c) => a + b + c;
const curried = curry(sum);
console.log(curried(1)(2)(3));   // 6
console.log(curried(1, 2)(3));   // 6
console.log(curried(1)(2, 3));   // 6 (flexible)
```

7. Flatten (Array.flat)

Nested arrays ని single level కి (depth తో).

```
function flatten(arr, depth = 1) {
  return depth > 0
    ? arr.reduce((acc, val) =>
        acc.concat(Array.isArray(val) ? flatten(val, depth - 1) : val), [])
    : arr.slice();
}
console.log(flatten([1, [2, [3, [4]]]]));           // [1, 2, [3, [4]]] (depth 1)
console.log(flatten([1, [2, [3, [4]]]], Infinity)); // [1, 2, 3, 4] (fully)

// iterative (stack — deep nesting కి safe, no recursion limit)
function flattenDeep(arr) {
  const stack = [...arr], result = [];
  while (stack.length) {
    const next = stack.pop();
    Array.isArray(next) ? stack.push(...next) : result.unshift(next);
  }
  return result;
}
```

8. EventEmitter

Pub-sub pattern (Topic 41, 47).

```
class EventEmitter {
  #events = {};
  on(event, listener) {
    (this.#events[event] ??= []).push(listener);
    return this; // chaining
  }
  once(event, listener) {
    const wrapper = (...args) => {
      listener(...args);
      this.off(event, wrapper);
    };
    return this.on(event, wrapper);
  }
  off(event, listener) {
    if (this.#events[event])
      this.#events[event] = this.#events[event].filter(l => l !== listener);
    return this;
  }
  emit(event, ...args) {
```

```
    (this.#events[event] || []).slice().forEach(l => l(...args));
    return this;
  }
}

const ee = new EventEmitter();
ee.on("data", d => console.log("got:", d));
ee.emit("data", 42); // "got: 42"
```

Bonus – quick polyfills

```
// Array.prototype.map
Array.prototype.myMap = function (cb, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++)
    if (i in this) result[i] = cb.call(thisArg, this[i], i, this);
  return result;
};

// call
Function.prototype.myCall = function (context, ...args) {
  context = context || globalThis;
  const key = Symbol();
  context[key] = this;
  const result = context[key](...args);
  delete context[key];
  return result;
};
```

Gotchas (సాధారణ తప్పులు)

- **bind + new** – bound function ని `new` తో వాడితే `this` original instance (context కాదు).
- **deepClone circular references** – WeakMap లేకపోతే infinite loop.
- **Promise.all order** – index వాడాలి (completion order కాదు).
- **curry `fn.length`** – default/rest params తో inaccurate.
- **flatten recursion limit** – చాలా deep అయితే stack overflow; iterative వాడండి.
- **EventEmitter emit లో `.slice()`** – listener తనని తాను remove చేస్తే iteration break అవ్వకుండా.

Key Points

- Polyfills – internal understanding + old browser support.
- bind = closure + apply (`this` fix); debounce/throttle = timer patterns.
- deepClone = recursion + WeakMap (circular); Promise.all = counter + index.
- curry = `fn.length` check; flatten = recursion/stack; EventEmitter = listener map.

Interview దృష్టి

- ఇవన్నీ classic interview questions – code నుండి రాయగలగాలి.
- "bind ఎలా?" → closure captures this + args, apply.
- "debounce vs throttle implement?" → clearTimeout reset vs time check.

- "deepClone edge cases?" → circular (WeakMap), Date/RegExp, functions.
- "curry logic?" → args.length >= fn.length అయ్యేదాకా collect.

---

50. Interview Q&A (rapid-fire senior questions + crisp answers)

వివరణ

ఇది మొత్తం guide యొక్క condensed revision – SDE2/SSE interviews లో అడిగే rapid-fire questions, crisp answers తో. ఒక్కో answer ని 1-2 lines లో confident గా చెప్పగలగాలి. Topic references తో deep dive కి వెళ్ళవచ్చు.

Real-life Scenario

> **Interview = ఒక cricket net practice లో fast bowling.** Rapid-fire questions = వరుసగా వచ్చే bouncers. మీరు ఒక్కో దానికి crisp గా, confident గా react అవ్వాలి – తడబడకూడదు. ఈ Q&A ను వరుసగా చదివి, ఎవరైనా అడిగినట్టు, గట్టిగా సమాధానం చెప్పే practice చేయండి. Concepts తెలిస్తే bouncers కూడా boundary కి కొట్టవచ్చు.

Core language

Q: `var`, `let`, `const` తేడా? A: `var` function-scoped, hoisted (undefined), redeclarable. `let` block-scoped, TDZ, reassignable. `const` block-scoped, TDZ, reassign కుదరదు (కానీ object mutate అవుతుంది). (Topic 37, 48)

Q: `==` vs `===`? A: `==` type coercion చేసి compare; `===` type + value strict compare (coercion లేదు). ఎప్పుడూ `===`. (Topic 48)

Q: Closure అంటే? A: ఒక function దాని outer scope variables ని "గుర్తుంచుకొని" access చేయగలగడం, outer function return అయ్యాకా కూడా. Private state, currying, memoization కి. (Topic 49)

Q: Hoisting? A: Declarations compile time (creation phase) లో scope పైకి move. var → undefined; let/const → TDZ; function declarations → full. (Topic 37, 48)

Q: `this` ఎలా determine అవుతుంది? A: Call విధానం బట్టి – default (global/undefined), implicit (object method), explicit (call/apply/bind), new (instance), arrow (lexical). (Topic 48)

Async

Q: Event loop ఎలా పనిచేస్తుంది? A: Call stack empty అయ్యాక – అన్ని microtasks (promises) drain, తర్వాత ఒక macrotask (setTimeout), మధ్యలో rendering. Repeat. (Topic 27)

Q: Microtask vs macrotask? A: Microtask (promises, queueMicrotask) higher priority – ప్రతి macrotask తర్వాత అన్నీ drain. Macrotask (setTimeout, I/O) ఒక్కొక్కటిగా. (Topic 27)

Q: `setTimeout(fn, 0)` నిజంగా 0ms? A: కాదు – minimum ~4ms clamp + stack empty అయ్యేదాకా wait. Promises దాని కంటే ముందు run. (Topic 27)

Q: Promise.all vs allSettled vs race vs any? A: all (అన్నీ/fail-fast), allSettled (అన్నీ report), race (మొదటి settle), any (మొదటి fulfill). (Topic 29)

Q: async/await internally? A: Generators + promises పైన syntactic sugar. `await` continuation ని microtask గా schedule చేస్తుంది. (Topic 30, 31)

Q: Sequential vs parallel async? A: Independent operations → `Promise.all` (parallel, fast). Dependent → sequential await. `forEach` లో await పనిచేయదు. (Topic 30)

Objects & prototypes

Q: Prototypal inheritance? A: Objects `[[Prototype]]` link ద్వారా properties inherit చేస్తాయి. Property లేకపోతే prototype chain పైకి వెతుకుతుంది, null దాకా. (Topic – Part 1)

Q: `Object.freeze` deep? A: కాదు – shallow. Nested objects mutate అవుతాయి. Deep freeze కి recursive. (Topic – Part 1)

Q: Prototype vs `__proto__` vs `Object.getPrototypeOf`? A: `prototype` = function property (instances కి); `__proto__` = instance యొక్క prototype reference (legacy); `getPrototypeOf` = standard accessor. (Topic – Part 1)

DOM & Browser

Q: Event delegation? A: Parent కి ఒక్క listener, bubbling ద్వారా children events catch. Memory efficient, dynamic elements auto-handle. (Topic 39)

Q: `textContent` vs `innerHTML`? A: textContent = plain text (XSS-safe); innerHTML = HTML parse (XSS risk). User data కి textContent. (Topic 38, 46)

Q: localStorage vs sessionStorage vs cookies? A: localStorage (persistent, ~5MB, client). sessionStorage (tab lifetime). cookies (~4KB, server కి పంపబడతాయి). (Topic 40)

Q: Debounce vs throttle? A: Debounce – events ఆగాక ఒక్కసారి (search box). Throttle – rate limit, ప్రతి N ms (scroll). (Topic 45)

Advanced

Q: JS memory ఎలా manage అవుతుంది? A: Automatic GC, mark-and-sweep, reachability from roots. Circular references handle అవుతాయి. (Topic 36)

Q: Common memory leaks? A: Forgotten timers, event listeners, detached DOM nodes, closures holding big data, unbounded caches. (Topic 36)

Q: Proxy ఎప్పుడు? A: Meta-programming – reactivity (Vue 3), validation, logging, API wrappers. Traps: get/set/has/deleteProperty. (Topic 35)

Q: CJS vs ESM? A: CJS synchronous, dynamic, copies (`require`). ESM static, async, live bindings, tree-shakeable (`import`). (Topic 32)

Q: Generators ఉపయోగం? A: Lazy evaluation, infinite sequences, custom iterators, async flow control. `function*` , `yield` pause/resume. (Topic 31)

Security & performance

Q: XSS prevention? A: textContent (innerHTML కాదు), sanitize (DOMPurify), CSP, HttpOnly cookies, output encoding. (Topic 46)

Q: CSRF vs XSS? A: CSRF – authenticated session misuse (tokens, SameSite defense). XSS – malicious script injection. (Topic 46)

Q: Big-O – array.includes vs Set.has? A: array.includes O(n), Set/Map.has O(1). పెద్ద lookups కి Set/Map. (Topic 45)

Q: Reflow vs repaint? A: Reflow – layout recalculate (ఖరీదైనది). Repaint – visual redraw. Batch DOM ops, class toggle. (Topic 45)

Tricky (rapid-fire)

```
[1,2,3] + [4,5,6]           // "1,2,34,5,6"
typeof NaN                  // "number"
0.1 + 0.2 === 0.3           // false
[] == ![]                   // true (both → 0)
"5" - - "2"                 // 7
[10, 1, 3].sort()           // [1, 10, 3] (lexicographic)
(function(){ return typeof arguments; })() // "object"
null ?? "x"                 // "x"
0 ?? "x"                    // 0
[1,2,3].map(parseInt)       // [1, NaN, NaN] (index radix!)
```

Q: `[1,2,3].map(parseInt)` ఎందుకు `[1, NaN, NaN]` ? A: `map` (value, index) పంపుతుంది → `parseInt("1",0)=1` , `parseInt("2",1)=NaN` , `parseInt("3",2)=NaN` (radix issue). (Topic 48)

System-design-ish (senior)

Q: పెద్ద list (10k rows) render performance? A: Virtualization (visible items మాత్రమే), pagination, lazy loading, DocumentFragment. (Topic 45)

Q: API calls optimize ఎలా? A: Caching (memoization), debounce, `Promise.all` (parallel), pagination, AbortController (cancel). (Topic 29, 45)

Q: Heavy computation UI freeze అవ్వకుండా? A: Web Worker (background thread), chunking (requestIdleCallback/setTimeout). (Topic 40)

Key Points

- Core: var/let/const, ===, closures, hoisting, this – foundational.

- Async: event loop, microtask/macrotask, Promise combinators, async/await.
- Advanced: GC, Proxy, modules, generators.
- Practical: XSS/CSRF, Big-O, debounce/throttle, performance.
- Tricky outputs practice చేయండి – confidence కీలకం.

Interview దృష్టి

- Answers crisp, confident గా (1-2 lines) చెప్పండి; అవసరమైతే code తో.
- "ఎందుకు" అని అడిగితే internals (event loop, prototype chain, coercion rules) explain చేయండి.
- Trade-offs discuss చేయండి – senior signal (ఒక్క answer కాదు, options + reasoning).

---

51. Memory Tips + Common Mistakes cheat-sheet

వివరణ

ఇది మొత్తం guide ని గుర్తుంచుకోడానికి **mnemonics, analogies, quick-reference cheat-sheets**. "ఒకసారి చదివితే మర్చిపోకూడదు" అనేది ఈ topic ముఖ్య లక్ష్యం. ప్రతి concept కి ఒక memory hook + అత్యంత సాధారణ mistakes జాబితా.

Real-life Scenario

> **మెదడు = ఒక library.** సమాచారం ఉంచడం సులభం, కానీ సరైన సమయంలో తీయడం (recall) కష్టం. Mnemonics, analogies = ఆ library లో "సైన్‌బోర్డులు, colored labels" – అవసరమైనప్పుడు సరైన shelf కి వెంటనే తీసుకెళ్తాయి. "Event loop = restaurant waiter", "closure = backpack of variables" – ఇలాంటి vivid images concepts ని permanent memory లోకి మారుస్తాయి.

Memory hooks – ప్రతి concept కి ఒక image

Concept	Memory Hook (analogy)
Event loop	Restaurant waiter – VIP (microtask) ముందు, normal (macrotask) తర్వాత
Closure	Backpack – function తన variables ని వెంట తీసుకెళ్తుంది
Promise	Pizza token – pending → ready/refund
Hoisting	Movie trailer – declarations ముందే చూపిస్తారు
`this`	"ఎవరు call చేసారు?" – call site నిర్ణయిస్తుంది (arrow తప్ప)
Prototype chain	Family tree – property లేకపోతే పెద్దవాళ్ళని అడగడం
Debounce	Lift door – అందరూ ఎక్కేదాకా ఆగి close

Concept	Memory Hook (analogy)
Throttle	Metro – ప్రతి 5 min కి ఒకటి
GC	Librarian – ఎవరూ reference లేని పుస్తకం తీసేయడం
Proxy	Secretary – ప్రతి request intercept
Generator	Netflix next episode – lazy, on-demand
Currying	Vending machine – ఒక్కో coin, పూర్తయ్యాక product

Mnemonics

- **Promise states:** "**P**ending → **F**ulfilled/**R**ejected" – "**PFR**".
- `??` **vs** `||` **:** `??` = "**N**ull/**u**ndefined only" (Nu); `||` = "all falsy".
- **Event phases:** "**C**apture **T**arget **B**ubble" – "**CTB**" (పైనుండి కిందకి, తర్వాత కిందనుండి పైకి).
- **Falsy values (8):** `false, 0, -0, 0n, "", null, undefined, NaN` – మిగతావన్నీ truthy (`[]` , `{}` కూడా!).
- **HTTP-safe (idempotent):** GET, PUT, DELETE; NOT POST.

Falsy values – గుర్తుంచుకోండి (8 మాత్రమే)

```
// ఇవి 8 మాత్రమే falsy – మిగతావన్నీ truthy
Boolean(false)      // false
Boolean(0)          // false
Boolean(-0)         // false
Boolean(0n)         // false (BigInt zero)
Boolean("")         // false (empty string)
Boolean(null)       // false
Boolean(undefined)  // false
Boolean(NaN)        // false

// ⚠️ ఇవి TRUTHY (సాధారణ తప్పు!)
Boolean([])         // true (empty array!)
Boolean({})         // true (empty object!)
Boolean("0")        // true (non-empty string)
Boolean("false")    // true
Boolean(function(){}) // true
```

Top common mistakes cheat-sheet

#	తప్పు	సరైనది
1	`==` వాడటం	`===` (coercion లేదు)
2	`var` loops + closures	`let` (block scope)
3	`forEach` లో `await`	`for...of` / `Promise.all`
4	Sequential independent awaits	`Promise.all` (parallel)

#	తప్పు	సరైనది
5	`innerHTML` user data	`textContent` (XSS)
6	`fetch` `res.ok` check మర్చిపోవడం	`if (!res.ok) throw`
7	Missing `.catch` / try-catch	ఎప్పుడూ handle
8	Closure loop `var i` → అన్నీ చివరి value	`let` / IIFE
9	`Object.freeze` deep అనుకోవడం	shallow – recursive freeze
10	`0.1+0.2===0.3`	epsilon compare
11	`Array(3).fill` shared reference (objects)	`Array.from({length:3}, () => ({}))`
12	`sort()` numbers	`sort((a,b) => a-b)`
13	`setInterval` clear మర్చిపోవడం	`clearInterval` (leak)
14	Arrow function `this` object method	regular function
15	Mutating props/state directly	immutable copies (`{...obj}`)

Quick decision guides

ఏ loop వాడాలి?

```
Array iterate + value → for...of
Index అవసరం → for / entries()
Transform → map    Filter → filter    Sum → reduce
Object keys → Object.keys/entries + for...of
Async sequential → for...of + await
Async parallel → Promise.all(map)
```

ఏ equality?

```
సాధారణం → === (strict)
NaN check → Number.isNaN(x)
Object/array deep → JSON / lodash.isEqual / manual
null-ish → x == null (null + undefined రెండూ)
```

Copy ఎలా?

```
Shallow → {...obj} / [...arr] / Object.assign
Deep → structuredClone(obj) / deepClone
Array → slice / spread / Array.from
```

Modern JS quick wins (ఇవి వాడండి)

```
// Optional chaining + nullish
const city = user?.address?.city ?? "Unknown";
// Destructuring + defaults
const { name = "Guest", ...rest } = user;
// Spread merge
const updated = { ...state, count: state.count + 1 };
// Array methods (loops కంటే readable)
const active = users.filter(u => u.active).map(u => u.name);
// Short-circuit
isLoggedIn && renderDashboard();
// Immutable update
const newArr = arr.with(0, 99); // arr మార్చకుండా
```

30-second mental model of JS

```
JS = single-threaded + event loop (async concurrency)
  ├─ Values: primitives (stack) + objects (heap, references)
  ├─ Scope: lexical, closures remember outer variables
  ├─ this: call site (arrow = lexical)
  ├─ Objects: prototype chain inheritance
  ├─ Async: callbacks → promises → async/await
  │         microtasks > macrotasks
  ├─ Memory: automatic GC (mark-and-sweep, reachability)
  └─ Modules: ESM (static, tree-shakeable)
```

Final Key Points (మొత్తం guide సారాంశం)

- **JavaScript** single-threaded, event-loop driven; async concurrency కి microtask/macrotask.
- **Values** primitives (copy) vs objects (reference); **scope** lexical + closures.
- `this` call site నిర్ణయిస్తుంది; **prototype chain** inheritance.
- **Async** journey: callbacks → promises → async/await; parallel కి `Promise.all`.
- **Memory** automatic GC; leaks (timers, listeners, closures) జాగ్రత్త.
- **Modern JS** (ES6+): destructuring, `?.`, `??`, spread, immutable methods.
- **Quality:** `===`, error handling, security (XSS/CSRF), performance (debounce, Big-O), testing.

Interview దృష్టి – చివరి మాట

- Concepts ని **analogies తో** గుర్తుంచుకోండి – recall వేగంగా.
- **"ఎందుకు"** ఎప్పుడూ తెలుసుకోండి – internals senior signal.
- **Trade-offs** discuss చేయండి – ఒక్క answer కాదు, reasoning.
- Common mistakes cheat-sheet ని ఇంటర్వ్యూ ముందు ఒక్కసారి చూడండి.
- **అభ్యాసమే** master key – ఈ guide ని పదేపదే revise చేయండి. All the best! 🚀